Appendix F: ResNet50V2 Tests 2

In [1]:
```
!nvidia-smi
```

```
Fri Mar 14 14:30:17 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 550.54.15              Driver Version: 550.54.15      CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA A100-SXM4-40GB          Off |   00000000:00:04.0 Off |                    0 |
| N/A   31C    P0             41W /  400W |       0MiB /  40960MiB |      0%      Default |
|                                         |                        |             Disabled |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                              GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

In [2]:
```
#!pip install tensorflow
import tensorflow as tf
print(tf.config.list_physical_devices('GPU'))
```

```
[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

Import libraries

In [3]:
```
import tensorflow as tf # Access to models, datasets and training
from tensorflow.keras.datasets import cifar100 # Access to CIFAR-100
from tensorflow.keras.applications import ResNet50 # Access to pre-trained model
from tensorflow.keras import layers, models, optimizers # Access to building blo
```

```
from tensorflow import keras # Access to stuff for model
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLRO
from sklearn.model_selection import train_test_split # To help split dataset up
from sklearn.metrics import confusion_matrix, classification_report # Analysis o
import numpy as np # Manipulate data
import pandas as pd # Statistical analysis of data
import seaborn as sns
import matplotlib.pyplot as plt # Plot data
```

With 224x224

In [4]:
```
for repeat_2_times in range(2):
    #### <<<<<<<<<<Load and process data>>>>>>>>>>
    # Load CIFAR-100 dataset
    (X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

    # Split (8000) of training data into temporary set
    X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_s
    print(f"X_temp.shape: {X_temp.shape}\n")

    # Split temp data into equal validation (4000) and testing (4000) data
    X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp,
    print(f"X_temp_val.shape: {X_temp_val.shape}")
    print(f"y_temp_val.shape: {y_temp_val.shape}")
    print(f"X_temp_test.shape: {X_temp_test.shape}")
    print(f"y_temp_test.shape: {y_temp_test.shape}\n")

    # Split test data into validation (5000) and testing (5000)
    X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.

    # Add temp_val to validation (9000) and temp_test to testing (9000) to get a
    X_val = np.concatenate((X_val, X_temp_val), axis=0)
    y_val = np.concatenate((y_val, y_temp_val), axis=0)
    X_test = np.concatenate((X_test, X_temp_test), axis=0)
    y_test = np.concatenate((y_test, y_temp_test), axis=0)

    print(f"X_train.shape: {X_train.shape}")
    print(f"y_train.shape: {y_train.shape}")
    print(f"X_val.shape: {X_val.shape}")
    print(f"y_val.shape: {y_val.shape}")
    print(f"X_test.shape: {X_test.shape}")
    print(f"y_test.shape: {y_test.shape}\n")

    def display_imgs(imgs, labels):
        plt.subplots(figsize=(10,10))
        for i in range(16):
            plt.subplot(4, 4, i+1)
            k = np.random.randint(0, imgs.shape[0])
            if i == 0:
                print(f"labels[{k}].shape: {labels[k].shape}")
                print(f"imgs[{k}].shape: {imgs[k].shape}")
            plt.imshow(imgs[k])
            #plt.title(labels[k])
            plt.axis('off')
        plt.tight_layout()
        plt.show()

    display_imgs(X_train, y_train)
```

```
# Normalise images (scale to range [0, 1]) - Improves convergence speed & ac
X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
display_imgs(X_train, y_train)

labels_names = ['beaver','dolphin','otter','seal','whale','aquarium fish','f
                'orchids','poppies','roses','sunflowers','tulips','bottles','
                'apples','mushrooms','oranges','pears','sweet peppers','clock
                'telephone','television','bed','chair','couch','table','wardr
                'caterpillar','cockroach','bear','leopard','lion','tiger','wo
                'road','skyscraper','cloud','forest','mountain','plain','sea'
                'elephant','kangaroo','fox','porcupine','possum','raccoon','s
                'spider','worm','baby','boy','girl','man','woman','crocodile'
                'turtle','hamster','mouse','rabbit','shrew','squirrel','maple
                'bicycle','bus','motorcycle','pickup truck','train','lawn-mow
                'tractor']

def class_distrib(y, labels_names, dataset_name):
    counts = pd.DataFrame(data=y).value_counts().sort_index()
    #print(f"counts:\n{counts}")
    fig, ax = plt.subplots(figsize=(20,10))
    ax.bar(labels_names, counts)
    ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
    plt.title(f"Distribution of '{dataset_name}' Dataset", fontsize=25)
    plt.grid()
    plt.tight_layout()
    plt.show()
class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64
train_dataset = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                 .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                    tf.squeeze(tf.one_hot(y, depth=100, dtyp
                 .batch(batch_size)
                 .prefetch(tf.data.experimental.AUTOTUNE))

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                  tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
                .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtyp
                .batch(batch_size)
                .prefetch(tf.data.experimental.AUTOTUNE))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}")  # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}")  # Should be (batch_size, 10)
    del img,lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
```

```
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}")  # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}")  # Should be (batch_size, 10)
    del img,lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}")  # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}")  # Should be (batch_size, 10)
    del img,lbl


#### <<<<<<<<<<Pre-trained model>>>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

# Freeze the layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.4),
    layers.Dense(100, activation='softmax')  # CIFAR-100 has 100 classes
])

for sample in test_dataset.take(1):
    print(type(sample))  # Should be <class 'tuple'>
    print(len(sample))  # Should be 2
    print(type(sample[0]), type(sample[1]))  # Both should be <class 'tensor
    print(sample[0].shape)  # Should be (batch_size, 224, 224, 3)
    print(sample[1].shape)  # Should be (batch_size, 100)
print(f"Model input shape: {model.input_shape}")
print(f"Model output shape: {model.output_shape}")
sample = next(iter(test_dataset.as_numpy_iterator()))
print(len(sample))  # Should be 2
print(type(sample[0]), type(sample[1]))  # Both should be <class 'numpy.ndar
print(sample[0].shape, sample[1].shape)  # Should match model input and outp
print("\n")
#for x, y in test_dataset.take(1):
#    print(type(x), type(y))  # Both should be <class 'tensorflow.Tensor'>
#for x_batch, y_batch in test_dataset.take(1):
#    test_loss, test_acc = model.evaluate(x_batch, y_batch)
#    print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

# Compile the model
#tensorboard_callback = keras.callbacks.TensorBoard(log_dir="./logs")
model.compile(optimizer=optimizers.Adam(learning_rate=1e-3),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>
early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
                               patience=5, # Num. epochs with no improvement
                               restore_best_weights=True)
#reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
```

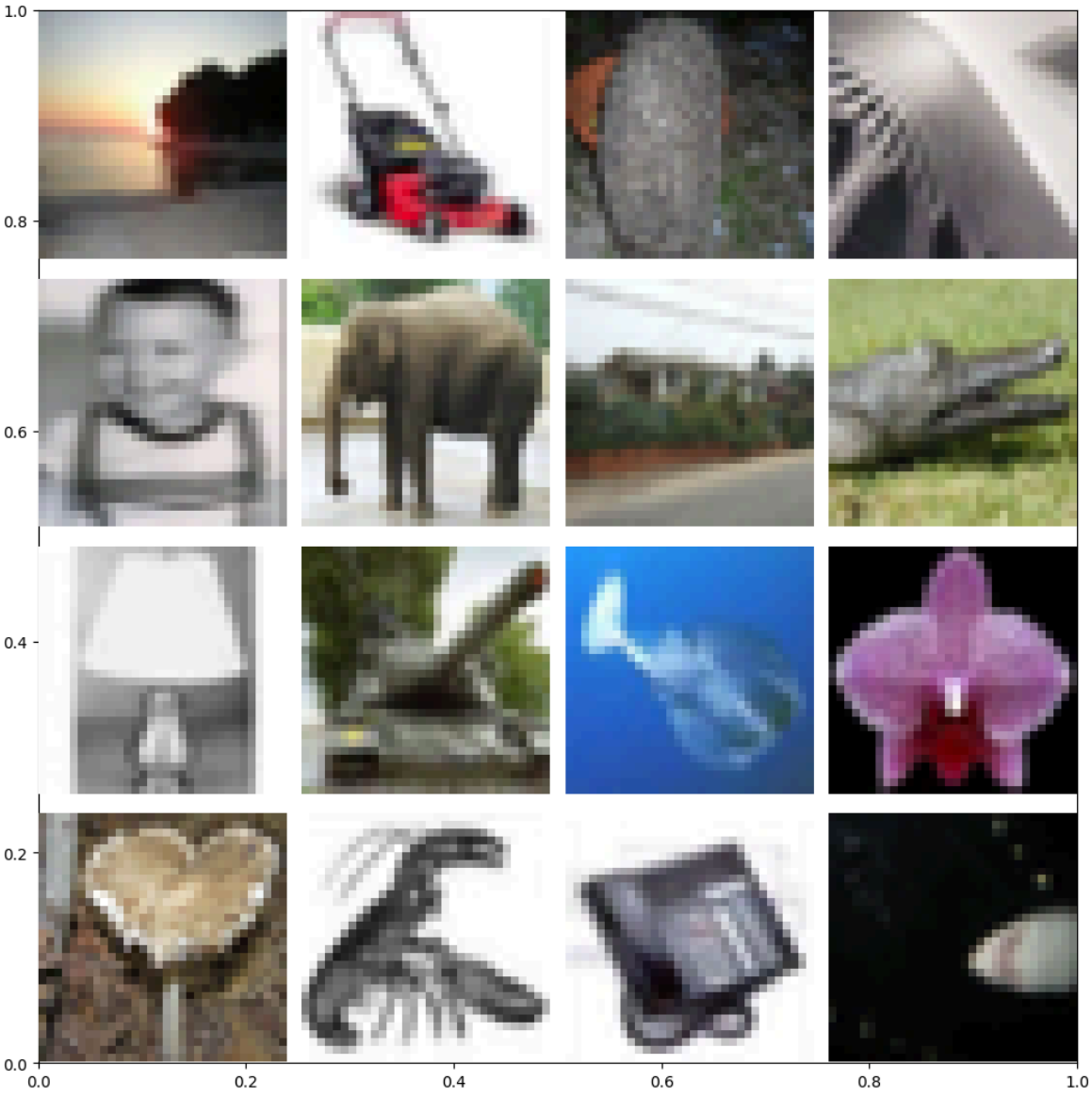
```
#                               factor=0.1, # Reduce lr by a factor
#                               patience=3, # Num epochs w/ no improvement
#                               min_lr=1e-6, # Min lr
#                               verbose=1)
#tensorboard = TensorBoard(log_dir='./logs', # Logs directory
#                          histogram_freq=1, # Logs histograms for weights/ac
#                          write_graph=True, # Logs graph of model
#                          write_images=True) # Log images like weight histog
#checkpoint = ModelCheckpoint('best_model.h5',
#                             monitor='val_loss', # or val_accuracy
#                             save_best_only=True, # Save only best model
#                             mode='min', # min for loss or max for accuracy
#                             verbose=1)
#cvs_logger = CSVLogger('training_log.csv', seperator=',', append=True) # Sa

# Train the model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
                    batch_size=batch_size, callbacks=[early_stopping], verbo

#### <<<<<<<<<<Plot Training & Validation Error>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1,len(history.history['loss'])+1)
train_loss = history.history['loss']
val_loss = history.history['val_loss']
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

def plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc):
    # Plot Training and Validation Loss
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.plot(epochs, train_loss, label='Training Loss')
    plt.plot(epochs, val_loss, label='Validation Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Training and Validation Loss')
    plt.legend()
    plt.grid()

    # Plot Training and Validation Accuracy
    plt.subplot(1, 2, 2)
    plt.plot(epochs, train_acc, label='Training Accuracy')
    plt.plot(epochs, val_acc, label='Validation Accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.title('Training and Validation Accuracy')
    plt.legend()
    plt.grid()

    plt.tight_layout()
    plt.show()

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>

# Evaluate on test data
results = model.evaluate(test_dataset)
test_loss = results[0]
```

```
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss*100:.2f}%")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<<<Generate Confusion Matrix>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<<<Fine-Tune>>>>>>>>>>
#### <<<<<<<<<<Adapt Model>>>>>>>>>>
# Unfreeze last 10 layers
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ lower learning rate (prevents destroying learned features
model.compile(optimizer=optimizers.Adam(learning_rate=1e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<<<Modify Dataset>>>>>>>>>>
def augment_dataset(x, y):
    x = tf.image.resize(x, (224, 224))  # Resize images
    x = tf.image.random_flip_left_right(x)  # Random horizontal flip
    x = tf.image.random_brightness(x, max_delta=0.2)  # Adjust brightness
    x = tf.image.random_contrast(x, lower=0.8, upper=1.2)  # Adjust contrast
    y = tf.squeeze(tf.one_hot(y, depth=100, dtype=tf.float32))  # One-hot en
    return x, y
```

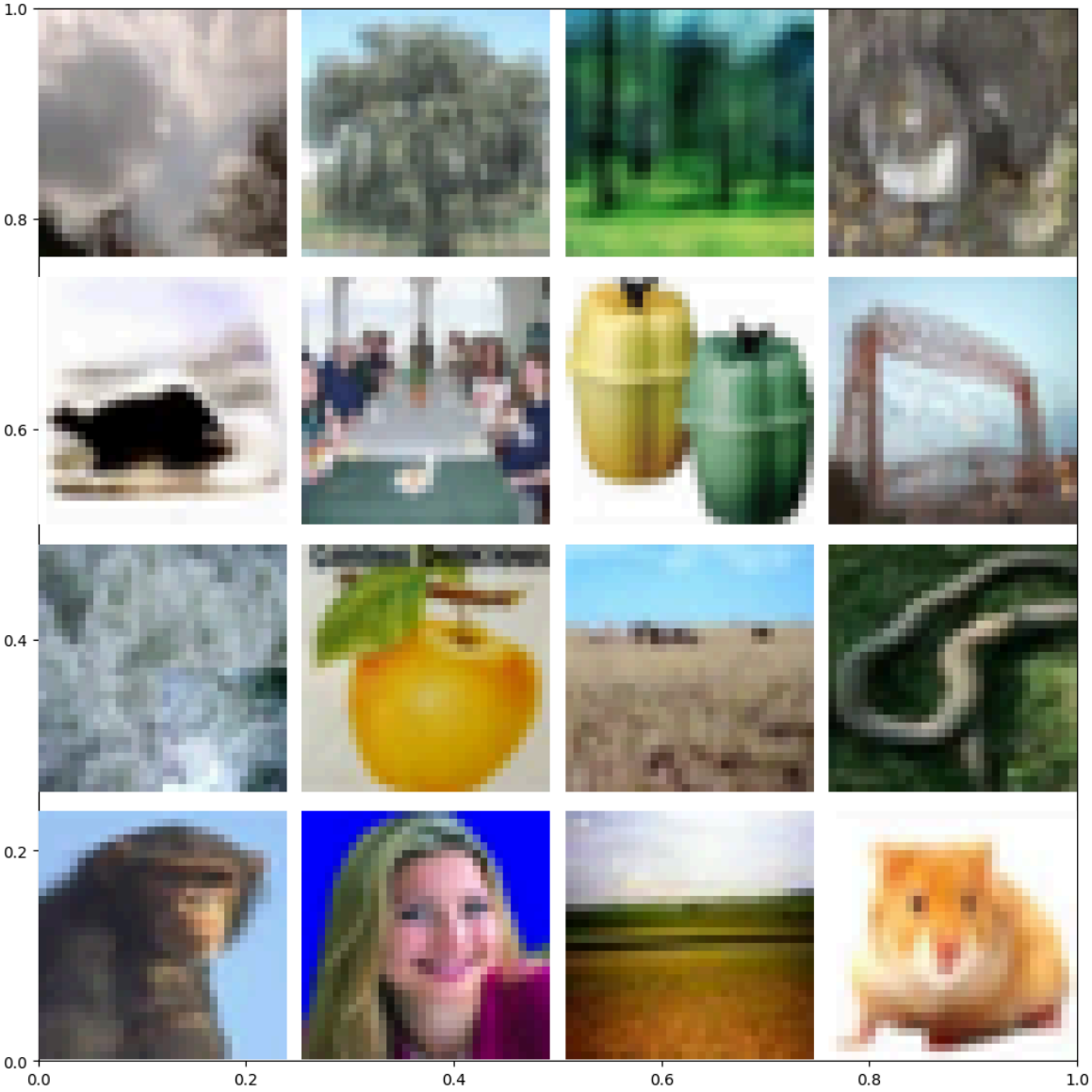
```
train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                .map(augment_dataset, num_parallel_calls=tf.data.experiment
                .batch(batch_size)
                .prefetch(tf.data.experimental.AUTOTUNE))
# Not val or test as augment train helps generalise better, but want to prov

#### <<<<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>

# Train the model
history_fine_tune = model.fit(train_dataset, validation_data=val_dataset, ep
                              batch_size=batch_size, callbacks=[early_stoppi

#### <<<<<<<<<<Plot Training & Validation Error>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1,len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>

# Evaluate on test data
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss*100:.2f}%")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<<<Generate Confusion Matrix>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
```

```
    hist = pd.DataFrame(history_fine_tune.history)
    hist['epoch'] = history_fine_tune.epoch

    # Finally, display the hist DataFrame.
    hist
```

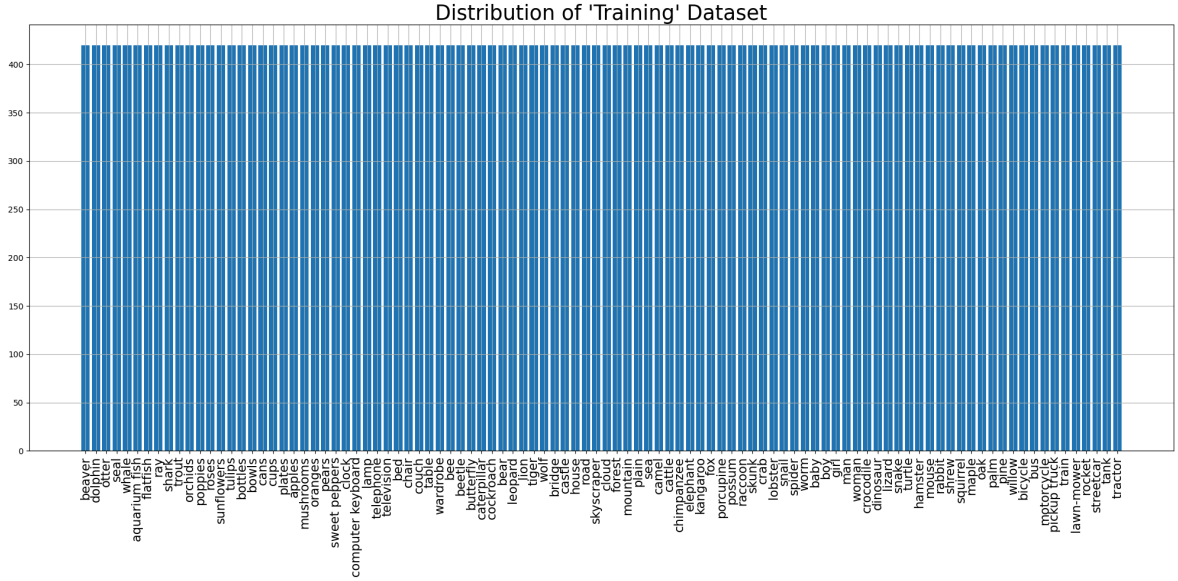
```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-100-python.tar.gz
169001437/169001437 ━━━━━━━━━━━━━━━━━━━━ 13s 0us/step
X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)
y_temp_val.shape: (4000, 1)
X_temp_test.shape: (4000, 32, 32, 3)
y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)
y_train.shape: (42000, 1)
X_val.shape: (9000, 32, 32, 3)
y_val.shape: (9000, 1)
X_test.shape: (9000, 32, 32, 3)
y_test.shape: (9000, 1)

labels[4270].shape: (1,)
imgs[4270].shape: (32, 32, 3)
```



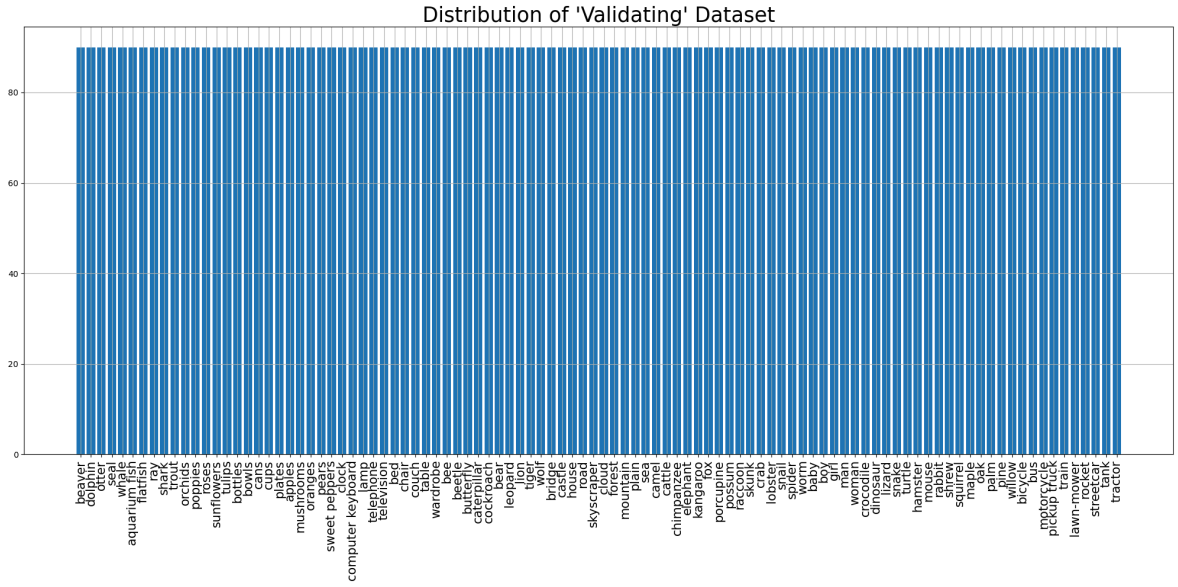

```
labels[2306].shape: (1,)
imgs[2306].shape: (32, 32, 3)
```



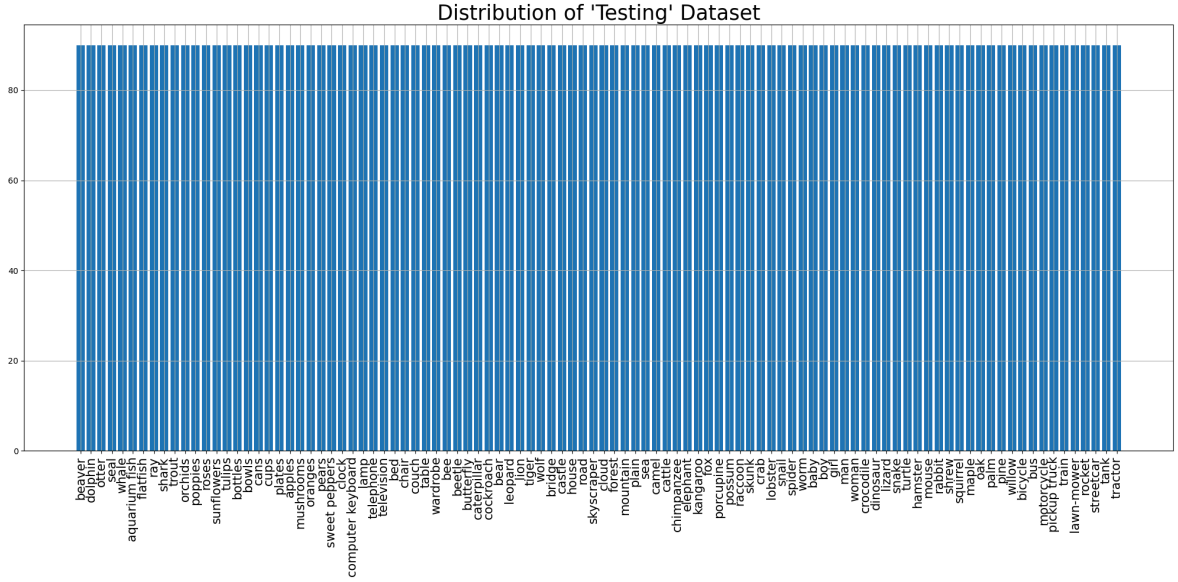
```
<ipython-input-4-b65b705d857f>:70: UserWarning: set_ticklabels() should only be u
sed with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```




```
<ipython-input-4-b65b705d857f>:70: UserWarning: set_ticklabels() should only be u
sed with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```




```
<ipython-input-4-b65b705d857f>:70: UserWarning: set_ticklabels() should only be u
sed with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
Training dataset:
 <_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.f
loat32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)

Validation dataset:
 <_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.f
loat32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)

Testing dataset:
 <_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.f
loat32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
Downloading data from https://storage.googleapis.com/tensorflow/keras-application
s/resnet/resnet50v2_weights_tf_dim_ordering_tf_kernels_notop.h5
94668760/94668760 ━━━━━━━━━━━━━━━━━━━━ 5s 0us/step
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.f
ramework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential"

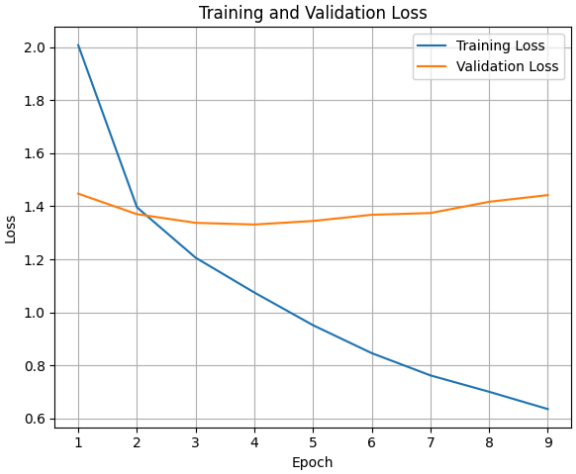
Layer (type)	Output Shape
resnet50v2 (Functional)	(None, 7, 7, 2048)
global_average_pooling2d (GlobalAveragePooling2D)	(None, 2048)
dense (Dense)	(None, 512)
dropout (Dropout)	(None, 512)
dense_1 (Dense)	(None, 100)

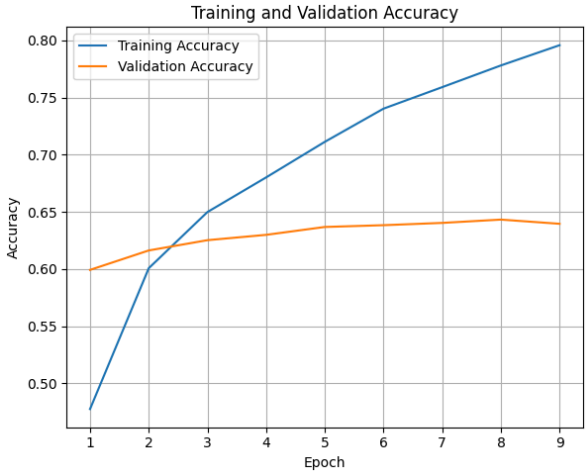
Total params: 24,665,188 (94.09 MB)

Trainable params: 1,100,388 (4.20 MB)

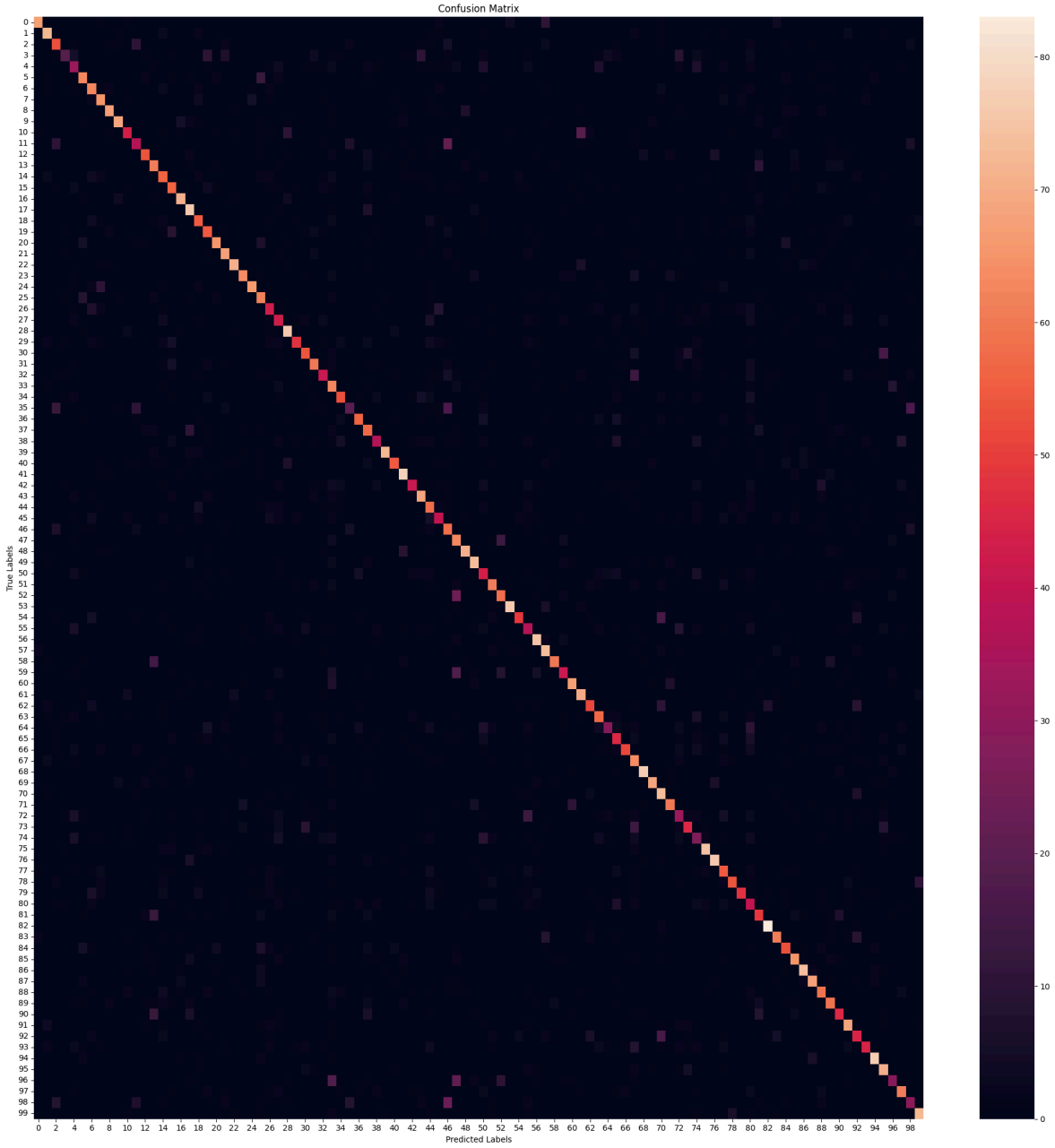
Non-trainable params: 23,564,800 (89.89 MB)

```
Epoch 1/25
657/657 ━━━━━━━━━━━━━━━━━━━━ 57s 58ms/step - accuracy: 0.3832 - f1_score: 0.3787
- loss: 2.5389 - precision: 0.6965 - val_accuracy: 0.5993 - val_f1_score: 0.5958
- val_loss: 1.4477 - val_precision: 0.8044
Epoch 2/25
657/657 ━━━━━━━━━━━━━━━━━━━━ 19s 29ms/step - accuracy: 0.5885 - f1_score: 0.5855
- loss: 1.4474 - precision: 0.7761 - val_accuracy: 0.6163 - val_f1_score: 0.6133
- val_loss: 1.3704 - val_precision: 0.7904
Epoch 3/25
657/657 ━━━━━━━━━━━━━━━━━━━━ 19s 28ms/step - accuracy: 0.6402 - f1_score: 0.6376
- loss: 1.2373 - precision: 0.7996 - val_accuracy: 0.6253 - val_f1_score: 0.6222
- val_loss: 1.3377 - val_precision: 0.7861
Epoch 4/25
657/657 ━━━━━━━━━━━━━━━━━━━━ 19s 28ms/step - accuracy: 0.6735 - f1_score: 0.6706
- loss: 1.1020 - precision: 0.8112 - val_accuracy: 0.6300 - val_f1_score: 0.6300
- val_loss: 1.3314 - val_precision: 0.7725
Epoch 5/25
657/657 ━━━━━━━━━━━━━━━━━━━━ 18s 28ms/step - accuracy: 0.7061 - f1_score: 0.7033
- loss: 0.9789 - precision: 0.8295 - val_accuracy: 0.6369 - val_f1_score: 0.6356
- val_loss: 1.3448 - val_precision: 0.7645
Epoch 6/25
657/657 ━━━━━━━━━━━━━━━━━━━━ 18s 28ms/step - accuracy: 0.7338 - f1_score: 0.7319
- loss: 0.8746 - precision: 0.8419 - val_accuracy: 0.6384 - val_f1_score: 0.6389
- val_loss: 1.3681 - val_precision: 0.7575
Epoch 7/25
657/657 ━━━━━━━━━━━━━━━━━━━━ 19s 28ms/step - accuracy: 0.7535 - f1_score: 0.7518
- loss: 0.7825 - precision: 0.8489 - val_accuracy: 0.6404 - val_f1_score: 0.6399
- val_loss: 1.3747 - val_precision: 0.7514
Epoch 8/25
657/657 ━━━━━━━━━━━━━━━━━━━━ 18s 28ms/step - accuracy: 0.7743 - f1_score: 0.7727
- loss: 0.7176 - precision: 0.8582 - val_accuracy: 0.6433 - val_f1_score: 0.6412
- val_loss: 1.4171 - val_precision: 0.7449
Epoch 9/25
657/657 ━━━━━━━━━━━━━━━━━━━━ 18s 28ms/step - accuracy: 0.7867 - f1_score: 0.7851
- loss: 0.6601 - precision: 0.8663 - val_accuracy: 0.6397 - val_f1_score: 0.6395
- val_loss: 1.4424 - val_precision: 0.7447
```





```
141/141 ━━━━━━━━━━━━━━━━━━━━ 3s 23ms/step - accuracy: 0.6244 - f1_score: 0.6183 -
loss: 1.3475 - precision: 0.7676
Test Accuracy: 63.57%
Test Loss: 131.81%
Test Precision: 77.76%
Test F1 Scores (Per Class): [82.20858  77.83783  55.49738  31.666664 36.66666  6
7.37968  62.06896
 68.42105  80.473366 81.17647  56.774185 44.57831  67.92453  59.512184
 56.852787 59.893044 79.775276 70.69766  59.139782 57.894737 77.8443
 73.11827  82.08092  76.190475 79.99999  59.04761  48.863632 44.79166
 77.15736  64.86486  61.363636 69.36415  51.89873  47.368416 64.24241
 30.303024 69.135796 56.15763  51.034473 86.22754  69.62025  79.59184
 56.55172  71.50259  52.017933 50.95541  47.736626 52.282154 81.60919
 78.49462  44.2211   68.53932  64.088394 87.86127  64.90066  40.217384
 77.7202   71.56862  75.94936  53.947365 77.01149  71.06599  63.354034
 61.956512 42.647057 48.93617  67.105255 52.892555 86.03351  82.142845
 63.203453 71.428566 36.871502 58.59872  29.999996 80.64516  83.9779
 55.555553 67.92453  64.       34.893616 53.551907 89.72973  70.11494
 59.886997 71.03825  82.22221  77.27273  61.7801   64.480865 57.499992
 68.99999  48.913036 59.999996 85.08286  67.94258  41.428566 67.40331
 37.735844 77.83783 ]
Average Test F1 Scores:63.60

282/282 ━━━━━━━━━━━━━━━━━━━━ 18s 38ms/step
```


Confusion Matrix
True Labels
Predicted Labels
0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52 54 56 58 60 62 64 66 68 70 72 74 76 78 80 82 84 86 88 90 92 94 96 98
0 10 20 30 40 50 60 70 80

	precision	recall	f1-score	support
beaver	0.92	0.74	0.82	90
dolphin	0.76	0.80	0.78	90
otter	0.52	0.59	0.55	90
seal	0.63	0.21	0.32	90
whale	0.37	0.37	0.37	90
aquarium fish	0.65	0.70	0.67	90
flatfish	0.56	0.70	0.62	90
ray	0.65	0.72	0.68	90
shark	0.86	0.76	0.80	90
trout	0.86	0.77	0.81	90
orchids	0.68	0.49	0.57	90
poppies	0.49	0.41	0.45	90
roses	0.78	0.60	0.68	90
sunflowers	0.53	0.68	0.60	90
tulips	0.52	0.62	0.57	90
bottles	0.58	0.62	0.60	90
bowls	0.81	0.79	0.80	90
cans	0.61	0.84	0.71	90
cups	0.57	0.61	0.59	90
plates	0.55	0.61	0.58	90
apples	0.84	0.72	0.78	90
mushrooms	0.71	0.76	0.73	90
oranges	0.86	0.79	0.82	90
pears	0.82	0.71	0.76	90
sweet peppers	0.88	0.73	0.80	90
clock	0.52	0.69	0.59	90
computer keyboard	0.50	0.48	0.49	90
lamp	0.42	0.48	0.45	90
telephone	0.71	0.84	0.77	90
television	0.83	0.53	0.65	90
bed	0.63	0.60	0.61	90
chair	0.72	0.67	0.69	90
couch	0.60	0.46	0.52	90
table	0.36	0.70	0.47	90
wardrobe	0.71	0.59	0.64	90
bee	0.48	0.22	0.30	90
beetle	0.78	0.62	0.69	90
butterfly	0.50	0.63	0.56	90
caterpillar	0.66	0.41	0.51	90
cockroach	0.94	0.80	0.86	90
bear	0.81	0.61	0.70	90
leopard	0.74	0.87	0.80	90
lion	0.75	0.46	0.57	90
tiger	0.67	0.77	0.72	90
wolf	0.44	0.64	0.52	90
bridge	0.59	0.44	0.51	90
castle	0.38	0.64	0.48	90
house	0.42	0.70	0.52	90
road	0.85	0.79	0.82	90
skyscraper	0.76	0.81	0.78	90
cloud	0.40	0.49	0.44	90
forest	0.69	0.68	0.69	90
mountain	0.64	0.64	0.64	90
plain	0.92	0.84	0.88	90
sea	0.80	0.54	0.65	90
camel	0.39	0.41	0.40	90
cattle	0.73	0.83	0.78	90
chimpanzee	0.64	0.81	0.72	90

elephant	0.88	0.67	0.76	90
kangaroo	0.66	0.46	0.54	90
fox	0.80	0.74	0.77	90
porcupine	0.65	0.78	0.71	90
possum	0.72	0.57	0.63	90
raccoon	0.61	0.63	0.62	90
skunk	0.63	0.32	0.43	90
crab	0.47	0.51	0.49	90
lobster	0.82	0.57	0.67	90
snail	0.42	0.71	0.53	90
spider	0.87	0.86	0.86	90
worm	0.88	0.77	0.82	90
baby	0.52	0.81	0.63	90
boy	0.77	0.67	0.71	90
girl	0.37	0.37	0.37	90
man	0.69	0.51	0.59	90
woman	0.30	0.30	0.30	90
crocodile	0.78	0.83	0.81	90
dinosaur	0.84	0.84	0.84	90
lizard	0.51	0.61	0.56	90
snake	0.78	0.60	0.68	90
turtle	0.80	0.53	0.64	90
hamster	0.28	0.44	0.34	90
mouse	0.53	0.54	0.54	90
rabbit	0.87	0.92	0.90	90
shrew	0.73	0.68	0.70	90
squirrel	0.61	0.59	0.60	90
maple	0.70	0.72	0.71	90
oak	0.82	0.82	0.82	90
palm	0.79	0.76	0.77	90
pine	0.58	0.66	0.62	90
willow	0.63	0.66	0.64	90
bicycle	0.66	0.51	0.57	90
bus	0.63	0.77	0.69	90
motorcycle	0.48	0.50	0.49	90
pickup truck	0.75	0.50	0.60	90
train	0.85	0.86	0.85	90
lawn-mower	0.60	0.79	0.68	90
rocket	0.58	0.32	0.41	90
streetcar	0.67	0.68	0.67	90
tank	0.43	0.33	0.38	90
tractor	0.76	0.80	0.78	90
accuracy			0.64	9000
macro avg	0.66	0.64	0.64	9000
weighted avg	0.66	0.64	0.64	9000

Model: "sequential"

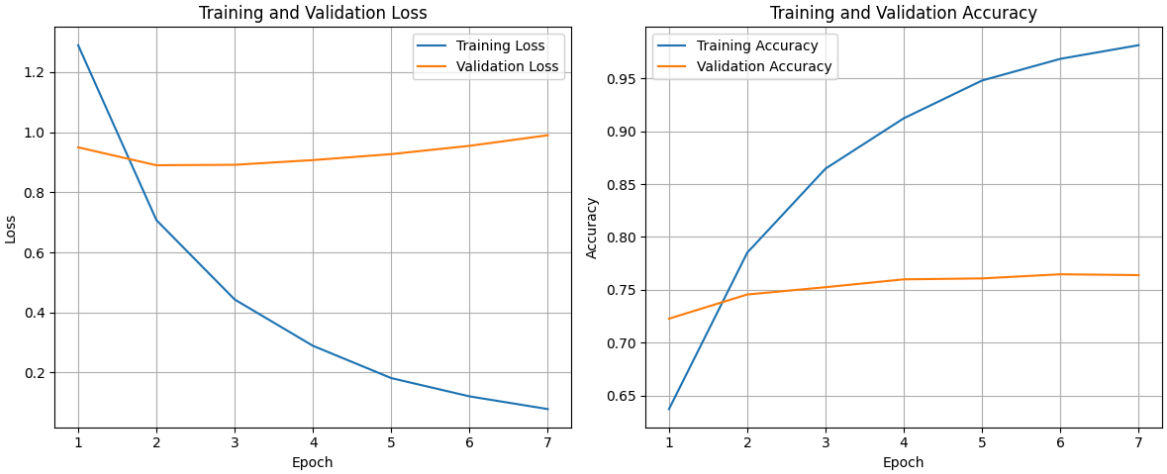
Layer (type)	Output Shape
resnet50v2 (Functional)	(None, 7, 7, 2048)
global_average_pooling2d (GlobalAveragePooling2D)	(None, 2048)
dense (Dense)	(None, 512)
dropout (Dropout)	(None, 512)
dense_1 (Dense)	(None, 100)

Total params: 24,665,188 (94.09 MB)

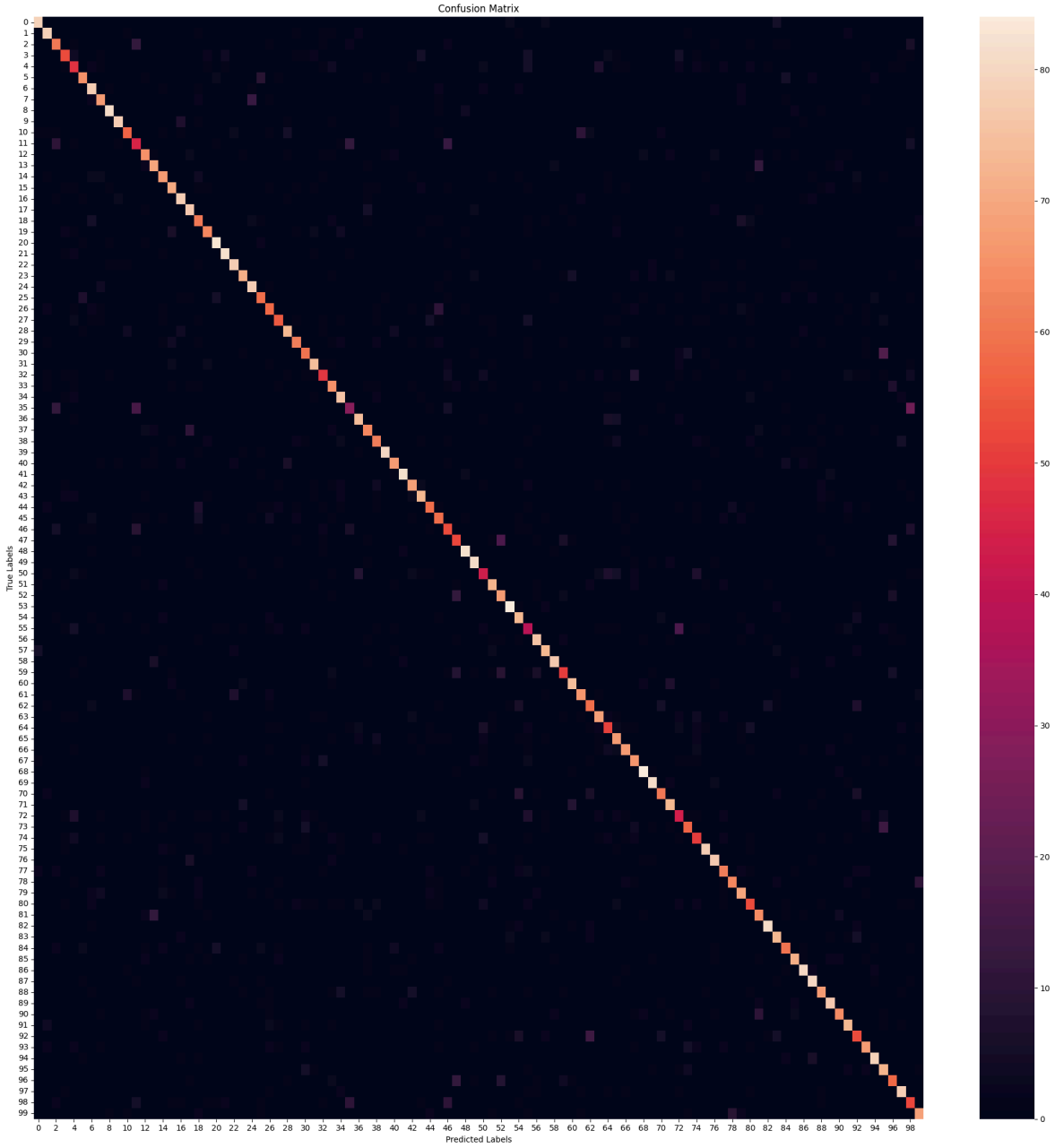
Trainable params: 24,619,748 (93.92 MB)

Non-trainable params: 45,440 (177.50 KB)

```
Epoch 1/15
657/657 ━━━━━━━━━━━━━━━━━━━━ 131s 116ms/step - accuracy: 0.5562 - f1_score: 0.555
6 - loss: 1.6464 - precision: 0.7658 - val_accuracy: 0.7227 - val_f1_score: 0.721
1 - val_loss: 0.9495 - val_precision: 0.8305
Epoch 2/15
657/657 ━━━━━━━━━━━━━━━━━━━━ 47s 72ms/step - accuracy: 0.7582 - f1_score: 0.7557
- loss: 0.8126 - precision: 0.8653 - val_accuracy: 0.7456 - val_f1_score: 0.7441
- val_loss: 0.8899 - val_precision: 0.8323
Epoch 3/15
657/657 ━━━━━━━━━━━━━━━━━━━━ 47s 72ms/step - accuracy: 0.8437 - f1_score: 0.8413
- loss: 0.5118 - precision: 0.9127 - val_accuracy: 0.7524 - val_f1_score: 0.7516
- val_loss: 0.8914 - val_precision: 0.8256
Epoch 4/15
657/657 ━━━━━━━━━━━━━━━━━━━━ 47s 71ms/step - accuracy: 0.8981 - f1_score: 0.8961
- loss: 0.3330 - precision: 0.9419 - val_accuracy: 0.7599 - val_f1_score: 0.7595
- val_loss: 0.9071 - val_precision: 0.8235
Epoch 5/15
657/657 ━━━━━━━━━━━━━━━━━━━━ 47s 71ms/step - accuracy: 0.9397 - f1_score: 0.9381
- loss: 0.2060 - precision: 0.9639 - val_accuracy: 0.7608 - val_f1_score: 0.7607
- val_loss: 0.9269 - val_precision: 0.8165
Epoch 6/15
657/657 ━━━━━━━━━━━━━━━━━━━━ 47s 71ms/step - accuracy: 0.9631 - f1_score: 0.9617
- loss: 0.1361 - precision: 0.9758 - val_accuracy: 0.7647 - val_f1_score: 0.7647
- val_loss: 0.9545 - val_precision: 0.8104
Epoch 7/15
657/657 ━━━━━━━━━━━━━━━━━━━━ 47s 72ms/step - accuracy: 0.9787 - f1_score: 0.9772
- loss: 0.0870 - precision: 0.9859 - val_accuracy: 0.7639 - val_f1_score: 0.7638
- val_loss: 0.9897 - val_precision: 0.8047
```




```
141/141 ━━━━━━━━━━━━━━━━━━━━ 3s 24ms/step - accuracy: 0.7357 - f1_score: 0.7270 -
loss: 0.9217 - precision: 0.8190
Test Accuracy: 74.30%
Test Loss: 90.23%
Test Precision: 82.69%
Test F1 Scores (Per Class): [88.26815  84.04254  64.893616 65.8385   52.459015 7
5.14451  78.78787
 74.444435 91.52542  89.65517  69.51219  49.72375  72.222206 74.999985
 74.03315  76.50272  82.53968  79.38144  62.5641   73.68421  87.83068
 89.61748  85.24589  78.88888  81.675385 69.005844 64.80447  65.47618
 81.56424  74.69879  67.796616 82.22221  60.869564 69.148926 79.166664
 38.410587 80.87432  71.99999  69.66292  92.48554  77.01149  91.11111
 75.55555  79.78141  69.005844 65.19337  56.084652 58.100555 88.17203
 86.17021  51.807224 78.68852  69.791664 91.304344 72.63681  43.33333
 83.060104 81.11111  87.49999  62.1118   81.31868  76.74419  65.55555
 75.55555  58.959538 68.0203   77.456635 71.35134  90.32257  89.01098
 72.18934  79.78141  48.61878  68.263466 57.142853 87.15083  83.24323
 72.51461  74.69879  74.59459  60.919537 68.449196 89.502754 77.65957
 68.965515 80.681816 87.43169  83.76962  77.456635 84.61538  75.73964
 78.68852  59.886997 76.57142  88.26815  69.23076  64.80447  83.87096
 52.525253 75.97765 ]
Average Test F1 Scores:74.17

282/282 ━━━━━━━━━━━━━━━━━━━━ 13s 31ms/step
```

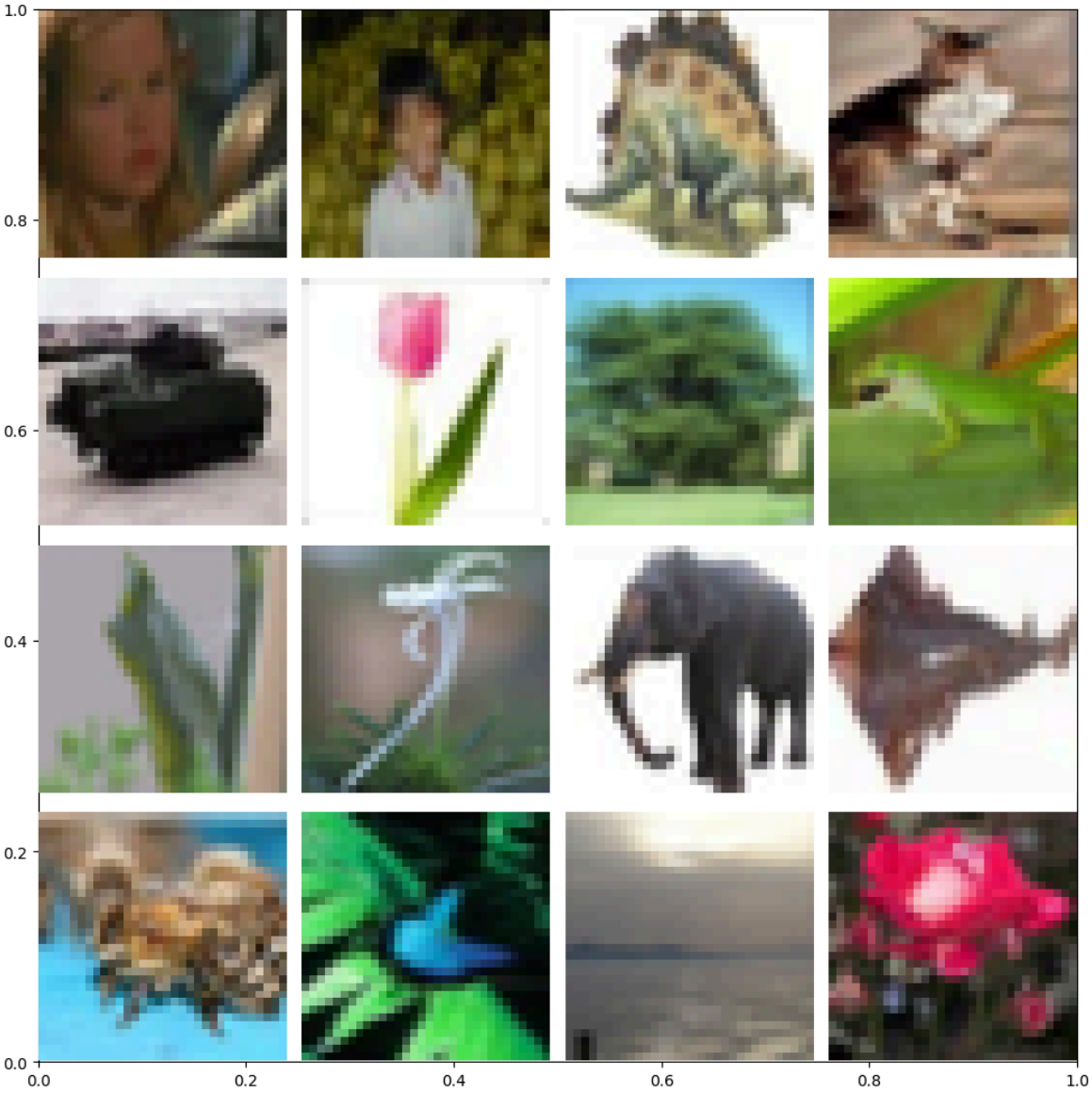

Confusion Matrix
True Labels
Predicted Labels

	precision	recall	f1-score	support
beaver	0.89	0.88	0.88	90
dolphin	0.81	0.88	0.84	90
otter	0.62	0.68	0.65	90
seal	0.75	0.59	0.66	90
whale	0.51	0.53	0.52	90
aquarium fish	0.78	0.72	0.75	90
flatfish	0.72	0.87	0.79	90
ray	0.74	0.74	0.74	90
shark	0.93	0.90	0.92	90
trout	0.93	0.87	0.90	90
orchids	0.77	0.63	0.70	90
poppies	0.49	0.50	0.50	90
roses	0.72	0.72	0.72	90
sunflowers	0.73	0.77	0.75	90
tulips	0.74	0.74	0.74	90
bottles	0.75	0.78	0.77	90
bowls	0.79	0.87	0.83	90
cans	0.74	0.86	0.79	90
cups	0.59	0.68	0.63	90
plates	0.78	0.70	0.74	90
apples	0.84	0.92	0.88	90
mushrooms	0.88	0.91	0.90	90
oranges	0.84	0.87	0.85	90
pears	0.79	0.79	0.79	90
sweet peppers	0.78	0.87	0.82	90
clock	0.73	0.66	0.69	90
computer keyboard	0.65	0.64	0.65	90
lamp	0.71	0.61	0.65	90
telephone	0.82	0.81	0.82	90
television	0.82	0.69	0.75	90
bed	0.69	0.67	0.68	90
chair	0.82	0.82	0.82	90
couch	0.69	0.54	0.61	90
table	0.66	0.72	0.69	90
wardrobe	0.75	0.84	0.79	90
bee	0.48	0.32	0.38	90
beetle	0.80	0.82	0.81	90
butterfly	0.74	0.70	0.72	90
caterpillar	0.70	0.69	0.70	90
cockroach	0.96	0.89	0.92	90
bear	0.80	0.74	0.77	90
leopard	0.91	0.91	0.91	90
lion	0.76	0.76	0.76	90
tiger	0.78	0.81	0.80	90
wolf	0.73	0.66	0.69	90
bridge	0.65	0.66	0.65	90
castle	0.54	0.59	0.56	90
house	0.58	0.58	0.58	90
road	0.85	0.91	0.88	90
skyscraper	0.83	0.90	0.86	90
cloud	0.57	0.48	0.52	90
forest	0.77	0.80	0.79	90
mountain	0.66	0.74	0.70	90
plain	0.89	0.93	0.91	90
sea	0.66	0.81	0.73	90
camel	0.44	0.43	0.44	90
cattle	0.82	0.84	0.83	90
chimpanzee	0.81	0.81	0.81	90

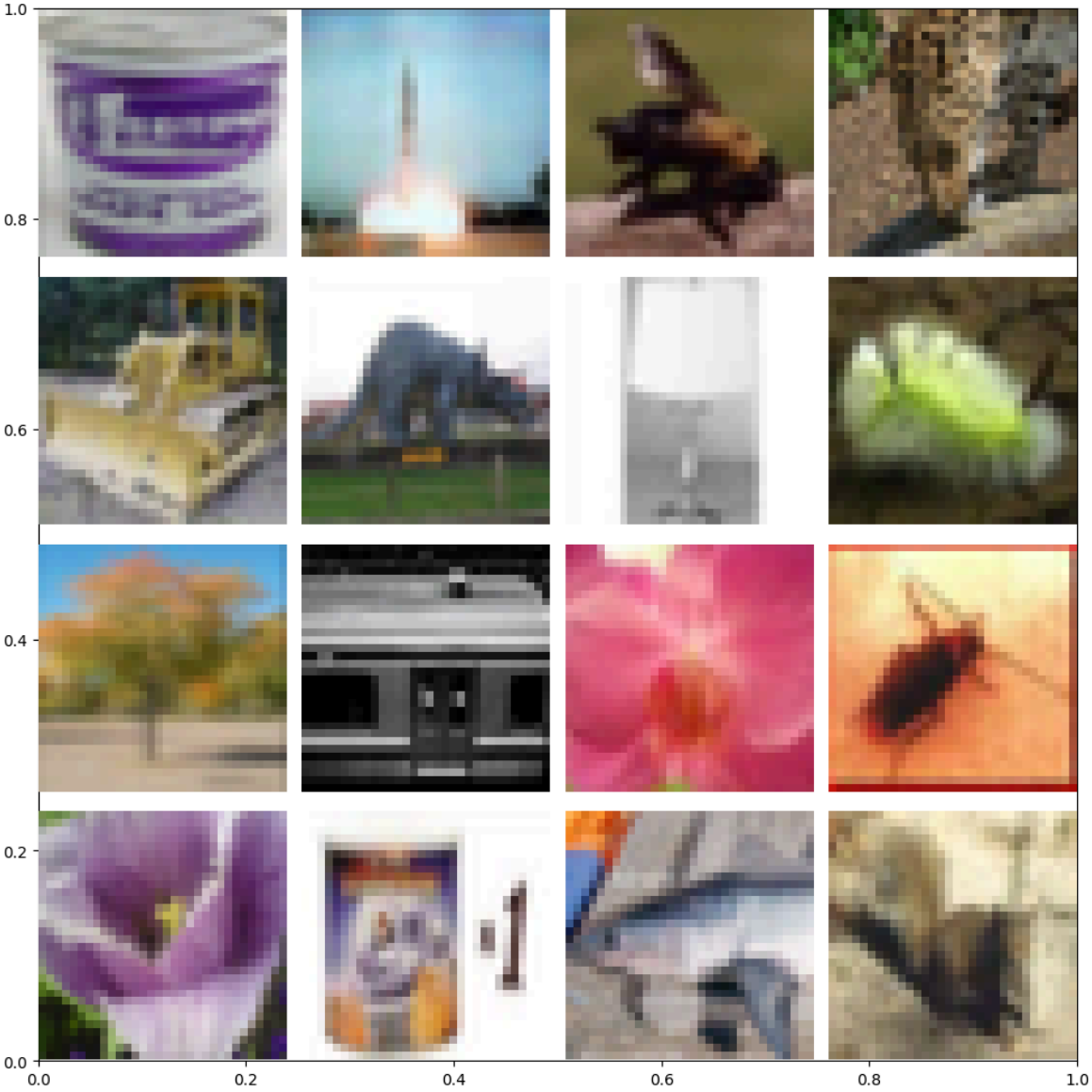
elephant	0.90	0.86	0.88	90
kangaroo	0.70	0.56	0.62	90
fox	0.80	0.82	0.81	90
porcupine	0.80	0.73	0.77	90
possum	0.66	0.66	0.66	90
raccoon	0.76	0.76	0.76	90
skunk	0.61	0.57	0.59	90
crab	0.63	0.74	0.68	90
lobster	0.81	0.74	0.77	90
snail	0.69	0.73	0.71	90
spider	0.88	0.93	0.90	90
worm	0.88	0.90	0.89	90
baby	0.77	0.68	0.72	90
boy	0.78	0.81	0.80	90
girl	0.48	0.49	0.49	90
man	0.74	0.63	0.68	90
woman	0.59	0.56	0.57	90
crocodile	0.88	0.87	0.87	90
dinosaur	0.81	0.86	0.83	90
lizard	0.77	0.69	0.73	90
snake	0.82	0.70	0.75	90
turtle	0.73	0.77	0.75	90
hamster	0.63	0.59	0.61	90
mouse	0.66	0.71	0.68	90
rabbit	0.89	0.90	0.90	90
shrew	0.74	0.81	0.78	90
squirrel	0.71	0.67	0.69	90
maple	0.83	0.79	0.81	90
oak	0.86	0.89	0.87	90
palm	0.79	0.89	0.84	90
pine	0.81	0.74	0.77	90
willow	0.84	0.86	0.85	90
bicycle	0.81	0.71	0.76	90
bus	0.78	0.81	0.79	90
motorcycle	0.61	0.59	0.60	90
pickup truck	0.79	0.74	0.77	90
train	0.89	0.88	0.88	90
lawn-mower	0.61	0.80	0.69	90
rocket	0.65	0.64	0.65	90
streetcar	0.81	0.87	0.84	90
tank	0.48	0.58	0.53	90
tractor	0.76	0.76	0.76	90
accuracy			0.74	9000
macro avg	0.74	0.74	0.74	9000
weighted avg	0.74	0.74	0.74	9000

```
X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)
y_temp_val.shape: (4000, 1)
X_temp_test.shape: (4000, 32, 32, 3)
y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)
y_train.shape: (42000, 1)
X_val.shape: (9000, 32, 32, 3)
y_val.shape: (9000, 1)
X_test.shape: (9000, 32, 32, 3)
y_test.shape: (9000, 1)
```

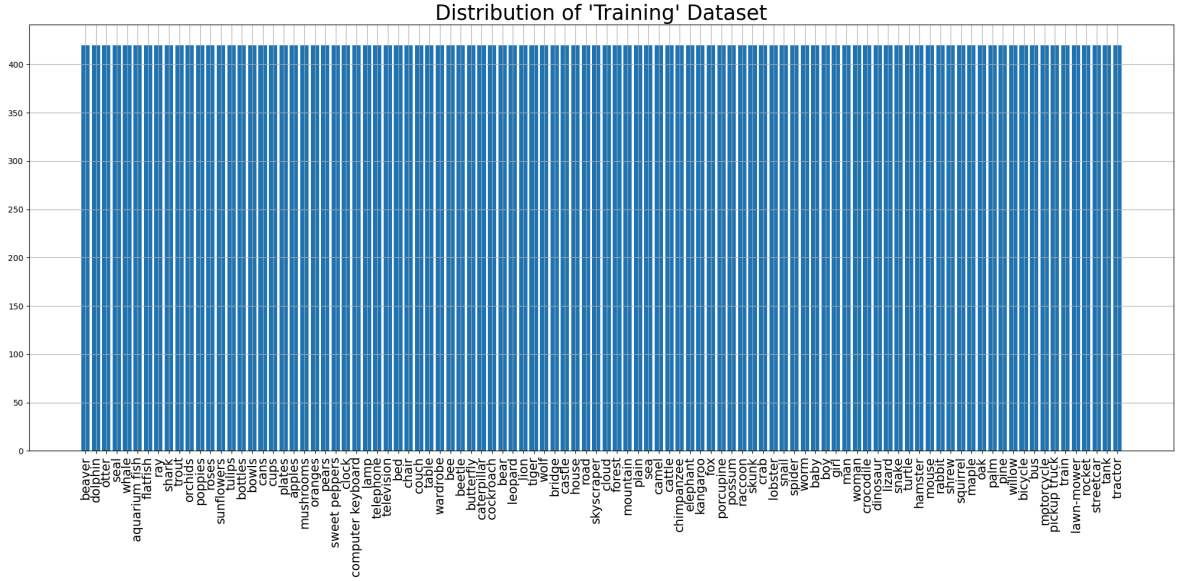
```
labels[31510].shape: (1,)
imgs[31510].shape: (32, 32, 3)
```



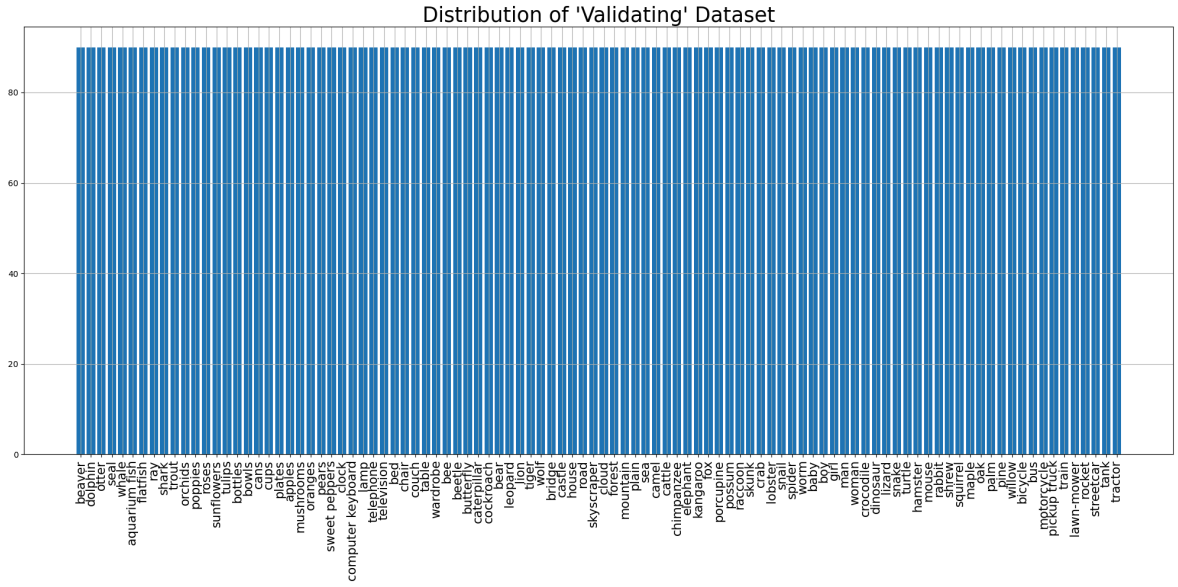

```
labels[38904].shape: (1,)
imgs[38904].shape: (32, 32, 3)
```



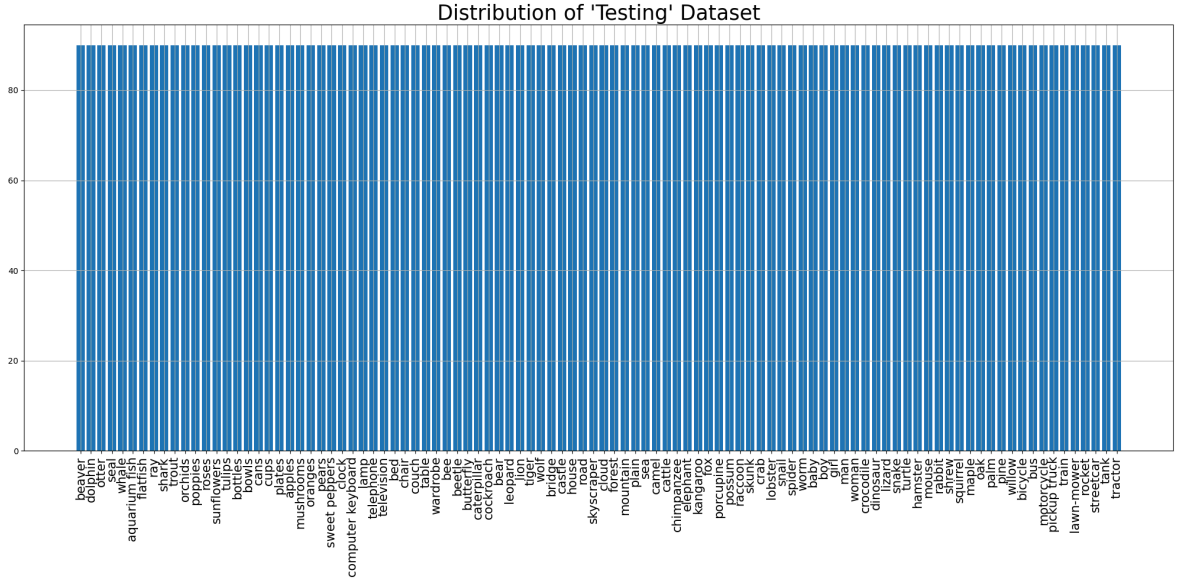
```
<ipython-input-4-b65b705d857f>:70: UserWarning: set_ticklabels() should only be u
sed with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```




```
<ipython-input-4-b65b705d857f>:70: UserWarning: set_ticklabels() should only be u
sed with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```




```
<ipython-input-4-b65b705d857f>:70: UserWarning: set_ticklabels() should only be u
sed with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
Training dataset:
 <_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.f
loat32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)

Validation dataset:
 <_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.f
loat32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)

Testing dataset:
 <_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.f
loat32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.f
ramework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_1"

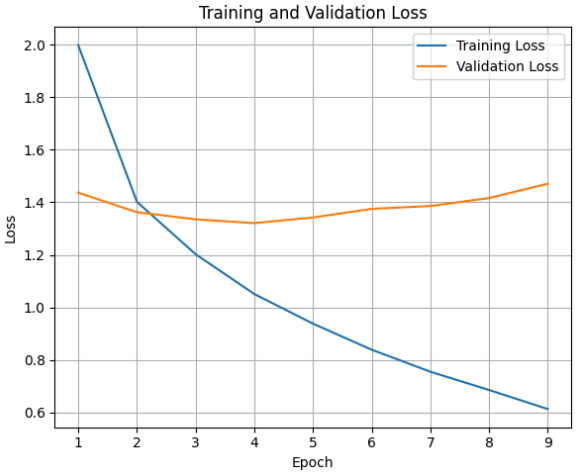
Layer (type)	Output Shape
resnet50v2 (Functional)	(None, 7, 7, 2048)
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 2048)
dense_2 (Dense)	(None, 512)
dropout_1 (Dropout)	(None, 512)
dense_3 (Dense)	(None, 100)

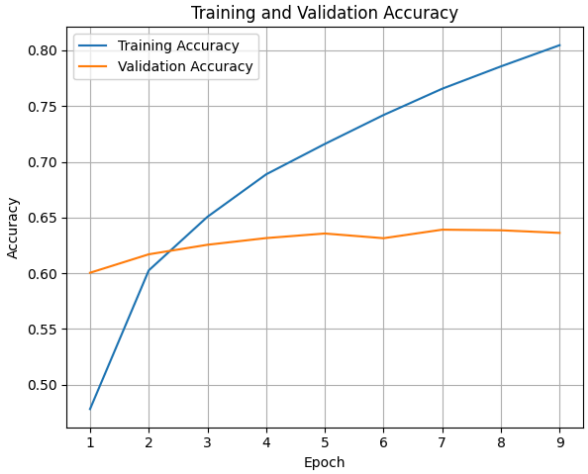
Total params: 24,665,188 (94.09 MB)

Trainable params: 1,100,388 (4.20 MB)

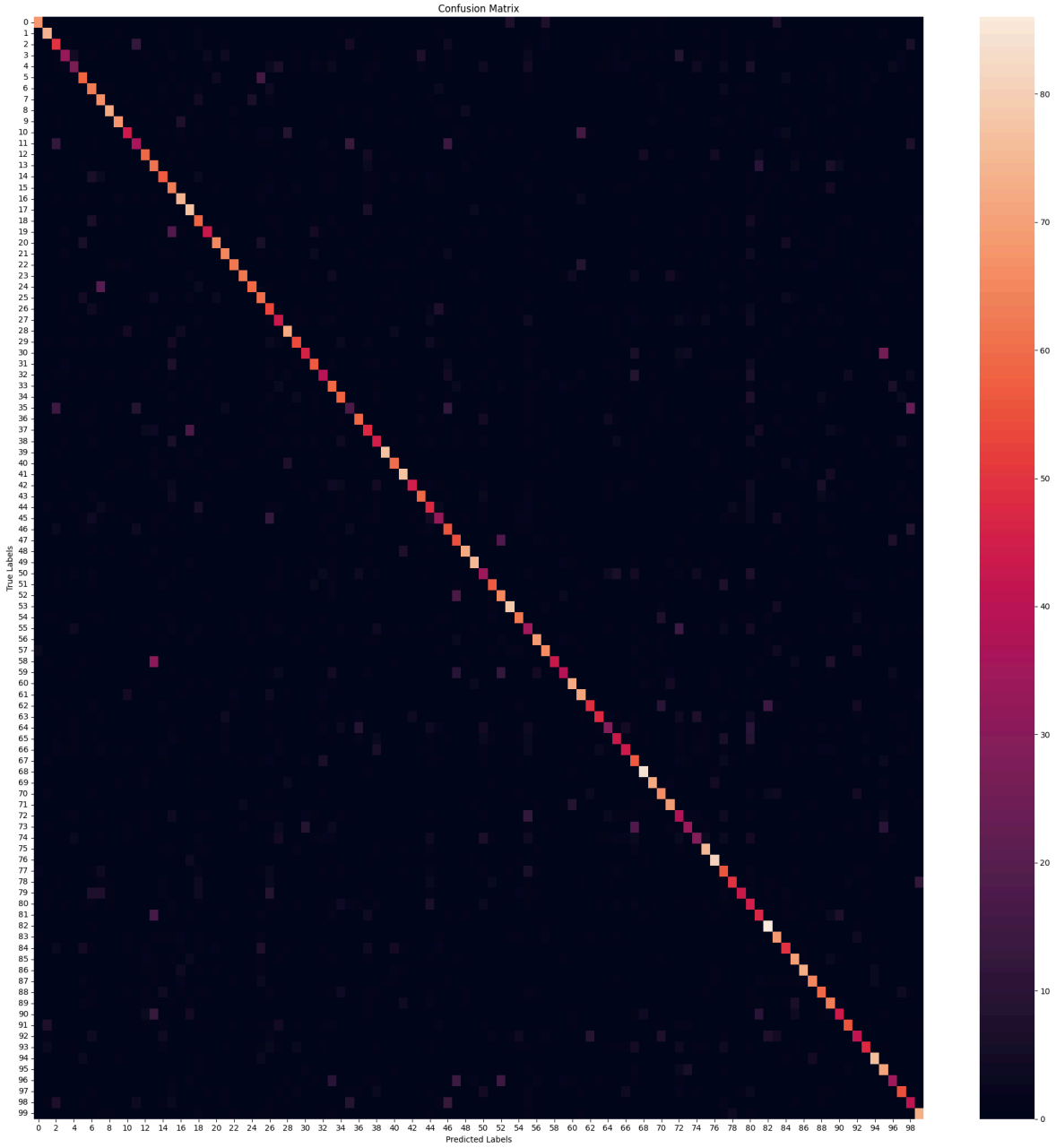
Non-trainable params: 23,564,800 (89.89 MB)

```
Epoch 1/25
657/657 ━━━━━━━━━━━━━━━━━━━━ 42s 48ms/step - accuracy: 0.3851 - f1_score: 0.3813
- loss: 2.5095 - precision: 0.6903 - val_accuracy: 0.6004 - val_f1_score: 0.5974
- val_loss: 1.4368 - val_precision: 0.8042
Epoch 2/25
657/657 ━━━━━━━━━━━━━━━━━━━━ 19s 29ms/step - accuracy: 0.5911 - f1_score: 0.5882
- loss: 1.4505 - precision: 0.7732 - val_accuracy: 0.6170 - val_f1_score: 0.6141
- val_loss: 1.3625 - val_precision: 0.7965
Epoch 3/25
657/657 ━━━━━━━━━━━━━━━━━━━━ 19s 29ms/step - accuracy: 0.6432 - f1_score: 0.6404
- loss: 1.2346 - precision: 0.8008 - val_accuracy: 0.6256 - val_f1_score: 0.6229
- val_loss: 1.3353 - val_precision: 0.7879
Epoch 4/25
657/657 ━━━━━━━━━━━━━━━━━━━━ 19s 29ms/step - accuracy: 0.6825 - f1_score: 0.6801
- loss: 1.0787 - precision: 0.8160 - val_accuracy: 0.6316 - val_f1_score: 0.6305
- val_loss: 1.3209 - val_precision: 0.7768
Epoch 5/25
657/657 ━━━━━━━━━━━━━━━━━━━━ 19s 29ms/step - accuracy: 0.7109 - f1_score: 0.7089
- loss: 0.9579 - precision: 0.8327 - val_accuracy: 0.6357 - val_f1_score: 0.6339
- val_loss: 1.3424 - val_precision: 0.7638
Epoch 6/25
657/657 ━━━━━━━━━━━━━━━━━━━━ 19s 29ms/step - accuracy: 0.7342 - f1_score: 0.7315
- loss: 0.8621 - precision: 0.8463 - val_accuracy: 0.6314 - val_f1_score: 0.6291
- val_loss: 1.3752 - val_precision: 0.7560
Epoch 7/25
657/657 ━━━━━━━━━━━━━━━━━━━━ 19s 29ms/step - accuracy: 0.7586 - f1_score: 0.7569
- loss: 0.7734 - precision: 0.8545 - val_accuracy: 0.6391 - val_f1_score: 0.6364
- val_loss: 1.3862 - val_precision: 0.7522
Epoch 8/25
657/657 ━━━━━━━━━━━━━━━━━━━━ 19s 29ms/step - accuracy: 0.7802 - f1_score: 0.7781
- loss: 0.7133 - precision: 0.8616 - val_accuracy: 0.6386 - val_f1_score: 0.6382
- val_loss: 1.4164 - val_precision: 0.7479
Epoch 9/25
657/657 ━━━━━━━━━━━━━━━━━━━━ 19s 28ms/step - accuracy: 0.7972 - f1_score: 0.7957
- loss: 0.6317 - precision: 0.8729 - val_accuracy: 0.6362 - val_f1_score: 0.6353
- val_loss: 1.4709 - val_precision: 0.7345
```





```
141/141 ━━━━━━━━━━━━━━━━━━━━ 3s 24ms/step - accuracy: 0.6161 - f1_score: 0.6096 -
loss: 1.3717 - precision: 0.7615
Test Accuracy: 62.86%
Test Loss: 133.00%
Test Precision: 77.59%
Test F1 Scores (Per Class): [82.92682  77.08333  51.54639  46.47887  38.028164 6
4.44444  58.878494
 64.73429  81.81818  81.6568   56.774185 46.15384  66.666664 51.476784
 60.63829  53.389828 76.28865  72.641495 55.50239  56.953632 77.8443
 72.625694 76.543205 77.01864  74.68353  57.971012 46.69603  47.77777
 75.789474 64.28571  59.740257 65.89595  48.48484  55.660374 64.088394
 24.11347  69.41176  50.793648 49.450542 90.05848  69.36415  81.481476
 62.499992 70.23808  47.999996 42.58064  53.080566 55.83756  81.81818
 82.41757  39.080452 69.51219  63.725483 86.18784  71.26436  33.018864
 77.52808  72.52747  64.17911  52.777775 78.68852  70.99999  63.2258
 65.306114 40.559433 52.12121  57.516335 50.44247  85.27918  83.72092
 67.005066 76.66666  38.974354 47.222214 39.189182 79.365074 82.2335
 52.582157 65.359474 59.722214 32.472317 52.51396  80.37383  69.346725
 54.945045 71.428566 81.56424  76.47058  64.13043  60.28707  55.345905
 67.878784 50.61728  61.146492 84.44444  65.1376   46.052628 64.70587
 41.83673  79.78141 ]
Average Test F1 Scores:62.87

282/282 ━━━━━━━━━━━━━━━━━━━━ 12s 29ms/step
```


Confusion Matrix
True Labels
Predicted Labels
0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52 54 56 58 60 62 64 66 68 70 72 74 76 78 80 82 84 86 88 90 92 94 96 98
0 10 20 30 40 50 60 70 80

	precision	recall	f1-score	support
beaver	0.92	0.76	0.83	90
dolphin	0.73	0.82	0.77	90
otter	0.48	0.56	0.52	90
seal	0.63	0.37	0.46	90
whale	0.52	0.30	0.38	90
aquarium fish	0.64	0.64	0.64	90
flatfish	0.51	0.70	0.59	90
ray	0.57	0.74	0.65	90
shark	0.84	0.80	0.82	90
trout	0.87	0.76	0.81	90
orchids	0.68	0.49	0.57	90
poppies	0.55	0.40	0.46	90
roses	0.68	0.66	0.67	90
sunflowers	0.41	0.68	0.51	90
tulips	0.58	0.63	0.61	90
bottles	0.43	0.70	0.53	90
bowls	0.71	0.82	0.76	90
cans	0.63	0.86	0.72	90
cups	0.48	0.64	0.55	90
plates	0.70	0.48	0.57	90
apples	0.84	0.72	0.78	90
mushrooms	0.72	0.72	0.72	90
oranges	0.86	0.69	0.77	90
pears	0.87	0.69	0.77	90
sweet peppers	0.87	0.66	0.75	90
clock	0.51	0.67	0.58	90
computer keyboard	0.39	0.59	0.47	90
lamp	0.48	0.48	0.48	90
telephone	0.72	0.80	0.76	90
television	0.69	0.60	0.64	90
bed	0.72	0.51	0.60	90
chair	0.69	0.63	0.66	90
couch	0.53	0.44	0.48	90
table	0.48	0.66	0.56	90
wardrobe	0.64	0.64	0.64	90
bee	0.33	0.19	0.24	90
beetle	0.74	0.66	0.69	90
butterfly	0.48	0.53	0.51	90
caterpillar	0.49	0.50	0.49	90
cockroach	0.95	0.86	0.90	90
bear	0.73	0.67	0.70	90
leopard	0.78	0.86	0.81	90
lion	0.85	0.50	0.63	90
tiger	0.76	0.66	0.70	90
wolf	0.44	0.53	0.48	90
bridge	0.51	0.37	0.43	90
castle	0.46	0.62	0.53	90
house	0.51	0.61	0.56	90
road	0.84	0.80	0.82	90
skyscraper	0.82	0.83	0.82	90
cloud	0.40	0.38	0.39	90
forest	0.77	0.63	0.70	90
mountain	0.57	0.72	0.64	90
plain	0.86	0.87	0.86	90
sea	0.74	0.69	0.71	90
camel	0.29	0.39	0.33	90
cattle	0.78	0.77	0.77	90
chimpanzee	0.72	0.73	0.73	90

elephant	0.98	0.48	0.64	90
kangaroo	0.72	0.42	0.53	90
fox	0.77	0.80	0.79	90
porcupine	0.65	0.79	0.71	90
possum	0.75	0.54	0.63	90
raccoon	0.84	0.53	0.65	90
skunk	0.55	0.32	0.41	90
crab	0.57	0.48	0.52	90
lobster	0.69	0.49	0.57	90
snail	0.42	0.63	0.50	90
spider	0.79	0.93	0.85	90
worm	0.87	0.80	0.83	90
baby	0.62	0.73	0.67	90
boy	0.77	0.77	0.77	90
girl	0.36	0.42	0.39	90
man	0.63	0.38	0.47	90
woman	0.50	0.32	0.39	90
crocodile	0.76	0.83	0.79	90
dinosaur	0.76	0.90	0.83	90
lizard	0.46	0.62	0.53	90
snake	0.79	0.56	0.65	90
turtle	0.81	0.48	0.60	90
hamster	0.25	0.50	0.33	90
mouse	0.53	0.52	0.53	90
rabbit	0.69	0.96	0.80	90
shrew	0.63	0.77	0.69	90
squirrel	0.54	0.56	0.55	90
maple	0.66	0.78	0.71	90
oak	0.82	0.81	0.82	90
palm	0.81	0.72	0.76	90
pine	0.63	0.66	0.64	90
willow	0.53	0.70	0.60	90
bicycle	0.64	0.49	0.55	90
bus	0.75	0.62	0.68	90
motorcycle	0.57	0.46	0.51	90
pickup truck	0.72	0.53	0.61	90
train	0.84	0.84	0.84	90
lawn-mower	0.55	0.79	0.65	90
rocket	0.56	0.39	0.46	90
streetcar	0.69	0.61	0.65	90
tank	0.39	0.46	0.42	90
tractor	0.78	0.81	0.80	90
accuracy			0.63	9000
macro avg	0.65	0.63	0.63	9000
weighted avg	0.65	0.63	0.63	9000

Model: "sequential_1"

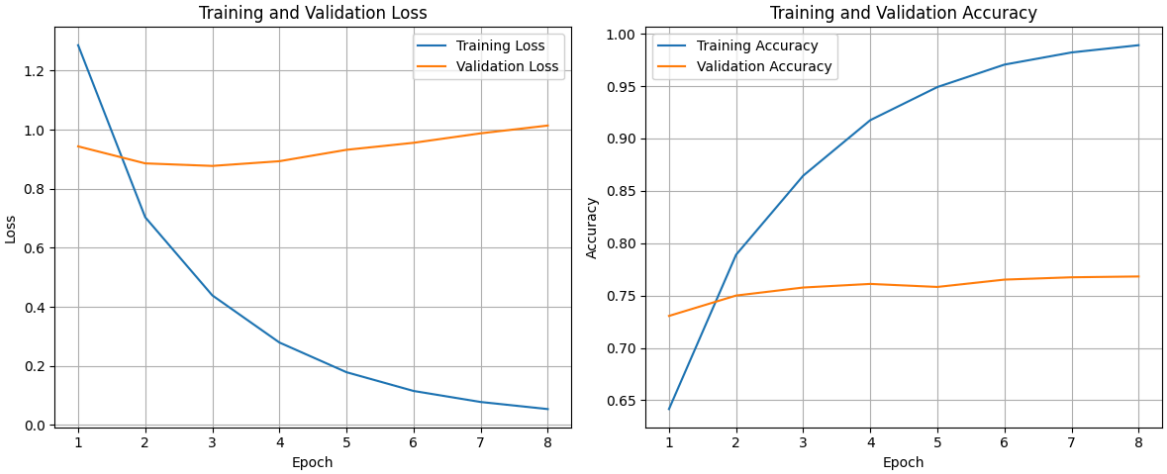
Layer (type)	Output Shape
resnet50v2 (Functional)	(None, 7, 7, 2048)
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 2048)
dense_2 (Dense)	(None, 512)
dropout_1 (Dropout)	(None, 512)
dense_3 (Dense)	(None, 100)

Total params: 24,665,188 (94.09 MB)

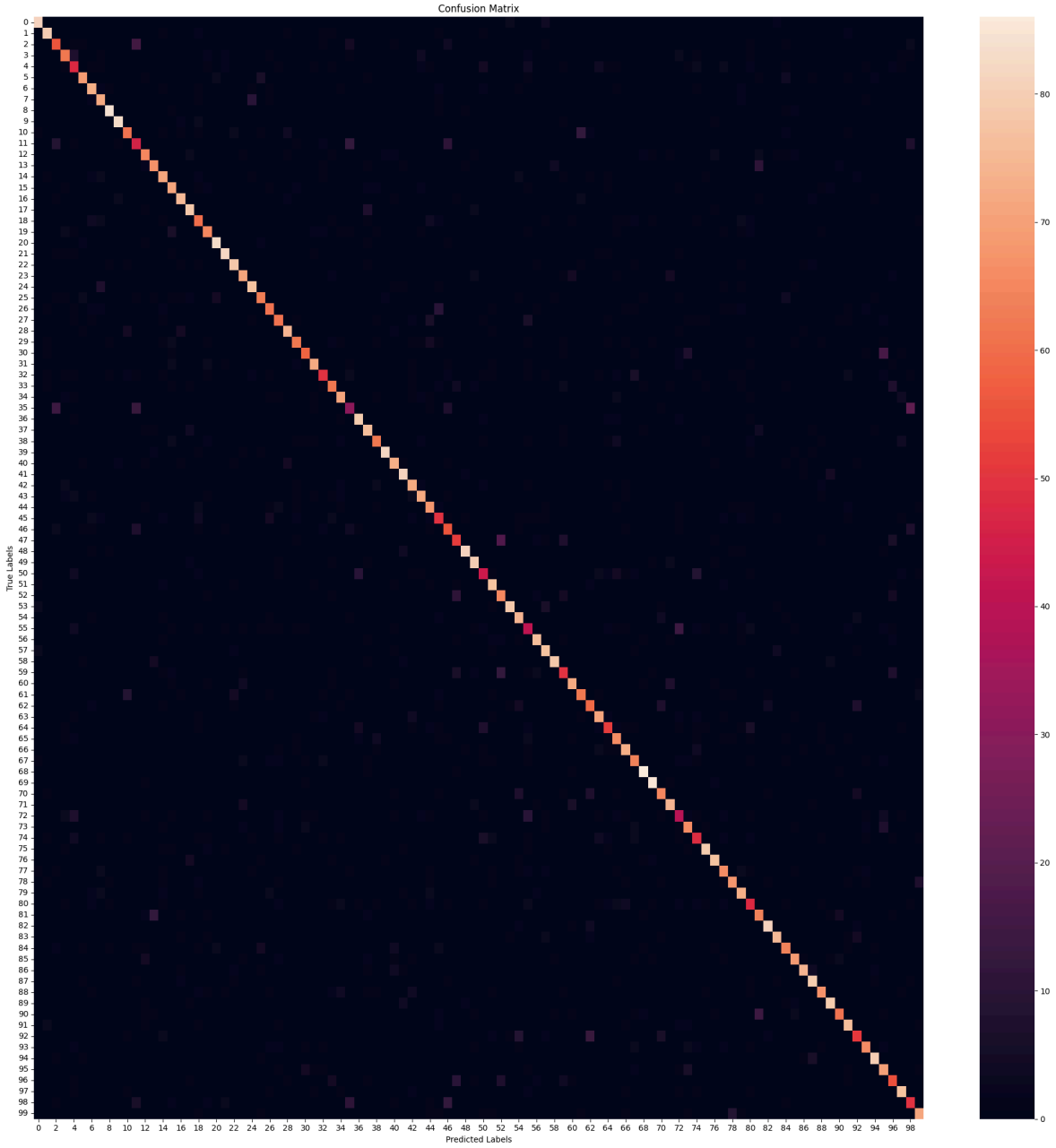
Trainable params: 24,619,748 (93.92 MB)

Non-trainable params: 45,440 (177.50 KB)

```
Epoch 1/15
657/657 ━━━━━━━━━━━━━━━━━━━━ 123s 113ms/step - accuracy: 0.5628 - f1_score: 0.562
4 - loss: 1.6490 - precision: 0.7709 - val_accuracy: 0.7306 - val_f1_score: 0.729
1 - val_loss: 0.9436 - val_precision: 0.8411
Epoch 2/15
657/657 ━━━━━━━━━━━━━━━━━━━━ 47s 72ms/step - accuracy: 0.7590 - f1_score: 0.7561
- loss: 0.8069 - precision: 0.8645 - val_accuracy: 0.7500 - val_f1_score: 0.7492
- val_loss: 0.8860 - val_precision: 0.8346
Epoch 3/15
657/657 ━━━━━━━━━━━━━━━━━━━━ 47s 71ms/step - accuracy: 0.8477 - f1_score: 0.8454
- loss: 0.4946 - precision: 0.9163 - val_accuracy: 0.7577 - val_f1_score: 0.7570
- val_loss: 0.8774 - val_precision: 0.8273
Epoch 4/15
657/657 ━━━━━━━━━━━━━━━━━━━━ 47s 71ms/step - accuracy: 0.9040 - f1_score: 0.9020
- loss: 0.3197 - precision: 0.9464 - val_accuracy: 0.7611 - val_f1_score: 0.7602
- val_loss: 0.8933 - val_precision: 0.8212
Epoch 5/15
657/657 ━━━━━━━━━━━━━━━━━━━━ 47s 71ms/step - accuracy: 0.9418 - f1_score: 0.9397
- loss: 0.2027 - precision: 0.9671 - val_accuracy: 0.7582 - val_f1_score: 0.7582
- val_loss: 0.9319 - val_precision: 0.8102
Epoch 6/15
657/657 ━━━━━━━━━━━━━━━━━━━━ 47s 71ms/step - accuracy: 0.9645 - f1_score: 0.9628
- loss: 0.1314 - precision: 0.9778 - val_accuracy: 0.7652 - val_f1_score: 0.7649
- val_loss: 0.9557 - val_precision: 0.8080
Epoch 7/15
657/657 ━━━━━━━━━━━━━━━━━━━━ 47s 71ms/step - accuracy: 0.9797 - f1_score: 0.9783
- loss: 0.0866 - precision: 0.9869 - val_accuracy: 0.7674 - val_f1_score: 0.7671
- val_loss: 0.9877 - val_precision: 0.8072
Epoch 8/15
657/657 ━━━━━━━━━━━━━━━━━━━━ 47s 71ms/step - accuracy: 0.9881 - f1_score: 0.9866
- loss: 0.0593 - precision: 0.9914 - val_accuracy: 0.7682 - val_f1_score: 0.7684
- val_loss: 1.0141 - val_precision: 0.8030
```




```
141/141 ━━━━━━━━━━━━━━━━━━━━ 3s 24ms/step - accuracy: 0.7452 - f1_score: 0.7381 -
loss: 0.9213 - precision: 0.8140
Test Accuracy: 75.24%
Test Loss: 88.82%
Test Precision: 81.92%
Test F1 Scores (Per Class): [89.502754 87.91208  62.222218 71.26436  52.173912 8
0.23255  77.24867
 75.789474 92.30768  92.73743  71.345024 50.549446 73.863625 74.86033
 75.53191  75.13227  82.41757  83.422455 64.86485  72.625694 90.217384
 89.13043  84.7826   78.88888  83.69564  72.83237  69.71428  69.31818
 83.14606  73.37277  71.604935 82.285706 62.5       70.45454  78.02197
 38.2716   84.04254  79.166664 73.37277  92.65536  78.07486  87.09677
 76.59574  79.55801  70.83333  55.865913 58.947365 58.757053 91.11111
 84.656075 52.071007 81.91489  67.01031  89.65517  74.747475 45.55555
 83.9779   80.20833  89.14285  58.479523 80.446915 72.94117  65.55555
 78.02197  63.030296 69.47368  80.66298  70.329666 91.489365 92.3913
 73.44632  80.87432  46.242767 72.13114  56.64739  88.76403  85.55555
 73.33333  78.61271  76.8421   54.545456 68.0851   89.99999  85.71427
 72.72727  78.40909  86.549706 84.491974 78.362564 86.81318  74.84662
 81.72042  60.355026 78.1065   89.385475 69.99999  60.439552 83.69564
 54.34782  79.329605]
Average Test F1 Scores:75.12

282/282 ━━━━━━━━━━━━━━━━━━━━ 12s 30ms/step
```


Confusion Matrix
True Labels
Predicted Labels
0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52 54 56 58 60 62 64 66 68 70 72 74 76 78 80 82 84 86 88 90 92 94 96 98
0 10 20 30 40 50 60 70 80

	precision	recall	f1-score	support
beaver	0.89	0.90	0.90	90
dolphin	0.87	0.89	0.88	90
otter	0.62	0.62	0.62	90
seal	0.74	0.69	0.71	90
whale	0.51	0.53	0.52	90
aquarium fish	0.84	0.77	0.80	90
flatfish	0.74	0.81	0.77	90
ray	0.72	0.80	0.76	90
shark	0.91	0.93	0.92	90
trout	0.93	0.92	0.93	90
orchids	0.75	0.68	0.71	90
poppies	0.50	0.51	0.51	90
roses	0.76	0.72	0.74	90
sunflowers	0.75	0.74	0.75	90
tulips	0.72	0.79	0.76	90
bottles	0.72	0.79	0.75	90
bowls	0.82	0.83	0.82	90
cans	0.80	0.87	0.83	90
cups	0.63	0.67	0.65	90
plates	0.73	0.72	0.73	90
apples	0.88	0.92	0.90	90
mushrooms	0.87	0.91	0.89	90
oranges	0.83	0.87	0.85	90
pears	0.79	0.79	0.79	90
sweet peppers	0.82	0.86	0.84	90
clock	0.76	0.70	0.73	90
computer keyboard	0.72	0.68	0.70	90
lamp	0.71	0.68	0.69	90
telephone	0.84	0.82	0.83	90
television	0.78	0.69	0.73	90
bed	0.81	0.64	0.72	90
chair	0.85	0.80	0.82	90
couch	0.71	0.56	0.62	90
table	0.71	0.69	0.70	90
wardrobe	0.77	0.79	0.78	90
bee	0.43	0.34	0.38	90
beetle	0.81	0.88	0.84	90
butterfly	0.75	0.84	0.79	90
caterpillar	0.78	0.69	0.73	90
cockroach	0.94	0.91	0.93	90
bear	0.75	0.81	0.78	90
leopard	0.84	0.90	0.87	90
lion	0.73	0.80	0.77	90
tiger	0.79	0.80	0.80	90
wolf	0.67	0.76	0.71	90
bridge	0.56	0.56	0.56	90
castle	0.56	0.62	0.59	90
house	0.60	0.58	0.59	90
road	0.91	0.90	0.91	90
skyscraper	0.81	0.89	0.85	90
cloud	0.56	0.49	0.52	90
forest	0.79	0.86	0.82	90
mountain	0.62	0.72	0.67	90
plain	0.93	0.87	0.90	90
sea	0.69	0.82	0.75	90
camel	0.46	0.46	0.46	90
cattle	0.84	0.84	0.84	90
chimpanzee	0.75	0.86	0.80	90

elephant	0.92	0.87	0.89	90
kangaroo	0.62	0.56	0.58	90
fox	0.81	0.80	0.80	90
porcupine	0.78	0.69	0.73	90
possum	0.66	0.66	0.66	90
raccoon	0.77	0.79	0.78	90
skunk	0.69	0.58	0.63	90
crab	0.66	0.73	0.69	90
lobster	0.80	0.81	0.81	90
snail	0.70	0.71	0.70	90
spider	0.88	0.96	0.91	90
worm	0.90	0.94	0.92	90
baby	0.75	0.72	0.73	90
boy	0.80	0.82	0.81	90
girl	0.48	0.44	0.46	90
man	0.71	0.73	0.72	90
woman	0.59	0.54	0.57	90
crocodile	0.90	0.88	0.89	90
dinosaur	0.86	0.86	0.86	90
lizard	0.73	0.73	0.73	90
snake	0.82	0.76	0.79	90
turtle	0.73	0.81	0.77	90
hamster	0.56	0.53	0.55	90
mouse	0.65	0.71	0.68	90
rabbit	0.90	0.90	0.90	90
shrew	0.88	0.83	0.86	90
squirrel	0.74	0.71	0.73	90
maple	0.80	0.77	0.78	90
oak	0.91	0.82	0.87	90
palm	0.81	0.88	0.84	90
pine	0.84	0.74	0.79	90
willow	0.86	0.88	0.87	90
bicycle	0.84	0.68	0.75	90
bus	0.79	0.84	0.82	90
motorcycle	0.65	0.57	0.60	90
pickup truck	0.84	0.73	0.78	90
train	0.90	0.89	0.89	90
lawn-mower	0.64	0.78	0.70	90
rocket	0.60	0.61	0.60	90
streetcar	0.82	0.86	0.84	90
tank	0.53	0.56	0.54	90
tractor	0.80	0.79	0.79	90
accuracy			0.75	9000
macro avg	0.75	0.75	0.75	9000
weighted avg	0.75	0.75	0.75	9000

224x224 w/ augment dataset

In [5]:
```
for repeat_2_times in range(2):
    #### <<<<<<<<<<Load and process data>>>>>>>>>>
    # Load CIFAR-100 dataset
    (X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

    # Split (8000) of training data into temporary set
    X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_s
    print(f"X_temp.shape: {X_temp.shape}\n")
```

```
# Split temp data into equal validation (4000) and testing (4000) data
X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp,
print(f"X_temp_val.shape: {X_temp_val.shape}")
print(f"y_temp_val.shape: {y_temp_val.shape}")
print(f"X_temp_test.shape: {X_temp_test.shape}")
print(f"y_temp_test.shape: {y_temp_test.shape}\n")

# Split test data into validation (5000) and testing (5000)
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.

# Add temp_val to validation (9000) and temp_test to testing (9000) to get a
X_val = np.concatenate((X_val, X_temp_val), axis=0)
y_val = np.concatenate((y_val, y_temp_val), axis=0)
X_test = np.concatenate((X_test, X_temp_test), axis=0)
y_test = np.concatenate((y_test, y_temp_test), axis=0)

print(f"X_train.shape: {X_train.shape}")
print(f"y_train.shape: {y_train.shape}")
print(f"X_val.shape: {X_val.shape}")
print(f"y_val.shape: {y_val.shape}")
print(f"X_test.shape: {X_test.shape}")
print(f"y_test.shape: {y_test.shape}\n")

def display_imgs(imgs, labels):
    plt.subplots(figsize=(10,10))
    for i in range(16):
        plt.subplot(4, 4, i+1)
        k = np.random.randint(0, imgs.shape[0])
        if i == 0:
            print(f"labels[{k}].shape: {labels[k].shape}")
            print(f"imgs[{k}].shape: {imgs[k].shape}")
        plt.imshow(imgs[k])
        #plt.title(labels[k])
        plt.axis('off')
    plt.tight_layout()
    plt.show()

display_imgs(X_train, y_train)

# Normalise images (scale to range [0, 1]) - Improves convergence speed & ac
X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
display_imgs(X_train, y_train)

labels_names = ['beaver','dolphin','otter','seal','whale','aquarium fish','f
                'orchids','poppies','roses','sunflowers','tulips','bottles','
                'apples','mushrooms','oranges','pears','sweet peppers','clock
                'telephone','television','bed','chair','couch','table','wardr
                'caterpillar','cockroach','bear','leopard','lion','tiger','wo
                'road','skyscraper','cloud','forest','mountain','plain','sea'
                'elephant','kangaroo','fox','porcupine','possum','raccoon','s
                'spider','worm','baby','boy','girl','man','woman','crocodile'
                'turtle','hamster','mouse','rabbit','shrew','squirrel','maple
                'bicycle','bus','motorcycle','pickup truck','train','lawn-mow
                'tractor']

def class_distrib(y, labels_names, dataset_name):
    counts = pd.DataFrame(data=y).value_counts().sort_index()
    #print(f"counts:\n{counts}")
    fig, ax = plt.subplots(figsize=(20,10))
```

```
    ax.bar(labels_names, counts)
    ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
    plt.title(f"Distribution of '{dataset_name}' Dataset", fontsize=25)
    plt.grid()
    plt.tight_layout()
    plt.show()
class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64
train_dataset_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                     tf.squeeze(tf.one_hot(y, depth=100, dtyp
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE))

def augment_dataset(x, y):
    x = tf.image.resize(x, (224, 224))  # Resize images
    x = tf.image.random_flip_left_right(x)  # Random horizontal flip
    x = tf.image.random_brightness(x, max_delta=0.2)  # Adjust brightness
    x = tf.image.random_contrast(x, lower=0.8, upper=1.2)  # Adjust contrast
    y = tf.squeeze(tf.one_hot(y, depth=100, dtype=tf.float32))  # One-hot en
    return x, y
train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(augment_dataset, num_parallel_calls=tf.data.experiment
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE))

# Combine the original dataset and the augmented dataset
train_dataset = train_dataset_.concatenate(train_dataset_aug)

# Optionally, shuffle the combined dataset if needed
#combined_dataset = combined_dataset.shuffle(buffer_size=1000)  # Adjust buf

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
                .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
                .batch(batch_size)
                .prefetch(tf.data.experimental.AUTOTUNE))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                     tf.squeeze(tf.one_hot(y, depth=100, dtyp
                 .batch(batch_size)
                 .prefetch(tf.data.experimental.AUTOTUNE))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}")  # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}")  # Should be (batch_size, 10)
    del img,lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}")  # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}")  # Should be (batch_size, 10)
```

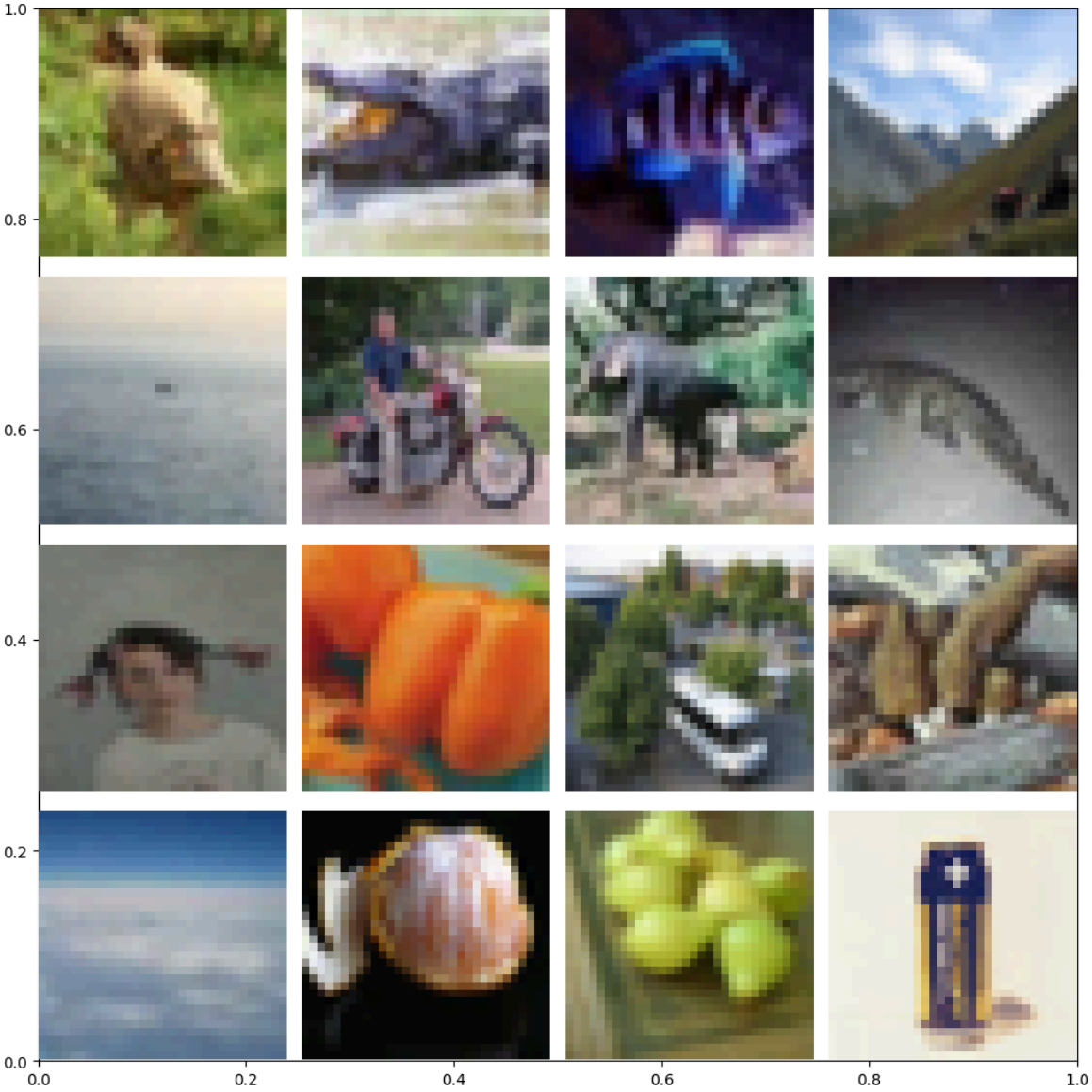
```
    del img,lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}")  # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}")  # Should be (batch_size, 10)
    del img,lbl


#### <<<<<<<<<<Pre-trained model>>>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

# Freeze the layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.4),
    layers.Dense(100, activation='softmax')  # CIFAR-100 has 100 classes
])

for sample in test_dataset.take(1):
    print(type(sample))  # Should be <class 'tuple'>
    print(len(sample))  # Should be 2
    print(type(sample[0]), type(sample[1]))  # Both should be <class 'tensor
    print(sample[0].shape)  # Should be (batch_size, 224, 224, 3)
    print(sample[1].shape)  # Should be (batch_size, 100)
print(f"Model input shape: {model.input_shape}")
print(f"Model output shape: {model.output_shape}")
sample = next(iter(test_dataset.as_numpy_iterator()))
print(len(sample))  # Should be 2
print(type(sample[0]), type(sample[1]))  # Both should be <class 'numpy.ndar
print(sample[0].shape, sample[1].shape)  # Should match model input and outp
print("\n")
#for x, y in test_dataset.take(1):
#    print(type(x), type(y))  # Both should be <class 'tensorflow.Tensor'>
#for x_batch, y_batch in test_dataset.take(1):
#    test_loss, test_acc = model.evaluate(x_batch, y_batch)
#    print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

# Compile the model
#tensorboard_callback = keras.callbacks.TensorBoard(log_dir="./logs")
model.compile(optimizer=optimizers.Adam(learning_rate=1e-3),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>
early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
                               patience=5, # Num. epochs with no improvement
                               restore_best_weights=True)
#reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
#                              factor=0.1, # Reduce lr by a factor
#                              patience=3, # Num epochs w/ no improvement
#                              min_lr=1e-6, # Min lr
```

```
    #                                verbose=1)
    #tensorboard = TensorBoard(log_dir='./logs', # Logs directory
    #                          histogram_freq=1, # Logs histograms for weights/ac
    #                          write_graph=True, # Logs graph of model
    #                          write_images=True) # Log images like weight histog
    #checkpoint = ModelCheckpoint('best_model.h5',
    #                             monitor='val_loss', # or val_accuracy
    #                             save_best_only=True, # Save only best model
    #                             mode='min', # min for loss or max for accuracy
    #                             verbose=1)
    #cvs_logger = CSVLogger('training_log.csv', seperator=',', append=True) # Sa

    # Train the model
    history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
                        batch_size=batch_size, callbacks=[early_stopping], verbo

    #### <<<<<<<<<<Plot Training & Validation Error>>>>>>>>>>

    # Extract loss and accuracy
    epochs = range(1,len(history.history['loss'])+1)
    train_loss = history.history['loss']
    val_loss = history.history['val_loss']
    train_acc = history.history['accuracy']
    val_acc = history.history['val_accuracy']

    def plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc):
        # Plot Training and Validation Loss
        plt.figure(figsize=(12, 5))
        plt.subplot(1, 2, 1)
        plt.plot(epochs, train_loss, label='Training Loss')
        plt.plot(epochs, val_loss, label='Validation Loss')
        plt.xlabel('Epoch')
        plt.ylabel('Loss')
        plt.title('Training and Validation Loss')
        plt.legend()
        plt.grid()

        # Plot Training and Validation Accuracy
        plt.subplot(1, 2, 2)
        plt.plot(epochs, train_acc, label='Training Accuracy')
        plt.plot(epochs, val_acc, label='Validation Accuracy')
        plt.xlabel('Epoch')
        plt.ylabel('Accuracy')
        plt.title('Training and Validation Accuracy')
        plt.legend()
        plt.grid()

        plt.tight_layout()
        plt.show()

    plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

    #### <<<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>

    # Evaluate on test data
    results = model.evaluate(test_dataset)
    test_loss = results[0]
    test_acc = results[1]
    test_precision = results[2]
    test_f1_scores = results[3]
```

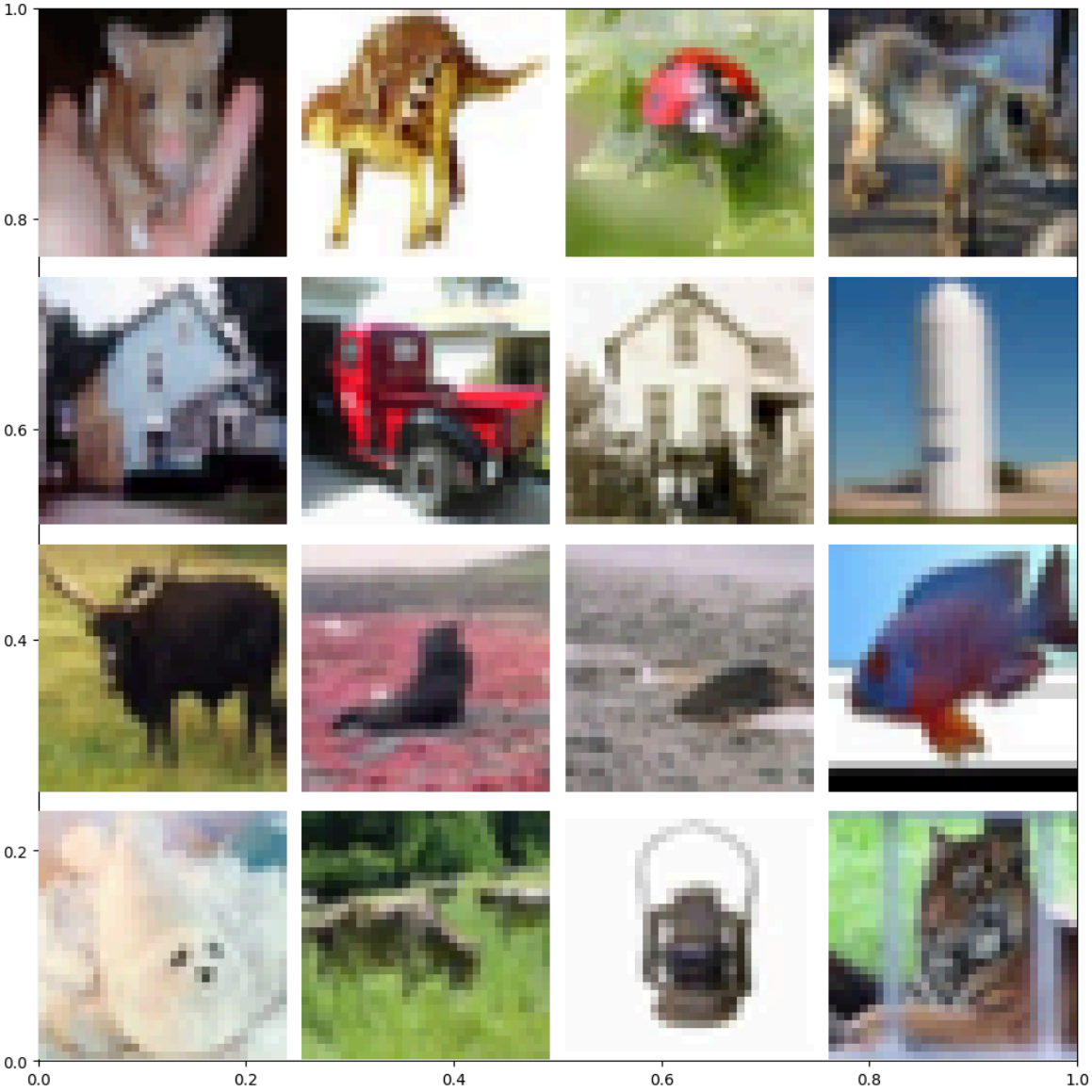
```
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss*100:.2f}%")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<<<Generate Confusion Matrix>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<<<Fine-Tune>>>>>>>>>>
#### <<<<<<<<<<Adapt Model>>>>>>>>>>
# Unfreeze last 10 layers
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ lower learning rate (prevents destroying learned features
model.compile(optimizer=optimizers.Adam(learning_rate=1e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<<<Modify Dataset>>>>>>>>>>
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                .map(augment_dataset, num_parallel_calls=tf.data.experiment
                .batch(batch_size)
                .prefetch(tf.data.experimental.AUTOTUNE))
train_dataset_aug = train_dataset.concatenate(train_dataset_aug)

# Not val or test as augment train helps generalise better, but want to prov

#### <<<<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>
```

```
# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset
                              batch_size=batch_size, callbacks=[early_stoppi

#### <<<<<<<<<<Plot Training & Validation Error>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1,len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>

# Evaluate on test data
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss*100:.2f}%")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<<<Generate Confusion Matrix>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

# Finally, display the hist DataFrame.
hist
```

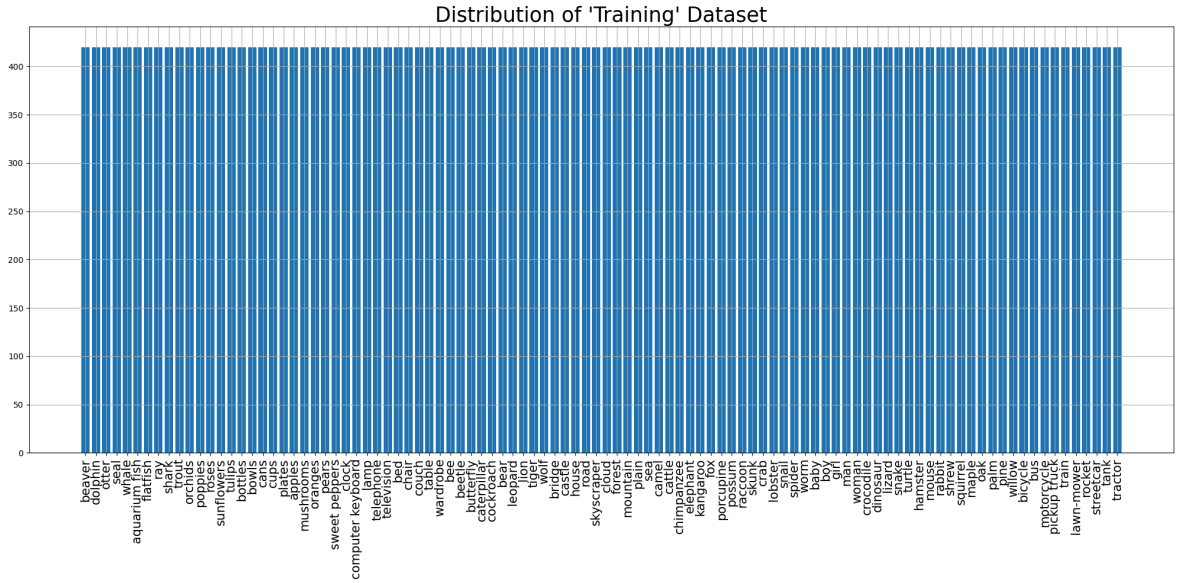
```
X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)
y_temp_val.shape: (4000, 1)
X_temp_test.shape: (4000, 32, 32, 3)
y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)
y_train.shape: (42000, 1)
X_val.shape: (9000, 32, 32, 3)
y_val.shape: (9000, 1)
X_test.shape: (9000, 32, 32, 3)
y_test.shape: (9000, 1)

labels[31576].shape: (1,)
imgs[31576].shape: (32, 32, 3)
```



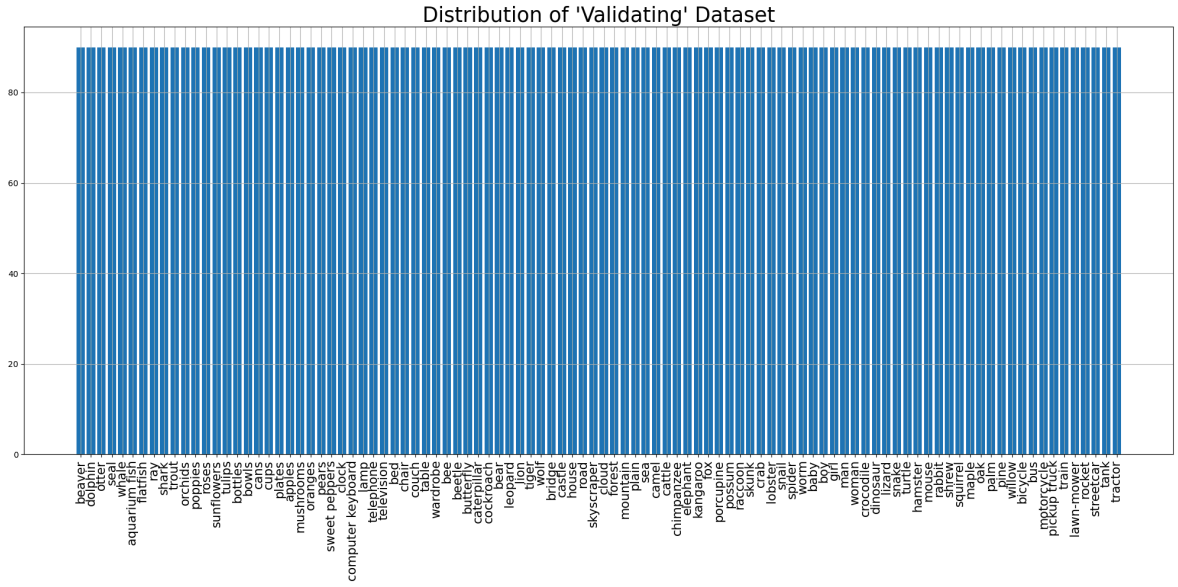

```
labels[31151].shape: (1,)
imgs[31151].shape: (32, 32, 3)
```



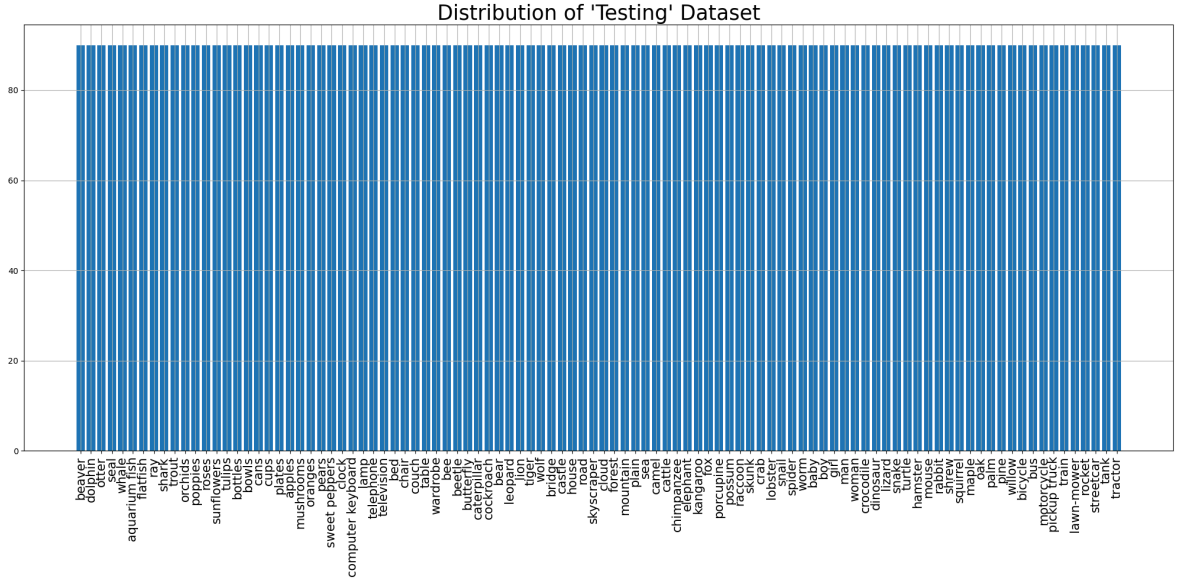

```
<ipython-input-5-791493d3fa27>:70: UserWarning: set_ticklabels() should only be u
sed with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```




```
<ipython-input-5-791493d3fa27>:70: UserWarning: set_ticklabels() should only be u
sed with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```




```
<ipython-input-5-791493d3fa27>:70: UserWarning: set_ticklabels() should only be u
sed with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
Training dataset:
 <_ConcatenateDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=t
f.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=Non
e))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)

Validation dataset:
 <_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.f
loat32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)

Testing dataset:
 <_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.f
loat32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.f
ramework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_2"

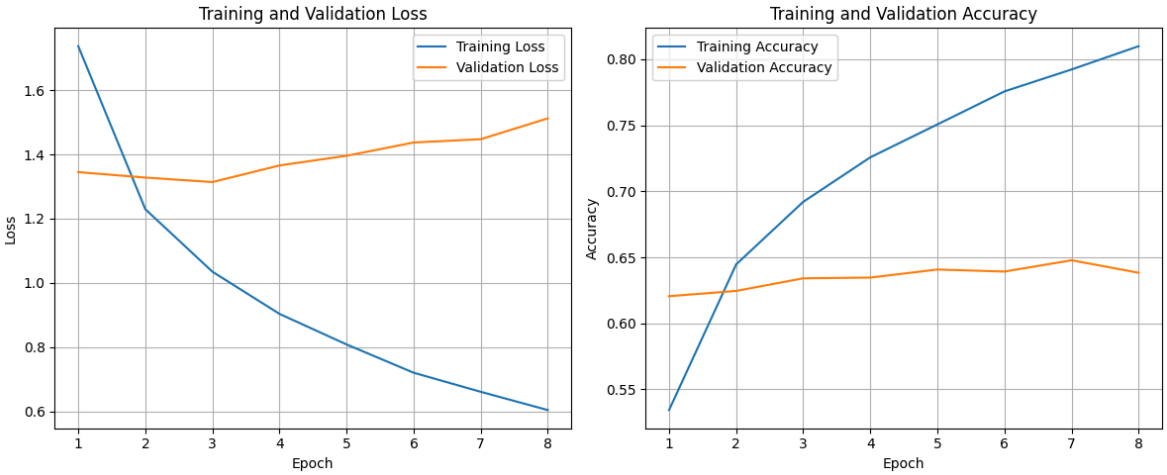
Layer (type)	Output Shape
resnet50v2 (Functional)	(None, 7, 7, 2048)
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None, 2048)
dense_4 (Dense)	(None, 512)
dropout_2 (Dropout)	(None, 512)
dense_5 (Dense)	(None, 100)

Total params: 24,665,188 (94.09 MB)

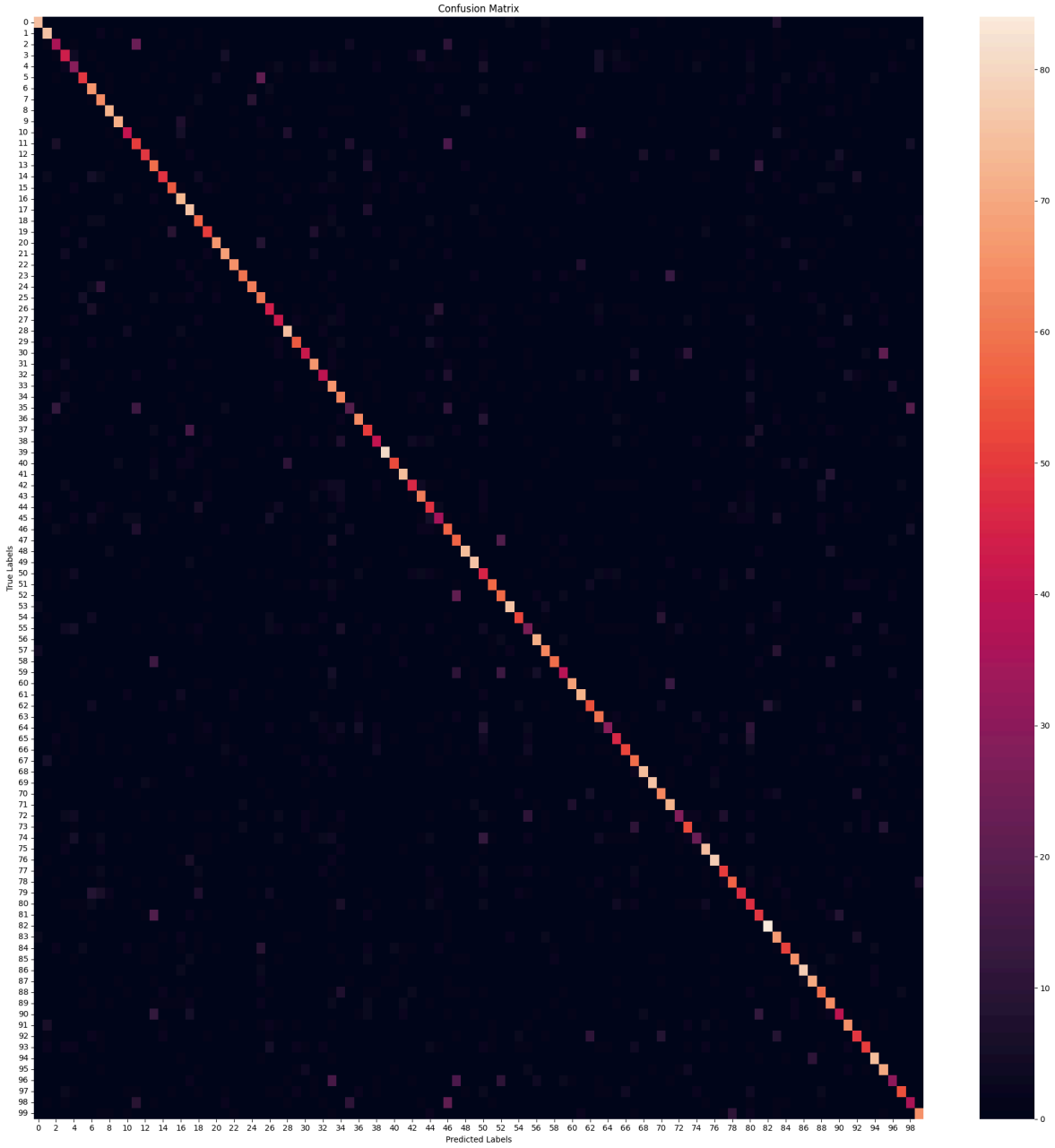
Trainable params: 1,100,388 (4.20 MB)

Non-trainable params: 23,564,800 (89.89 MB)

```
Epoch 1/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 58s 36ms/step - accuracy: 0.4483 - f1_score: 0.444
8 - loss: 2.1937 - precision: 0.7304 - val_accuracy: 0.6206 - val_f1_score: 0.617
2 - val_loss: 1.3451 - val_precision: 0.7973
Epoch 2/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 36s 28ms/step - accuracy: 0.6377 - f1_score: 0.635
6 - loss: 1.2608 - precision: 0.7978 - val_accuracy: 0.6246 - val_f1_score: 0.623
2 - val_loss: 1.3281 - val_precision: 0.7801
Epoch 3/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 36s 27ms/step - accuracy: 0.6913 - f1_score: 0.689
8 - loss: 1.0368 - precision: 0.8177 - val_accuracy: 0.6341 - val_f1_score: 0.632
3 - val_loss: 1.3141 - val_precision: 0.7785
Epoch 4/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 36s 27ms/step - accuracy: 0.7277 - f1_score: 0.726
1 - loss: 0.8927 - precision: 0.8369 - val_accuracy: 0.6347 - val_f1_score: 0.634
6 - val_loss: 1.3659 - val_precision: 0.7579
Epoch 5/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 36s 27ms/step - accuracy: 0.7573 - f1_score: 0.756
1 - loss: 0.7794 - precision: 0.8505 - val_accuracy: 0.6408 - val_f1_score: 0.639
1 - val_loss: 1.3960 - val_precision: 0.7483
Epoch 6/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 36s 27ms/step - accuracy: 0.7842 - f1_score: 0.783
0 - loss: 0.6852 - precision: 0.8619 - val_accuracy: 0.6392 - val_f1_score: 0.637
8 - val_loss: 1.4373 - val_precision: 0.7433
Epoch 7/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 36s 27ms/step - accuracy: 0.8065 - f1_score: 0.805
5 - loss: 0.6064 - precision: 0.8727 - val_accuracy: 0.6478 - val_f1_score: 0.646
5 - val_loss: 1.4477 - val_precision: 0.7436
Epoch 8/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 36s 27ms/step - accuracy: 0.8269 - f1_score: 0.825
9 - loss: 0.5418 - precision: 0.8833 - val_accuracy: 0.6383 - val_f1_score: 0.636
7 - val_loss: 1.5125 - val_precision: 0.7324
```



```
141/141 ━━━━━━━━━━━━━━━━━━━━ 3s 24ms/step - accuracy: 0.6200 - f1_score: 0.6117 -
loss: 1.3497 - precision: 0.7668
Test Accuracy: 63.37%
Test Loss: 131.65%
Test Precision: 78.19%
Test F1 Scores (Per Class): [84.26965  74.87685  45.85987  47.252743 35.36585  6
2.420376 63.46153
 63.681583 78.68852  81.14285  55.405403 48.07692  65.33333  54.128437
 62.745094 63.218384 71.219505 68.46846  57.575756 60.60605  79.51807
 74.999985 79.26829  74.99999  74.69879  57.819897 49.999992 50.602406
 78.12499  64.70587  56.756752 66.33166  47.904186 51.361866 57.798153
 27.73722  74.4186   53.763435 51.25     92.57143  65.43209  79.569885
 58.227837 68.508286 49.48453  43.749992 48.305077 54.54545  81.31868
 83.9779   39.301308 71.16564  59.487175 86.85714  64.197525 33.33333
 78.02197  71.99999  74.21383  53.424656 79.99999  71.99999  61.714287
 63.82978  43.60902  53.179188 66.25     57.281548 81.31868  86.85714
 67.37968  71.99999  40.875904 61.271667 33.333324 76.923065 83.422455
 57.142853 68.263466 62.66666  37.45019  53.84615  87.04663  57.510723
 60.714275 76.92307  82.10526  74.074066 59.595955 62.745094 52.28757
 67.01031  51.578945 60.975605 82.68156  66.66666  40.268448 65.85365
 43.636356 76.47058 ]
Average Test F1 Scores:63.22

282/282 ━━━━━━━━━━━━━━━━━━━━ 12s 29ms/step
```


Confusion Matrix
True Labels
Predicted Labels

	precision	recall	f1-score	support
beaver	0.85	0.83	0.84	90
dolphin	0.67	0.84	0.75	90
otter	0.54	0.40	0.46	90
seal	0.47	0.48	0.47	90
whale	0.39	0.32	0.35	90
aquarium fish	0.73	0.54	0.62	90
flatfish	0.56	0.73	0.63	90
ray	0.58	0.71	0.64	90
shark	0.77	0.80	0.79	90
trout	0.84	0.79	0.81	90
orchids	0.71	0.46	0.55	90
poppies	0.42	0.56	0.48	90
roses	0.82	0.54	0.65	90
sunflowers	0.46	0.66	0.54	90
tulips	0.76	0.53	0.63	90
bottles	0.65	0.61	0.63	90
bowls	0.63	0.81	0.71	90
cans	0.58	0.84	0.68	90
cups	0.53	0.63	0.58	90
plates	0.67	0.56	0.61	90
apples	0.87	0.73	0.80	90
mushrooms	0.73	0.76	0.74	90
oranges	0.88	0.72	0.79	90
pears	0.86	0.67	0.75	90
sweet peppers	0.82	0.69	0.75	90
clock	0.50	0.68	0.58	90
computer keyboard	0.51	0.49	0.50	90
lamp	0.55	0.47	0.51	90
telephone	0.74	0.83	0.78	90
television	0.69	0.61	0.65	90
bed	0.72	0.47	0.57	90
chair	0.61	0.73	0.66	90
couch	0.52	0.44	0.48	90
table	0.40	0.73	0.51	90
wardrobe	0.49	0.70	0.58	90
bee	0.40	0.21	0.28	90
beetle	0.78	0.71	0.74	90
butterfly	0.53	0.56	0.54	90
caterpillar	0.59	0.46	0.51	90
cockroach	0.95	0.90	0.93	90
bear	0.74	0.59	0.65	90
leopard	0.77	0.82	0.80	90
lion	0.68	0.51	0.58	90
tiger	0.68	0.69	0.69	90
wolf	0.46	0.53	0.49	90
bridge	0.50	0.39	0.44	90
castle	0.39	0.63	0.48	90
house	0.47	0.63	0.54	90
road	0.80	0.82	0.81	90
skyscraper	0.84	0.84	0.84	90
cloud	0.32	0.50	0.39	90
forest	0.79	0.64	0.71	90
mountain	0.55	0.64	0.59	90
plain	0.89	0.84	0.87	90
sea	0.72	0.58	0.64	90
camel	0.39	0.29	0.33	90
cattle	0.77	0.79	0.78	90
chimpanzee	0.74	0.70	0.72	90

elephant	0.86	0.66	0.74	90
kangaroo	0.70	0.43	0.53	90
fox	0.84	0.76	0.80	90
porcupine	0.65	0.80	0.72	90
possum	0.64	0.60	0.62	90
raccoon	0.61	0.67	0.64	90
skunk	0.67	0.32	0.44	90
crab	0.55	0.51	0.53	90
lobster	0.76	0.58	0.66	90
snail	0.51	0.66	0.57	90
spider	0.80	0.82	0.81	90
worm	0.89	0.84	0.87	90
baby	0.65	0.70	0.67	90
boy	0.65	0.80	0.72	90
girl	0.60	0.31	0.41	90
man	0.64	0.59	0.61	90
woman	0.48	0.26	0.33	90
crocodile	0.71	0.83	0.77	90
dinosaur	0.80	0.87	0.83	90
lizard	0.59	0.56	0.57	90
snake	0.74	0.63	0.68	90
turtle	0.78	0.52	0.63	90
hamster	0.29	0.52	0.37	90
mouse	0.54	0.54	0.54	90
rabbit	0.82	0.93	0.87	90
shrew	0.47	0.74	0.58	90
squirrel	0.65	0.57	0.61	90
maple	0.82	0.72	0.77	90
oak	0.78	0.87	0.82	90
palm	0.71	0.78	0.74	90
pine	0.55	0.66	0.60	90
willow	0.56	0.71	0.62	90
bicycle	0.62	0.44	0.52	90
bus	0.62	0.72	0.67	90
motorcycle	0.49	0.54	0.52	90
pickup truck	0.68	0.56	0.61	90
train	0.83	0.82	0.83	90
lawn-mower	0.58	0.78	0.67	90
rocket	0.52	0.33	0.41	90
streetcar	0.73	0.60	0.66	90
tank	0.48	0.40	0.44	90
tractor	0.81	0.72	0.76	90
accuracy			0.63	9000
macro avg	0.65	0.63	0.63	9000
weighted avg	0.65	0.63	0.63	9000

Model: "sequential_2"

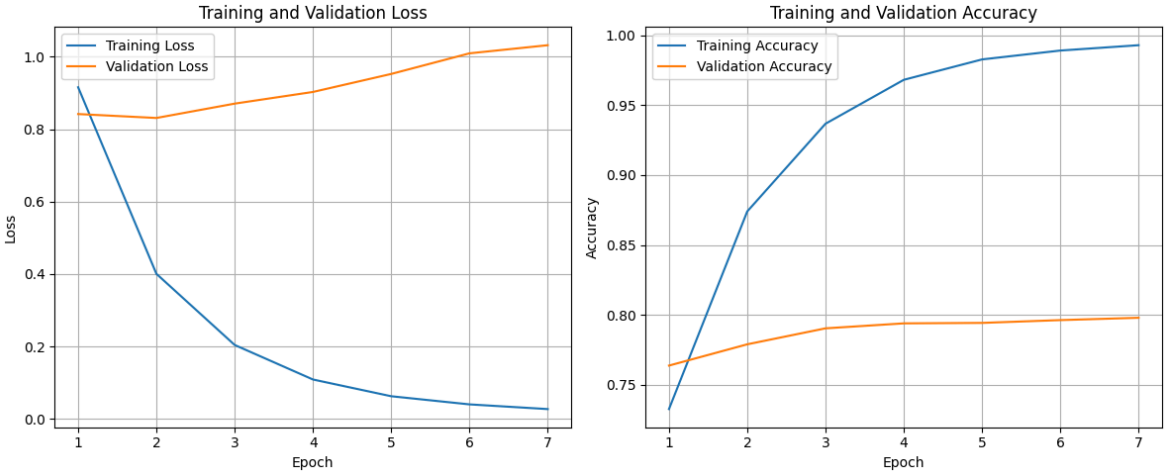
Layer (type)	Output Shape
resnet50v2 (Functional)	(None, 7, 7, 2048)
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None, 2048)
dense_4 (Dense)	(None, 512)
dropout_2 (Dropout)	(None, 512)
dense_5 (Dense)	(None, 100)

Total params: 24,665,188 (94.09 MB)

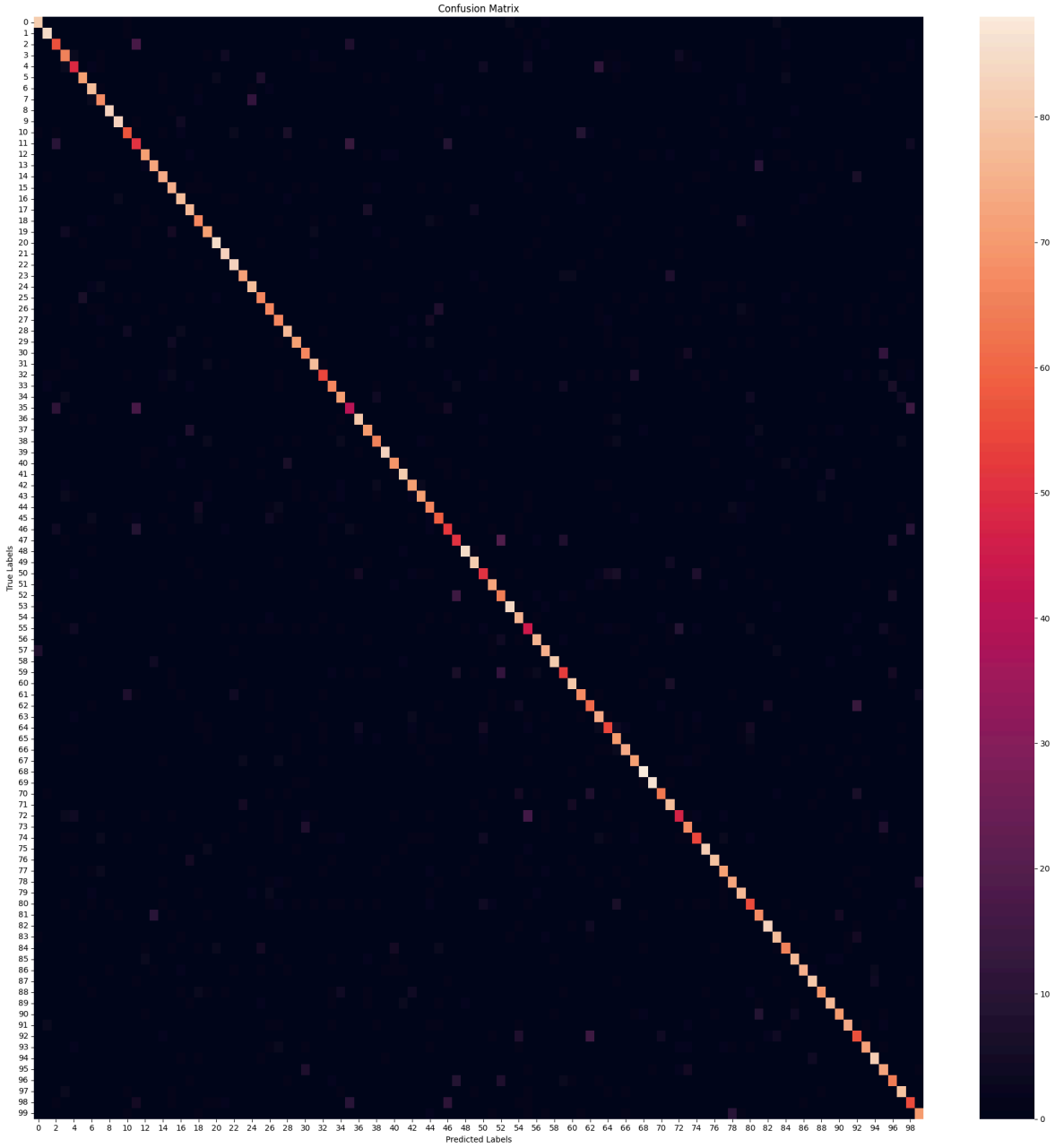
Trainable params: 24,619,748 (93.92 MB)

Non-trainable params: 45,440 (177.50 KB)

```
Epoch 1/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 215s 83ms/step - accuracy: 0.6475 - f1_score: 0.64
66 - loss: 1.2614 - precision: 0.8068 - val_accuracy: 0.7637 - val_f1_score: 0.76
29 - val_loss: 0.8419 - val_precision: 0.8399
Epoch 2/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 137s 69ms/step - accuracy: 0.8580 - f1_score: 0.85
69 - loss: 0.4557 - precision: 0.9146 - val_accuracy: 0.7789 - val_f1_score: 0.77
79 - val_loss: 0.8310 - val_precision: 0.8326
Epoch 3/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 136s 69ms/step - accuracy: 0.9349 - f1_score: 0.93
42 - loss: 0.2109 - precision: 0.9593 - val_accuracy: 0.7903 - val_f1_score: 0.79
04 - val_loss: 0.8706 - val_precision: 0.8286
Epoch 4/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 136s 69ms/step - accuracy: 0.9712 - f1_score: 0.97
07 - loss: 0.1046 - precision: 0.9800 - val_accuracy: 0.7939 - val_f1_score: 0.79
39 - val_loss: 0.9028 - val_precision: 0.8256
Epoch 5/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 136s 69ms/step - accuracy: 0.9861 - f1_score: 0.98
56 - loss: 0.0543 - precision: 0.9896 - val_accuracy: 0.7942 - val_f1_score: 0.79
41 - val_loss: 0.9525 - val_precision: 0.8187
Epoch 6/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 136s 69ms/step - accuracy: 0.9915 - f1_score: 0.99
09 - loss: 0.0341 - precision: 0.9934 - val_accuracy: 0.7962 - val_f1_score: 0.79
61 - val_loss: 1.0094 - val_precision: 0.8183
Epoch 7/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 136s 69ms/step - accuracy: 0.9946 - f1_score: 0.99
41 - loss: 0.0224 - precision: 0.9954 - val_accuracy: 0.7979 - val_f1_score: 0.79
81 - val_loss: 1.0319 - val_precision: 0.8174
```




```
141/141 ━━━━━━━━━━━━━━━━━━━━ 3s 24ms/step - accuracy: 0.7783 - f1_score: 0.7715 -
loss: 0.8059 - precision: 0.8247
Test Accuracy: 78.24%
Test Loss: 80.33%
Test Precision: 82.96%
Test F1 Scores (Per Class): [89.01098  92.3913   64.       71.03825  59.39393  8
2.08092  81.24999
 76.83616  93.7853   92.73743  68.67469  51.77665  81.60919  81.11111
 82.68156  77.7202   84.32432  81.481476 73.62636  78.453026 92.3913
 89.24731  89.72973  83.236984 84.32432  74.57626  74.157295 72.52747
 82.7957   82.558136 75.706215 84.7826   68.35443  76.13636  79.329605
 49.397583 86.95651  78.65168  74.71264  92.22221  77.52808  89.502754
 78.02197  81.60919  70.15707  67.816086 59.090908 58.620686 92.89617
 89.01098  58.620686 82.48587  65.30612  93.33333  76.76767  51.136356
 84.44444  82.872925 92.57143  64.28571  86.48647  77.71428  67.03297
 76.28865  64.670654 69.99999  86.04651  78.88888  92.63157  95.55555
 76.82927  80.82901  55.95237  76.40449  63.905323 90.60773  88.76403
 79.329605 80.66298  81.05263  60.109283 73.91304  91.208786 82.97872
 74.28571  84.61538  86.70519  87.43169  82.142845 87.00564  80.92485
 82.68156  60.21505  82.75862  90.10989  74.111664 68.8172   84.04254
 59.7826   80.92485 ]
Average Test F1 Scores:78.16

282/282 ━━━━━━━━━━━━━━━━━━━━ 12s 29ms/step
```

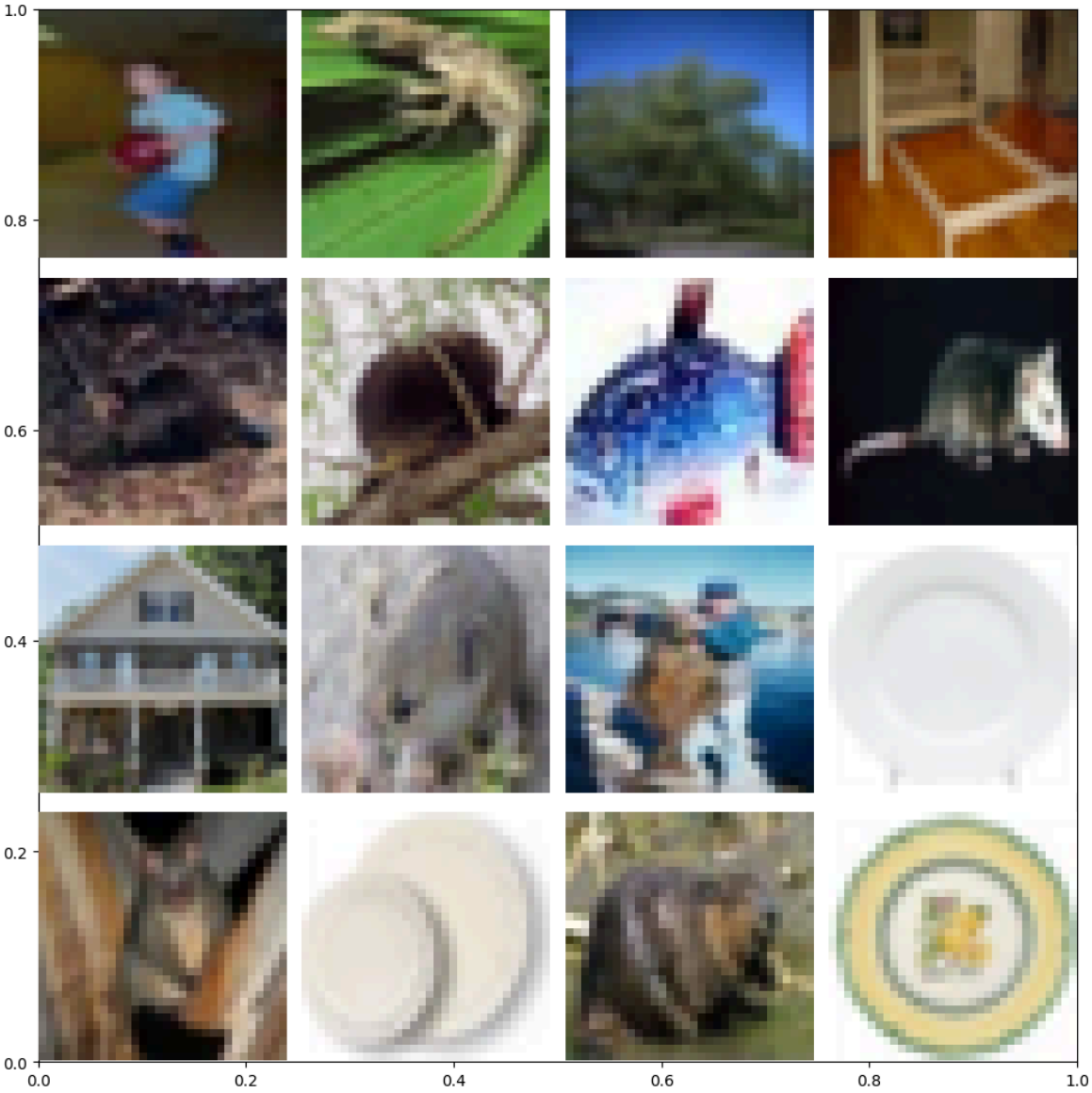

Confusion Matrix
True Labels
Predicted Labels

	precision	recall	f1-score	support
beaver	0.88	0.90	0.89	90
dolphin	0.90	0.94	0.92	90
otter	0.66	0.62	0.64	90
seal	0.70	0.72	0.71	90
whale	0.65	0.54	0.59	90
aquarium fish	0.86	0.79	0.82	90
flatfish	0.76	0.87	0.81	90
ray	0.78	0.76	0.77	90
shark	0.95	0.92	0.94	90
trout	0.93	0.92	0.93	90
orchids	0.75	0.63	0.69	90
poppies	0.48	0.57	0.52	90
roses	0.85	0.79	0.82	90
sunflowers	0.81	0.81	0.81	90
tulips	0.83	0.82	0.83	90
bottles	0.73	0.83	0.78	90
bowls	0.82	0.87	0.84	90
cans	0.78	0.86	0.81	90
cups	0.73	0.74	0.74	90
plates	0.78	0.79	0.78	90
apples	0.90	0.94	0.92	90
mushrooms	0.86	0.92	0.89	90
oranges	0.87	0.92	0.90	90
pears	0.87	0.80	0.83	90
sweet peppers	0.82	0.87	0.84	90
clock	0.76	0.73	0.75	90
computer keyboard	0.74	0.73	0.74	90
lamp	0.72	0.73	0.73	90
telephone	0.80	0.86	0.83	90
television	0.87	0.79	0.83	90
bed	0.77	0.74	0.76	90
chair	0.83	0.87	0.85	90
couch	0.79	0.60	0.68	90
table	0.78	0.74	0.76	90
wardrobe	0.80	0.79	0.79	90
bee	0.54	0.46	0.49	90
beetle	0.85	0.89	0.87	90
butterfly	0.80	0.78	0.79	90
caterpillar	0.77	0.72	0.75	90
cockroach	0.92	0.92	0.92	90
bear	0.78	0.77	0.78	90
leopard	0.89	0.90	0.90	90
lion	0.77	0.79	0.78	90
tiger	0.85	0.79	0.82	90
wolf	0.66	0.74	0.70	90
bridge	0.71	0.66	0.68	90
castle	0.60	0.58	0.59	90
house	0.61	0.57	0.59	90
road	0.91	0.94	0.93	90
skyscraper	0.88	0.90	0.89	90
cloud	0.61	0.57	0.59	90
forest	0.84	0.81	0.82	90
mountain	0.60	0.71	0.65	90
plain	0.93	0.93	0.93	90
sea	0.70	0.84	0.77	90
camel	0.52	0.50	0.51	90
cattle	0.84	0.84	0.84	90
chimpanzee	0.82	0.83	0.83	90

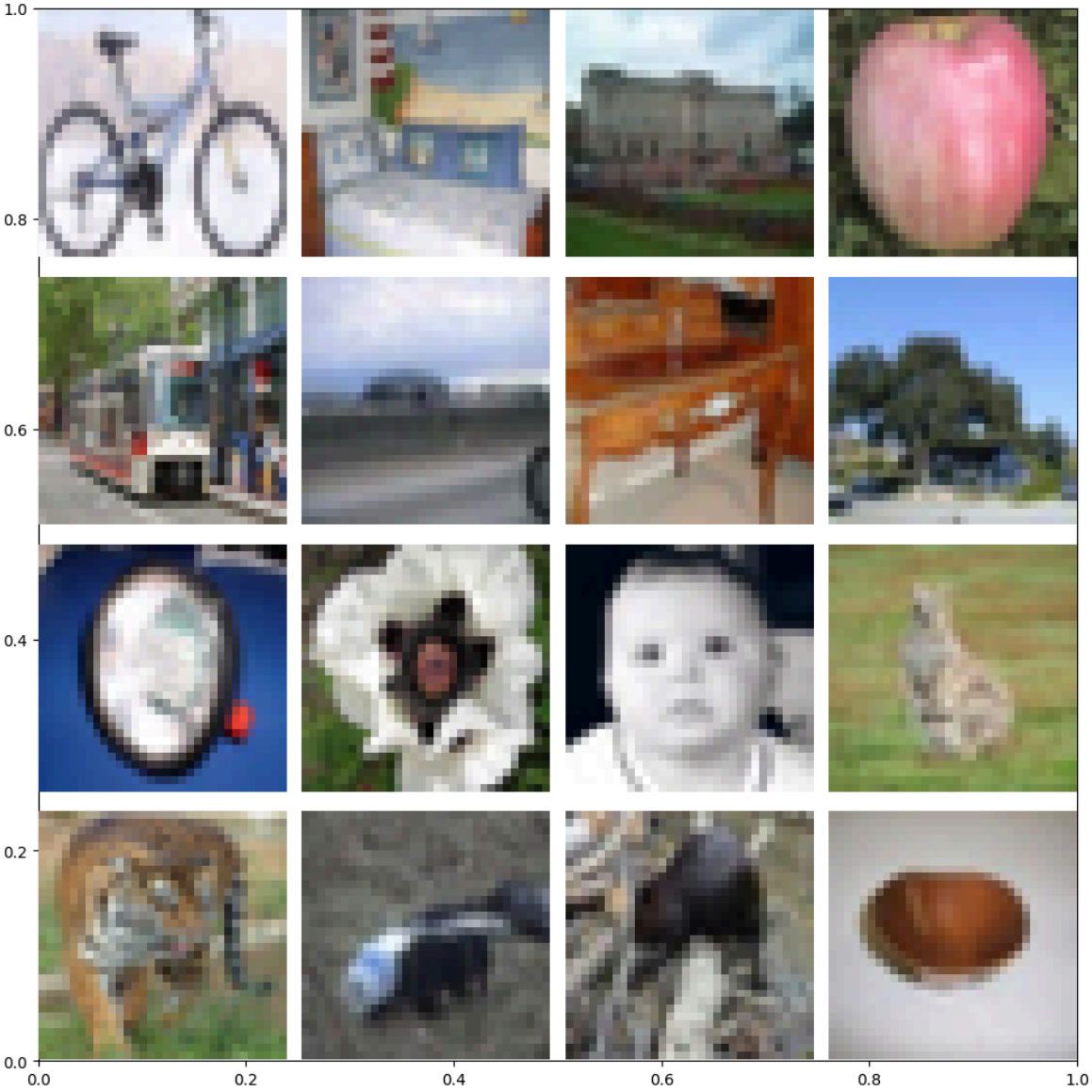
elephant	0.95	0.90	0.93	90
kangaroo	0.69	0.59	0.63	90
fox	0.84	0.89	0.86	90
porcupine	0.80	0.76	0.78	90
possum	0.66	0.68	0.67	90
raccoon	0.71	0.82	0.76	90
skunk	0.70	0.60	0.65	90
crab	0.64	0.78	0.70	90
lobster	0.90	0.82	0.86	90
snail	0.79	0.79	0.79	90
spider	0.88	0.98	0.93	90
worm	0.96	0.96	0.96	90
baby	0.85	0.70	0.77	90
boy	0.76	0.87	0.81	90
girl	0.60	0.52	0.56	90
man	0.77	0.76	0.76	90
woman	0.68	0.60	0.64	90
crocodile	0.90	0.91	0.91	90
dinosaur	0.90	0.88	0.89	90
lizard	0.80	0.79	0.79	90
snake	0.80	0.81	0.81	90
turtle	0.77	0.86	0.81	90
hamster	0.59	0.61	0.60	90
mouse	0.72	0.76	0.74	90
rabbit	0.90	0.92	0.91	90
shrew	0.80	0.87	0.83	90
squirrel	0.76	0.72	0.74	90
maple	0.84	0.86	0.85	90
oak	0.90	0.83	0.87	90
palm	0.86	0.89	0.87	90
pine	0.88	0.77	0.82	90
willow	0.89	0.86	0.87	90
bicycle	0.84	0.78	0.81	90
bus	0.83	0.82	0.83	90
motorcycle	0.58	0.62	0.60	90
pickup truck	0.86	0.80	0.83	90
train	0.89	0.91	0.90	90
lawn-mower	0.68	0.81	0.74	90
rocket	0.67	0.71	0.69	90
streetcar	0.81	0.88	0.84	90
tank	0.59	0.61	0.60	90
tractor	0.84	0.78	0.81	90
accuracy			0.78	9000
macro avg	0.78	0.78	0.78	9000
weighted avg	0.78	0.78	0.78	9000

```
X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)
y_temp_val.shape: (4000, 1)
X_temp_test.shape: (4000, 32, 32, 3)
y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)
y_train.shape: (42000, 1)
X_val.shape: (9000, 32, 32, 3)
y_val.shape: (9000, 1)
X_test.shape: (9000, 32, 32, 3)
y_test.shape: (9000, 1)
```

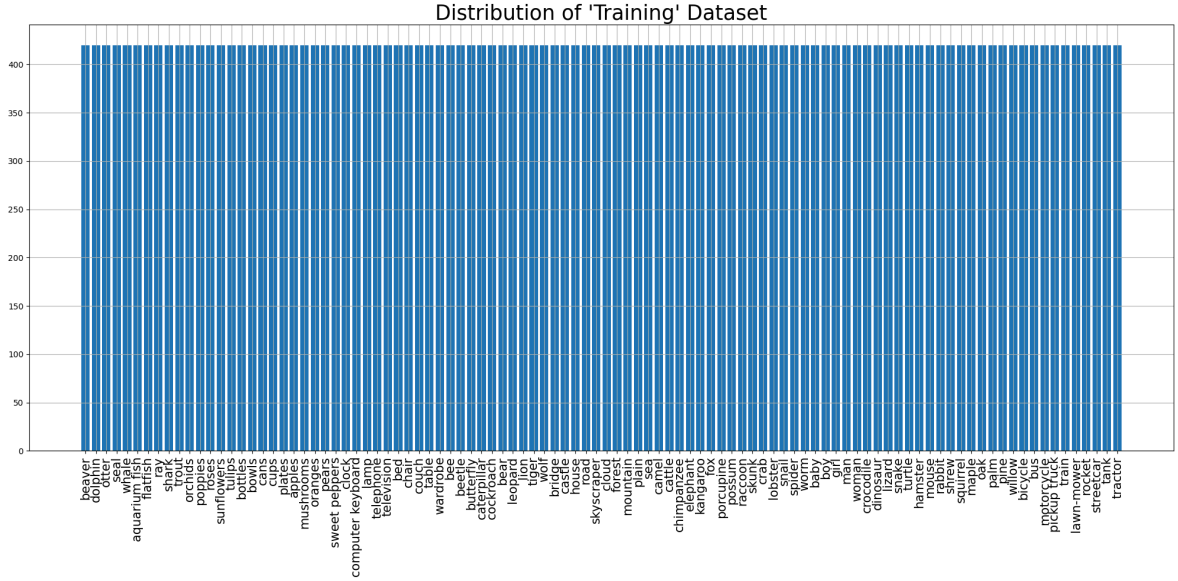
```
labels[36370].shape: (1,)
imgs[36370].shape: (32, 32, 3)
```



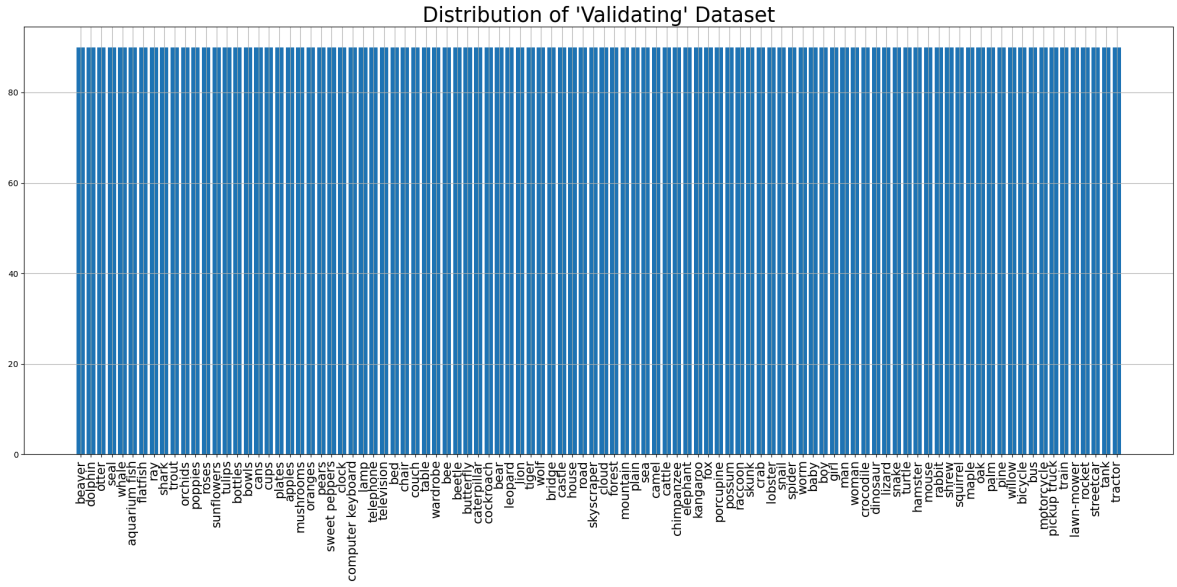

```
labels[37835].shape: (1,)
imgs[37835].shape: (32, 32, 3)
```



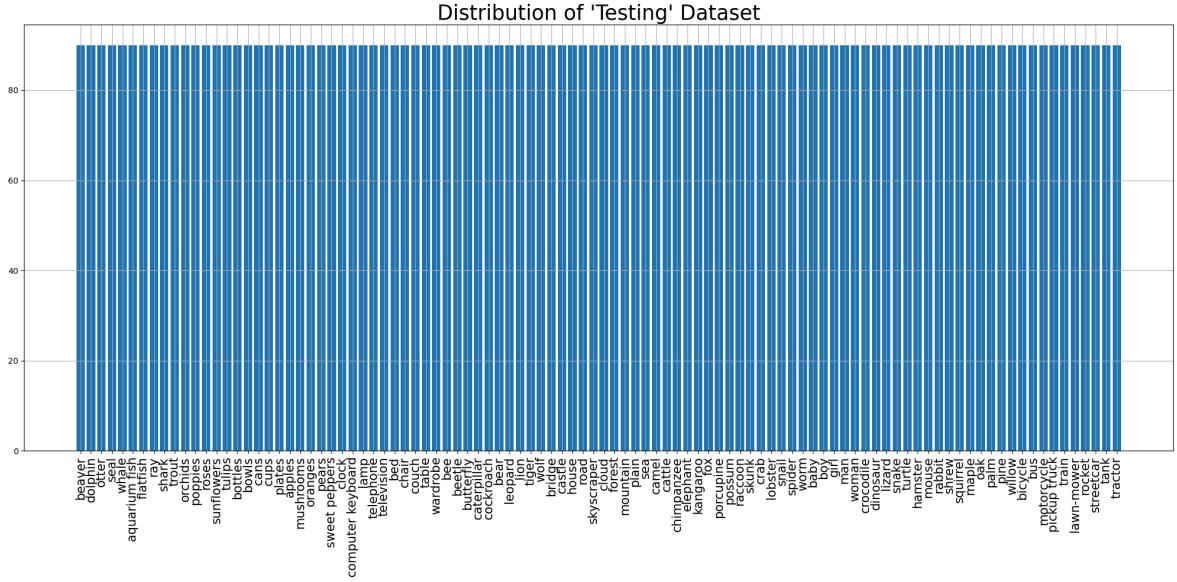

```
<ipython-input-5-791493d3fa27>:70: UserWarning: set_ticklabels() should only be u
sed with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```




```
<ipython-input-5-791493d3fa27>:70: UserWarning: set_ticklabels() should only be u
sed with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```




```
<ipython-input-5-791493d3fa27>:70: UserWarning: set_ticklabels() should only be u
sed with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
Training dataset:
 <_ConcatenateDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=t
f.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=Non
e))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)

Validation dataset:
 <_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.f
loat32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)

Testing dataset:
 <_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.f
loat32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.f
ramework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_3"

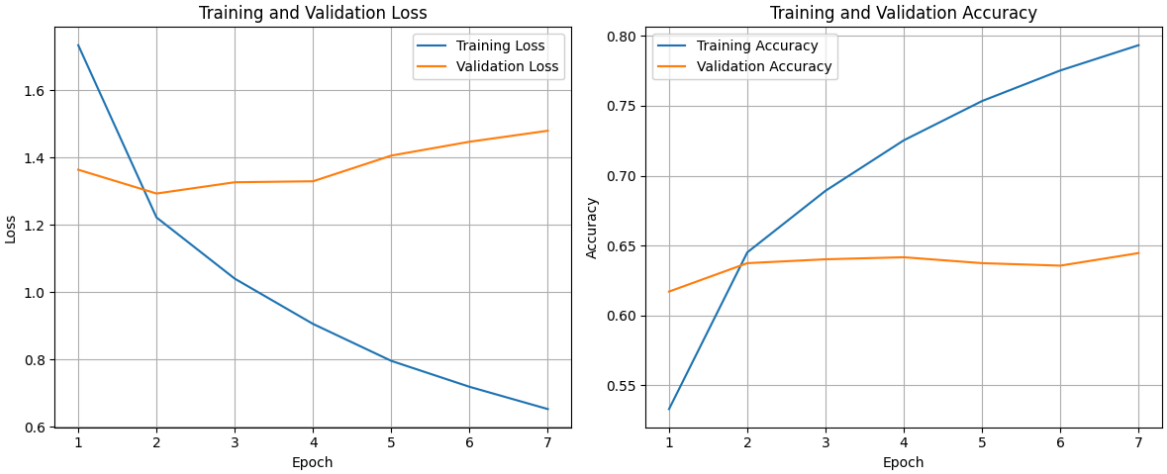
Layer (type)	Output Shape
resnet50v2 (Functional)	(None, 7, 7, 2048)
global_average_pooling2d_3 (GlobalAveragePooling2D)	(None, 2048)
dense_6 (Dense)	(None, 512)
dropout_3 (Dropout)	(None, 512)
dense_7 (Dense)	(None, 100)

Total params: 24,665,188 (94.09 MB)

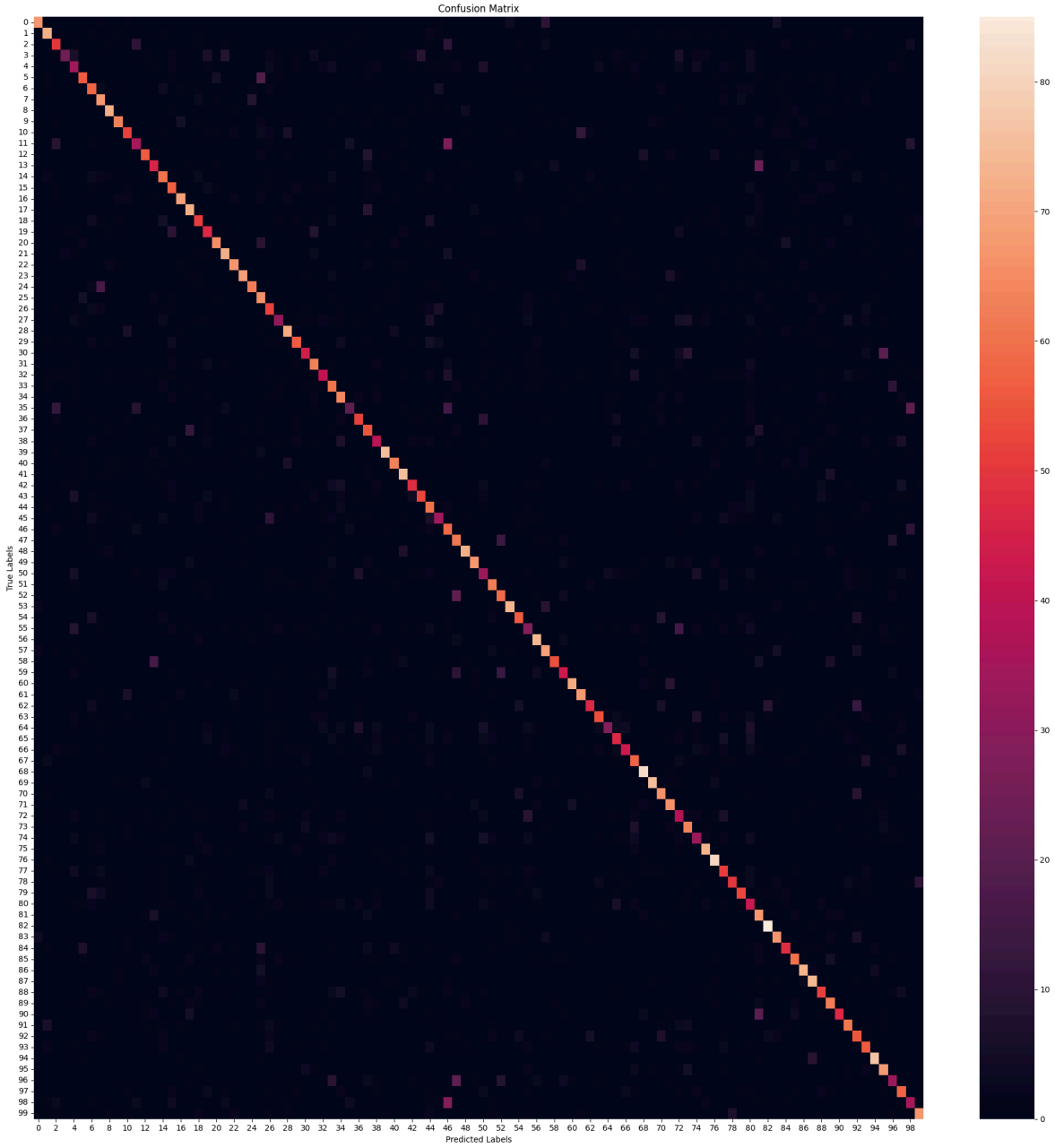
Trainable params: 1,100,388 (4.20 MB)

Non-trainable params: 23,564,800 (89.89 MB)

```
Epoch 1/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 58s 36ms/step - accuracy: 0.4473 - f1_score: 0.444
5 - loss: 2.1909 - precision: 0.7318 - val_accuracy: 0.6170 - val_f1_score: 0.613
3 - val_loss: 1.3642 - val_precision: 0.7960
Epoch 2/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 36s 28ms/step - accuracy: 0.6394 - f1_score: 0.637
0 - loss: 1.2537 - precision: 0.7963 - val_accuracy: 0.6373 - val_f1_score: 0.635
7 - val_loss: 1.2933 - val_precision: 0.7900
Epoch 3/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 36s 27ms/step - accuracy: 0.6872 - f1_score: 0.685
4 - loss: 1.0457 - precision: 0.8200 - val_accuracy: 0.6401 - val_f1_score: 0.637
8 - val_loss: 1.3269 - val_precision: 0.7772
Epoch 4/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 36s 27ms/step - accuracy: 0.7279 - f1_score: 0.726
3 - loss: 0.8914 - precision: 0.8365 - val_accuracy: 0.6416 - val_f1_score: 0.639
7 - val_loss: 1.3298 - val_precision: 0.7755
Epoch 5/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 35s 27ms/step - accuracy: 0.7600 - f1_score: 0.758
9 - loss: 0.7664 - precision: 0.8522 - val_accuracy: 0.6373 - val_f1_score: 0.636
3 - val_loss: 1.4062 - val_precision: 0.7510
Epoch 6/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 35s 27ms/step - accuracy: 0.7871 - f1_score: 0.785
9 - loss: 0.6754 - precision: 0.8631 - val_accuracy: 0.6356 - val_f1_score: 0.634
6 - val_loss: 1.4474 - val_precision: 0.7422
Epoch 7/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 35s 27ms/step - accuracy: 0.8076 - f1_score: 0.806
4 - loss: 0.6002 - precision: 0.8721 - val_accuracy: 0.6446 - val_f1_score: 0.643
8 - val_loss: 1.4801 - val_precision: 0.7335
```



```
141/141 ━━━━━━━━━━━━━━━━━━━━ 3s 24ms/step - accuracy: 0.6226 - f1_score: 0.6148 -
loss: 1.3204 - precision: 0.7714
Test Accuracy: 63.42%
Test Loss: 130.18%
Test Precision: 78.11%
Test F1 Scores (Per Class): [80.23952  77.41935  55.555553 39.669422 37.36263  6
6.666664 57.560966
 65.686264 80.       77.77777  61.90476  46.357613 70.44025  53.801163
 59.999996 56.71641  79.31035  73.09644  57.303364 52.222218 77.38094
 73.846146 77.906975 80.23255  75.60975  57.89473  49.999992 42.758617
 77.59562  67.06587  56.96202  67.37968  50.931675 55.299534 63.316578
 33.599995 61.53846  53.921562 47.27272  87.71929  71.99999  76.923065
 60.75949  65.03067  50.632904 45.94594  44.194756 56.22119  82.285706
 78.362564 35.955055 69.27374  63.783775 83.72092  64.739876 34.782604
 74.747475 70.76922  71.05263  55.128193 83.04093  74.725266 62.251644
 65.454544 44.094486 52.51396  55.84415  58.883244 82.       85.22727
 68.74999  71.35134  38.578674 66.31578  40.49079  80.66298  81.81817
 53.124992 64.51612  65.03067  37.66815  54.918026 88.08289  71.65774
 56.80473  67.796616 80.21977  76.28865  60.818714 63.589745 63.087242
 66.66666  54.99999  57.591618 86.03351  69.74358  40.243893 62.702698
 39.106144 76.57142 ]
Average Test F1 Scores:63.33

282/282 ━━━━━━━━━━━━━━━━━━━━ 12s 29ms/step
```


Confusion Matrix
True Labels
Predicted Labels

	precision	recall	f1-score	support
beaver	0.87	0.74	0.80	90
dolphin	0.75	0.80	0.77	90
otter	0.56	0.56	0.56	90
seal	0.77	0.27	0.40	90
whale	0.37	0.38	0.37	90
aquarium fish	0.72	0.62	0.67	90
flatfish	0.51	0.64	0.57	90
ray	0.59	0.74	0.66	90
shark	0.80	0.80	0.80	90
trout	0.88	0.70	0.78	90
orchids	0.68	0.58	0.62	90
poppies	0.57	0.39	0.46	90
roses	0.81	0.62	0.70	90
sunflowers	0.57	0.51	0.54	90
tulips	0.55	0.67	0.60	90
bottles	0.51	0.63	0.57	90
bowls	0.82	0.77	0.79	90
cans	0.67	0.80	0.73	90
cups	0.58	0.57	0.57	90
plates	0.52	0.52	0.52	90
apples	0.83	0.72	0.77	90
mushrooms	0.69	0.80	0.74	90
oranges	0.82	0.74	0.78	90
pears	0.84	0.77	0.80	90
sweet peppers	0.84	0.69	0.76	90
clock	0.48	0.73	0.58	90
computer keyboard	0.44	0.58	0.50	90
lamp	0.56	0.34	0.43	90
telephone	0.76	0.79	0.78	90
television	0.73	0.62	0.67	90
bed	0.67	0.50	0.57	90
chair	0.66	0.70	0.68	90
couch	0.59	0.46	0.51	90
table	0.47	0.67	0.55	90
wardrobe	0.58	0.71	0.64	90
bee	0.60	0.23	0.34	90
beetle	0.67	0.58	0.62	90
butterfly	0.48	0.61	0.54	90
caterpillar	0.52	0.43	0.47	90
cockroach	0.93	0.83	0.88	90
bear	0.74	0.70	0.72	90
leopard	0.71	0.83	0.77	90
lion	0.71	0.53	0.61	90
tiger	0.73	0.59	0.65	90
wolf	0.41	0.67	0.51	90
bridge	0.59	0.38	0.46	90
castle	0.33	0.66	0.44	90
house	0.48	0.68	0.56	90
road	0.85	0.80	0.82	90
skyscraper	0.83	0.74	0.78	90
cloud	0.37	0.37	0.37	90
forest	0.70	0.69	0.69	90
mountain	0.62	0.66	0.64	90
plain	0.88	0.81	0.84	90
sea	0.67	0.62	0.65	90
camel	0.39	0.31	0.35	90
cattle	0.69	0.82	0.75	90
chimpanzee	0.66	0.77	0.71	90

elephant	0.87	0.60	0.71	90
kangaroo	0.65	0.48	0.55	90
fox	0.88	0.79	0.83	90
porcupine	0.74	0.76	0.75	90
possum	0.77	0.52	0.62	90
raccoon	0.72	0.60	0.65	90
skunk	0.76	0.31	0.44	90
crab	0.53	0.52	0.53	90
lobster	0.67	0.48	0.56	90
snail	0.54	0.64	0.59	90
spider	0.75	0.91	0.82	90
worm	0.87	0.83	0.85	90
baby	0.64	0.73	0.68	90
boy	0.69	0.73	0.71	90
girl	0.35	0.42	0.38	90
man	0.63	0.70	0.66	90
woman	0.45	0.37	0.40	90
crocodile	0.80	0.81	0.81	90
dinosaur	0.75	0.90	0.82	90
lizard	0.50	0.57	0.53	90
snake	0.77	0.56	0.65	90
turtle	0.73	0.59	0.65	90
hamster	0.32	0.47	0.38	90
mouse	0.44	0.74	0.55	90
rabbit	0.83	0.94	0.88	90
shrew	0.69	0.74	0.72	90
squirrel	0.61	0.53	0.57	90
maple	0.69	0.67	0.68	90
oak	0.79	0.81	0.80	90
palm	0.71	0.82	0.76	90
pine	0.64	0.58	0.61	90
willow	0.59	0.69	0.64	90
bicycle	0.80	0.52	0.63	90
bus	0.66	0.68	0.67	90
motorcycle	0.50	0.61	0.55	90
pickup truck	0.55	0.62	0.59	90
train	0.87	0.86	0.86	90
lawn-mower	0.65	0.76	0.70	90
rocket	0.45	0.37	0.40	90
streetcar	0.61	0.64	0.63	90
tank	0.39	0.39	0.39	90
tractor	0.78	0.74	0.76	90
accuracy			0.63	9000
macro avg	0.65	0.63	0.63	9000
weighted avg	0.65	0.63	0.63	9000

Model: "sequential_3"

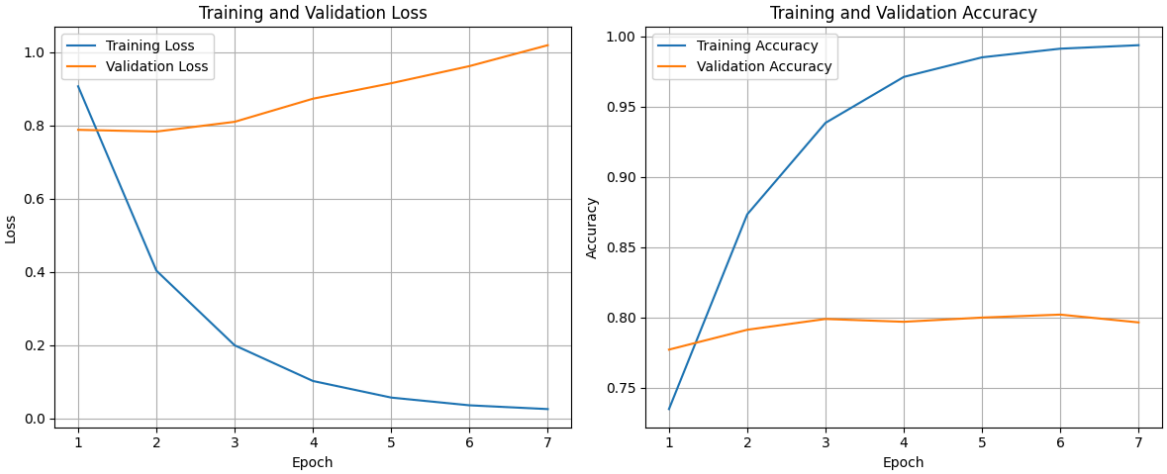
Layer (type)	Output Shape
resnet50v2 (Functional)	(None, 7, 7, 2048)
global_average_pooling2d_3 (GlobalAveragePooling2D)	(None, 2048)
dense_6 (Dense)	(None, 512)
dropout_3 (Dropout)	(None, 512)
dense_7 (Dense)	(None, 100)

Total params: 24,665,188 (94.09 MB)

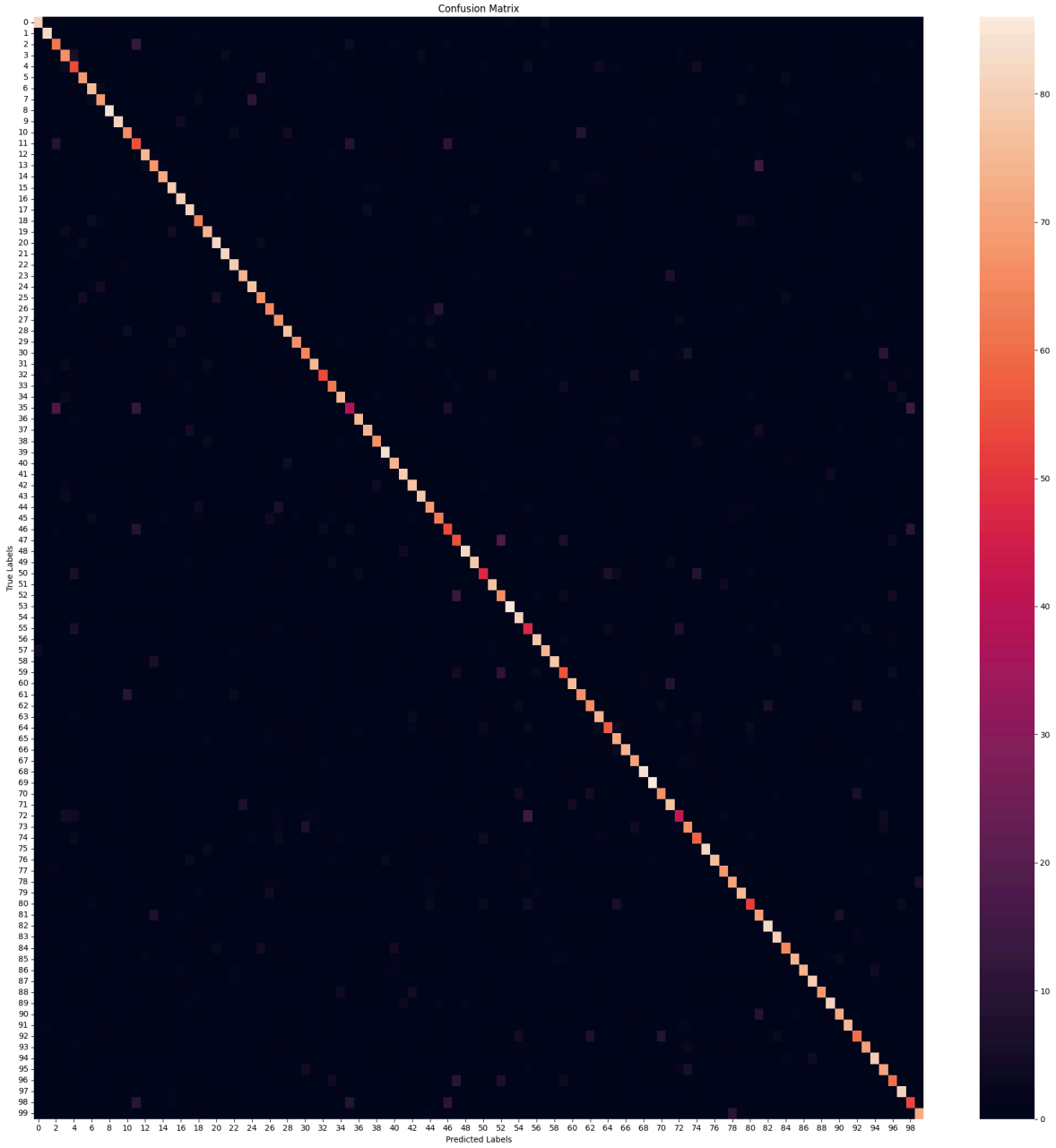
Trainable params: 24,619,748 (93.92 MB)

Non-trainable params: 45,440 (177.50 KB)

```
Epoch 1/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 210s 81ms/step - accuracy: 0.6557 - f1_score: 0.65
48 - loss: 1.2315 - precision: 0.8172 - val_accuracy: 0.7772 - val_f1_score: 0.77
63 - val_loss: 0.7880 - val_precision: 0.8452
Epoch 2/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 135s 69ms/step - accuracy: 0.8569 - f1_score: 0.85
58 - loss: 0.4583 - precision: 0.9140 - val_accuracy: 0.7913 - val_f1_score: 0.79
08 - val_loss: 0.7833 - val_precision: 0.8427
Epoch 3/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 135s 69ms/step - accuracy: 0.9357 - f1_score: 0.93
50 - loss: 0.2084 - precision: 0.9596 - val_accuracy: 0.7990 - val_f1_score: 0.79
87 - val_loss: 0.8099 - val_precision: 0.8365
Epoch 4/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 135s 68ms/step - accuracy: 0.9724 - f1_score: 0.97
19 - loss: 0.0981 - precision: 0.9812 - val_accuracy: 0.7970 - val_f1_score: 0.79
71 - val_loss: 0.8730 - val_precision: 0.8275
Epoch 5/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 135s 68ms/step - accuracy: 0.9879 - f1_score: 0.98
73 - loss: 0.0504 - precision: 0.9907 - val_accuracy: 0.8000 - val_f1_score: 0.79
99 - val_loss: 0.9152 - val_precision: 0.8259
Epoch 6/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 135s 68ms/step - accuracy: 0.9932 - f1_score: 0.99
27 - loss: 0.0300 - precision: 0.9946 - val_accuracy: 0.8021 - val_f1_score: 0.80
19 - val_loss: 0.9619 - val_precision: 0.8234
Epoch 7/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 135s 68ms/step - accuracy: 0.9956 - f1_score: 0.99
51 - loss: 0.0203 - precision: 0.9962 - val_accuracy: 0.7966 - val_f1_score: 0.79
66 - val_loss: 1.0190 - val_precision: 0.8158
```




```
141/141 ━━━━━━━━━━━━━━━━━━━━ 3s 24ms/step - accuracy: 0.7800 - f1_score: 0.7726 -
loss: 0.7727 - precision: 0.8386
Test Accuracy: 78.62%
Test Loss: 77.17%
Test Precision: 84.28%
Test F1 Scores (Per Class): [89.99999  90.71037  66.31015  72.82608  57.4468   7
8.85714  80.85106
 75.82417  94.38202  92.57143  73.74301  56.994812 81.52173  77.71428
 80.446915 82.291664 84.656075 86.63101  69.23077  79.99999  90.10989
 91.6201   86.81318  84.090904 83.24323  74.444435 71.428566 72.04301
 85.08286  80.722885 73.863625 84.26965  69.67742  69.27374  82.41757
 48.407635 86.20689  83.14606  77.01149  91.803276 81.31868  86.95651
 81.91489  86.666664 73.79678  69.61325  58.69565  61.452507 91.6201
 88.13559  58.895695 82.35293  68.74999  92.89617  81.81817  49.735443
 86.666664 82.68156  88.63635  65.88235  84.91619  77.01149  74.57626
 79.14438  65.89595  74.6114   86.549706 78.21229  91.803276 91.00529
 77.906975 79.79274  50.299393 75.28088  64.80447  91.11111  84.44444
 78.61271  79.329605 79.787224 60.1156   72.16493  90.217384 85.263145
 74.71264  85.549126 88.09524  90.39548  81.9277   88.52458  81.81818
 85.22727  68.965515 80.       87.43169  77.173904 67.415726 85.71427
 59.550552 83.236984]
Average Test F1 Scores:78.56

282/282 ━━━━━━━━━━━━━━━━━━━━ 12s 28ms/step
```


Confusion Matrix
True Labels
Predicted Labels

	precision	recall	f1-score	support
beaver	0.90	0.90	0.90	90
dolphin	0.89	0.92	0.91	90
otter	0.64	0.69	0.66	90
seal	0.71	0.74	0.73	90
whale	0.55	0.60	0.57	90
aquarium fish	0.81	0.77	0.79	90
flatfish	0.78	0.84	0.81	90
ray	0.75	0.77	0.76	90
shark	0.95	0.93	0.94	90
trout	0.95	0.90	0.93	90
orchids	0.74	0.73	0.74	90
poppies	0.53	0.61	0.57	90
roses	0.80	0.83	0.82	90
sunflowers	0.80	0.76	0.78	90
tulips	0.81	0.80	0.80	90
bottles	0.77	0.88	0.82	90
bowls	0.81	0.89	0.85	90
cans	0.84	0.90	0.87	90
cups	0.68	0.70	0.69	90
plates	0.78	0.82	0.80	90
apples	0.89	0.91	0.90	90
mushrooms	0.92	0.91	0.92	90
oranges	0.86	0.88	0.87	90
pears	0.86	0.82	0.84	90
sweet peppers	0.81	0.86	0.83	90
clock	0.74	0.74	0.74	90
computer keyboard	0.71	0.72	0.71	90
lamp	0.70	0.74	0.72	90
telephone	0.85	0.86	0.85	90
television	0.88	0.74	0.81	90
bed	0.76	0.72	0.74	90
chair	0.85	0.83	0.84	90
couch	0.83	0.60	0.70	90
table	0.70	0.69	0.69	90
wardrobe	0.82	0.83	0.82	90
bee	0.57	0.42	0.48	90
beetle	0.89	0.83	0.86	90
butterfly	0.84	0.82	0.83	90
caterpillar	0.80	0.74	0.77	90
cockroach	0.90	0.93	0.92	90
bear	0.80	0.82	0.81	90
leopard	0.85	0.89	0.87	90
lion	0.79	0.86	0.82	90
tiger	0.87	0.87	0.87	90
wolf	0.71	0.77	0.74	90
bridge	0.69	0.70	0.70	90
castle	0.57	0.60	0.59	90
house	0.62	0.61	0.61	90
road	0.92	0.91	0.92	90
skyscraper	0.90	0.87	0.88	90
cloud	0.66	0.53	0.59	90
forest	0.79	0.86	0.82	90
mountain	0.65	0.73	0.69	90
plain	0.91	0.94	0.93	90
sea	0.75	0.90	0.82	90
camel	0.47	0.52	0.50	90
cattle	0.87	0.87	0.87	90
chimpanzee	0.83	0.82	0.83	90

elephant	0.91	0.87	0.89	90
kangaroo	0.70	0.62	0.66	90
fox	0.85	0.84	0.85	90
porcupine	0.80	0.74	0.77	90
possum	0.76	0.73	0.75	90
raccoon	0.76	0.82	0.79	90
skunk	0.69	0.63	0.66	90
crab	0.70	0.80	0.75	90
lobster	0.91	0.82	0.87	90
snail	0.79	0.78	0.78	90
spider	0.90	0.93	0.92	90
worm	0.87	0.96	0.91	90
baby	0.82	0.74	0.78	90
boy	0.75	0.86	0.80	90
girl	0.55	0.47	0.50	90
man	0.76	0.74	0.75	90
woman	0.65	0.64	0.65	90
crocodile	0.91	0.91	0.91	90
dinosaur	0.84	0.84	0.84	90
lizard	0.82	0.76	0.79	90
snake	0.80	0.79	0.79	90
turtle	0.77	0.83	0.80	90
hamster	0.63	0.58	0.60	90
mouse	0.67	0.78	0.72	90
rabbit	0.88	0.92	0.90	90
shrew	0.81	0.90	0.85	90
squirrel	0.77	0.72	0.75	90
maple	0.89	0.82	0.86	90
oak	0.95	0.82	0.88	90
palm	0.92	0.89	0.90	90
pine	0.89	0.76	0.82	90
willow	0.87	0.90	0.89	90
bicycle	0.84	0.80	0.82	90
bus	0.87	0.83	0.85	90
motorcycle	0.71	0.67	0.69	90
pickup truck	0.82	0.78	0.80	90
train	0.86	0.89	0.87	90
lawn-mower	0.76	0.79	0.77	90
rocket	0.68	0.67	0.67	90
streetcar	0.82	0.90	0.86	90
tank	0.60	0.59	0.60	90
tractor	0.87	0.80	0.83	90
accuracy			0.79	9000
macro avg	0.79	0.79	0.79	9000
weighted avg	0.79	0.79	0.79	9000

224x224 with more dropout

In [6]:
```
for repeat_2_times in range(2):
    #### <<<<<<<<<<Load and process data>>>>>>>>>>
    # Load CIFAR-100 dataset
    (X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

    # Split (8000) of training data into temporary set
    X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_s
    print(f"X_temp.shape: {X_temp.shape}\n")
```

```
# Split temp data into equal validation (4000) and testing (4000) data
X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp,
print(f"X_temp_val.shape: {X_temp_val.shape}")
print(f"y_temp_val.shape: {y_temp_val.shape}")
print(f"X_temp_test.shape: {X_temp_test.shape}")
print(f"y_temp_test.shape: {y_temp_test.shape}\n")

# Split test data into validation (5000) and testing (5000)
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.

# Add temp_val to validation (9000) and temp_test to testing (9000) to get a
X_val = np.concatenate((X_val, X_temp_val), axis=0)
y_val = np.concatenate((y_val, y_temp_val), axis=0)
X_test = np.concatenate((X_test, X_temp_test), axis=0)
y_test = np.concatenate((y_test, y_temp_test), axis=0)

print(f"X_train.shape: {X_train.shape}")
print(f"y_train.shape: {y_train.shape}")
print(f"X_val.shape: {X_val.shape}")
print(f"y_val.shape: {y_val.shape}")
print(f"X_test.shape: {X_test.shape}")
print(f"y_test.shape: {y_test.shape}\n")

def display_imgs(imgs, labels):
    plt.subplots(figsize=(10,10))
    for i in range(16):
        plt.subplot(4, 4, i+1)
        k = np.random.randint(0, imgs.shape[0])
        if i == 0:
            print(f"labels[{k}].shape: {labels[k].shape}")
            print(f"imgs[{k}].shape: {imgs[k].shape}")
        plt.imshow(imgs[k])
        #plt.title(labels[k])
        plt.axis('off')
    plt.tight_layout()
    plt.show()

display_imgs(X_train, y_train)

# Normalise images (scale to range [0, 1]) - Improves convergence speed & ac
X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
display_imgs(X_train, y_train)

labels_names = ['beaver','dolphin','otter','seal','whale','aquarium fish','f
                'orchids','poppies','roses','sunflowers','tulips','bottles','
                'apples','mushrooms','oranges','pears','sweet peppers','clock
                'telephone','television','bed','chair','couch','table','wardr
                'caterpillar','cockroach','bear','leopard','lion','tiger','wo
                'road','skyscraper','cloud','forest','mountain','plain','sea'
                'elephant','kangaroo','fox','porcupine','possum','raccoon','s
                'spider','worm','baby','boy','girl','man','woman','crocodile'
                'turtle','hamster','mouse','rabbit','shrew','squirrel','maple
                'bicycle','bus','motorcycle','pickup truck','train','lawn-mow
                'tractor']

def class_distrib(y, labels_names, dataset_name):
    counts = pd.DataFrame(data=y).value_counts().sort_index()
    #print(f"counts:\n{counts}")
    fig, ax = plt.subplots(figsize=(20,10))
```

```
        ax.bar(labels_names, counts)
        ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
        plt.title(f"Distribution of '{dataset_name}' Dataset", fontsize=25)
        plt.grid()
        plt.tight_layout()
        plt.show()
    class_distrib(y_train, labels_names, "Training")
    class_distrib(y_val, labels_names, "Validating")
    class_distrib(y_test, labels_names, "Testing")

    # Create TensorFlow datasets

    batch_size = 64
    train_dataset_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                    .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                       tf.squeeze(tf.one_hot(y, depth=100, dtyp
                    .batch(batch_size)
                    .prefetch(tf.data.experimental.AUTOTUNE))

    def augment_dataset(x, y):
        x = tf.image.resize(x, (224, 224))  # Resize images
        x = tf.image.random_flip_left_right(x)  # Random horizontal flip
        x = tf.image.random_brightness(x, max_delta=0.2)  # Adjust brightness
        x = tf.image.random_contrast(x, lower=0.8, upper=1.2)  # Adjust contrast
        y = tf.squeeze(tf.one_hot(y, depth=100, dtype=tf.float32))  # One-hot en
        return x, y
    train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                    .map(augment_dataset, num_parallel_calls=tf.data.experiment
                    .batch(batch_size)
                    .prefetch(tf.data.experimental.AUTOTUNE))

    # Combine the original dataset and the augmented dataset
    train_dataset = train_dataset_.concatenate(train_dataset_aug)

    # Optionally, shuffle the combined dataset if needed
    #combined_dataset = combined_dataset.shuffle(buffer_size=1000)  # Adjust buf

    val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
                   .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                      tf.squeeze(tf.one_hot(y, depth=100, dtype=
                   .batch(batch_size)
                   .prefetch(tf.data.experimental.AUTOTUNE))

    test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
                     .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                        tf.squeeze(tf.one_hot(y, depth=100, dtyp
                    .batch(batch_size)
                    .prefetch(tf.data.experimental.AUTOTUNE))

    print(f"Training dataset:\n {train_dataset}")
    for img, lbl in train_dataset.take(1):
        #if isinstance(batch, tuple) and len(batch) == 2:
        print(f"Image shape: {img.shape}")  # Should be (batch_size, 224, 224, 3
        print(f"Label shape: {lbl.shape}")  # Should be (batch_size, 10)
        del img,lbl
    print(f"\nValidation dataset:\n {val_dataset}")
    for img, lbl in val_dataset.take(1):
        #if isinstance(batch, tuple) and len(batch) == 2:
        print(f"Image shape: {img.shape}")  # Should be (batch_size, 224, 224, 3
        print(f"Label shape: {lbl.shape}")  # Should be (batch_size, 10)
```

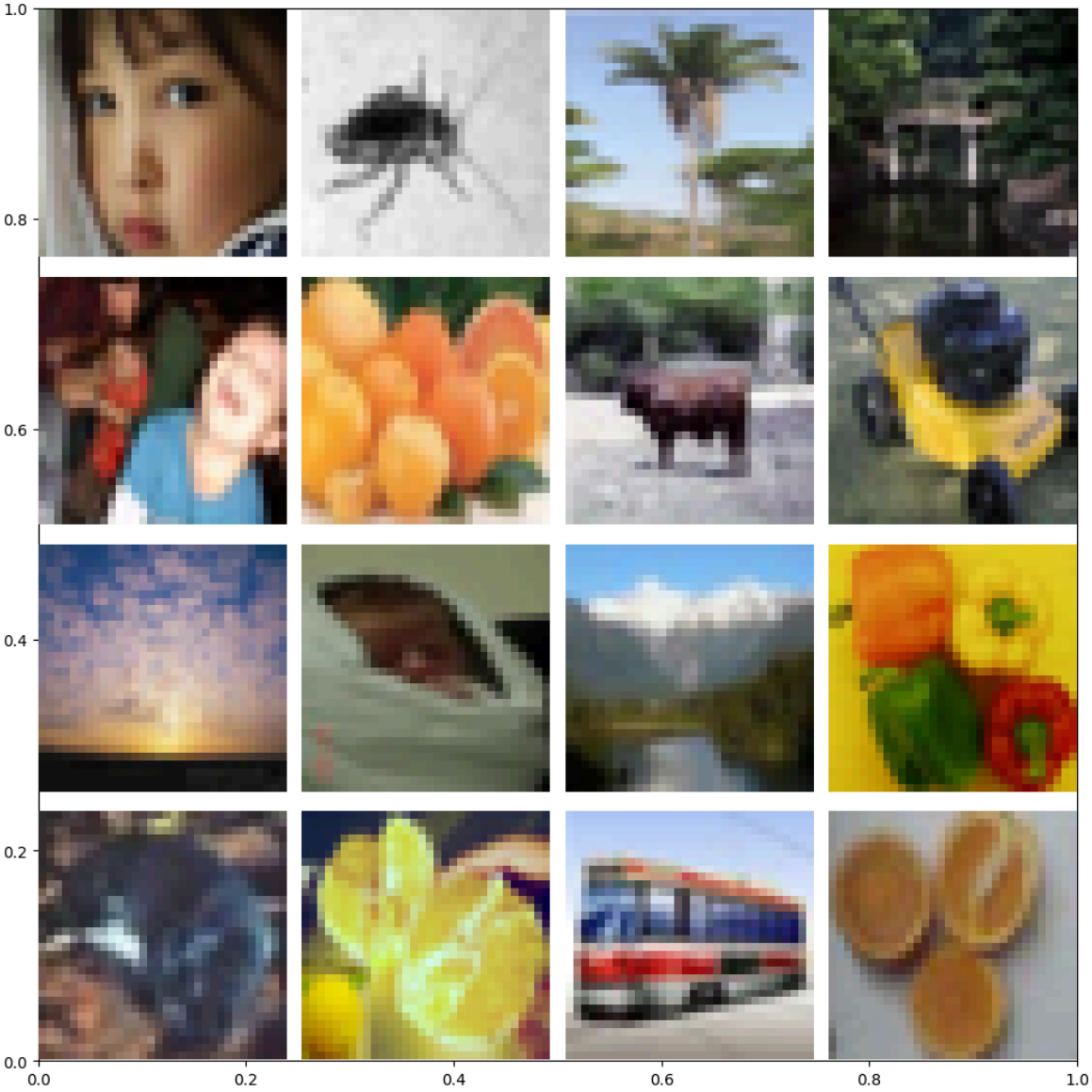
```
    del img,lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}")  # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}")  # Should be (batch_size, 10)
    del img,lbl


#### <<<<<<<<<<Pre-trained model>>>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

# Freeze the layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.6),
    layers.Dense(100, activation='softmax')  # CIFAR-100 has 100 classes
])

for sample in test_dataset.take(1):
    print(type(sample))  # Should be <class 'tuple'>
    print(len(sample))  # Should be 2
    print(type(sample[0]), type(sample[1]))  # Both should be <class 'tensor
    print(sample[0].shape)  # Should be (batch_size, 224, 224, 3)
    print(sample[1].shape)  # Should be (batch_size, 100)
print(f"Model input shape: {model.input_shape}")
print(f"Model output shape: {model.output_shape}")
sample = next(iter(test_dataset.as_numpy_iterator()))
print(len(sample))  # Should be 2
print(type(sample[0]), type(sample[1]))  # Both should be <class 'numpy.ndar
print(sample[0].shape, sample[1].shape)  # Should match model input and outp
print("\n")
#for x, y in test_dataset.take(1):
#    print(type(x), type(y))  # Both should be <class 'tensorflow.Tensor'>
#for x_batch, y_batch in test_dataset.take(1):
#    test_loss, test_acc = model.evaluate(x_batch, y_batch)
#    print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

# Compile the model
#tensorboard_callback = keras.callbacks.TensorBoard(log_dir="./logs")
model.compile(optimizer=optimizers.Adam(learning_rate=1e-3),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>
early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
                               patience=5, # Num. epochs with no improvement
                               restore_best_weights=True)
#reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
#                              factor=0.1, # Reduce lr by a factor
#                              patience=3, # Num epochs w/ no improvement
#                              min_lr=1e-6, # Min lr
```

```
#                                verbose=1)
#tensorboard = TensorBoard(log_dir='./logs', # Logs directory
#                           histogram_freq=1, # Logs histograms for weights/ac
#                           write_graph=True, # Logs graph of model
#                           write_images=True) # Log images like weight histog
#checkpoint = ModelCheckpoint('best_model.h5',
#                             monitor='val_loss', # or val_accuracy
#                             save_best_only=True, # Save only best model
#                             mode='min', # min for loss or max for accuracy
#                             verbose=1)
#cvs_logger = CSVLogger('training_log.csv', seperator=',', append=True) # Sa

# Train the model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
                    batch_size=batch_size, callbacks=[early_stopping], verbo

#### <<<<<<<<<<Plot Training & Validation Error>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1,len(history.history['loss'])+1)
train_loss = history.history['loss']
val_loss = history.history['val_loss']
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

def plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc):
    # Plot Training and Validation Loss
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.plot(epochs, train_loss, label='Training Loss')
    plt.plot(epochs, val_loss, label='Validation Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Training and Validation Loss')
    plt.legend()
    plt.grid()

    # Plot Training and Validation Accuracy
    plt.subplot(1, 2, 2)
    plt.plot(epochs, train_acc, label='Training Accuracy')
    plt.plot(epochs, val_acc, label='Validation Accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.title('Training and Validation Accuracy')
    plt.legend()
    plt.grid()

    plt.tight_layout()
    plt.show()

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>

# Evaluate on test data
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
```

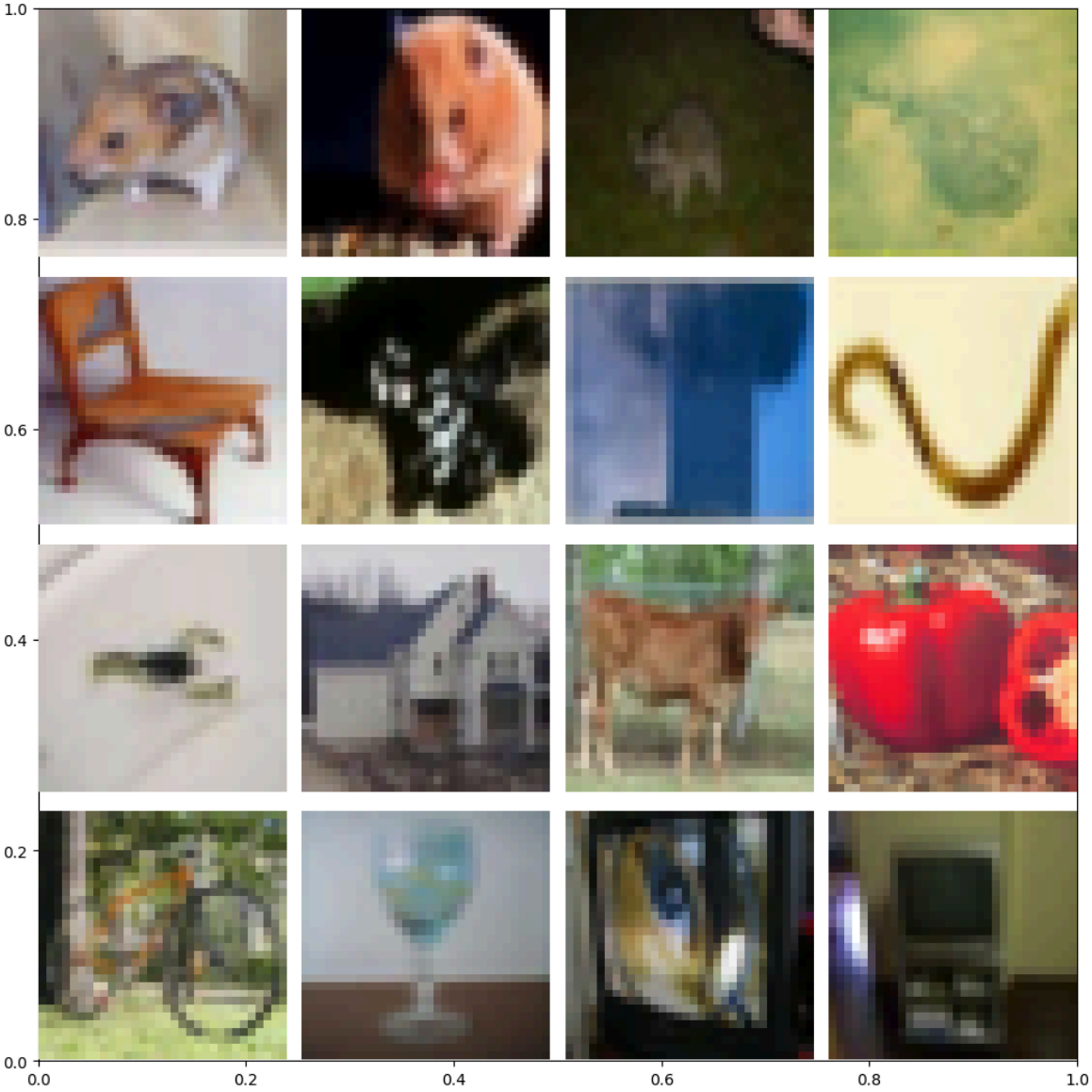
```
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss*100:.2f}%")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<<<Generate Confusion Matrix>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<<<Fine-Tune>>>>>>>>>>
#### <<<<<<<<<<Adapt Model>>>>>>>>>>
# Unfreeze last 10 layers
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ lower learning rate (prevents destroying learned features
model.compile(optimizer=optimizers.Adam(learning_rate=1e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<<<Modify Dataset>>>>>>>>>>
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                .map(augment_dataset, num_parallel_calls=tf.data.experiment
                .batch(batch_size)
                .prefetch(tf.data.experimental.AUTOTUNE))
train_dataset_aug = train_dataset.concatenate(train_dataset_aug)

# Not val or test as augment train helps generalise better, but want to prov

#### <<<<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>
```

```
# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset
                              batch_size=batch_size, callbacks=[early_stoppi

#### <<<<<<<<<<Plot Training & Validation Error>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1,len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>

# Evaluate on test data
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss*100:.2f}%")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<<<Generate Confusion Matrix>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

# Finally, display the hist DataFrame.
hist
```

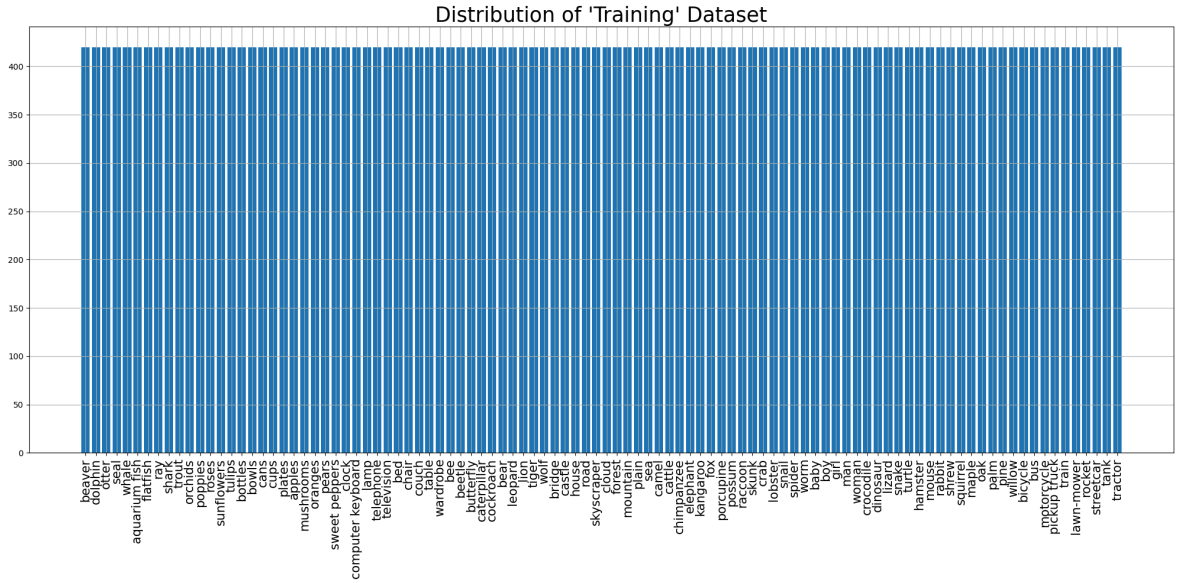
```
X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)
y_temp_val.shape: (4000, 1)
X_temp_test.shape: (4000, 32, 32, 3)
y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)
y_train.shape: (42000, 1)
X_val.shape: (9000, 32, 32, 3)
y_val.shape: (9000, 1)
X_test.shape: (9000, 32, 32, 3)
y_test.shape: (9000, 1)

labels[13806].shape: (1,)
imgs[13806].shape: (32, 32, 3)
```



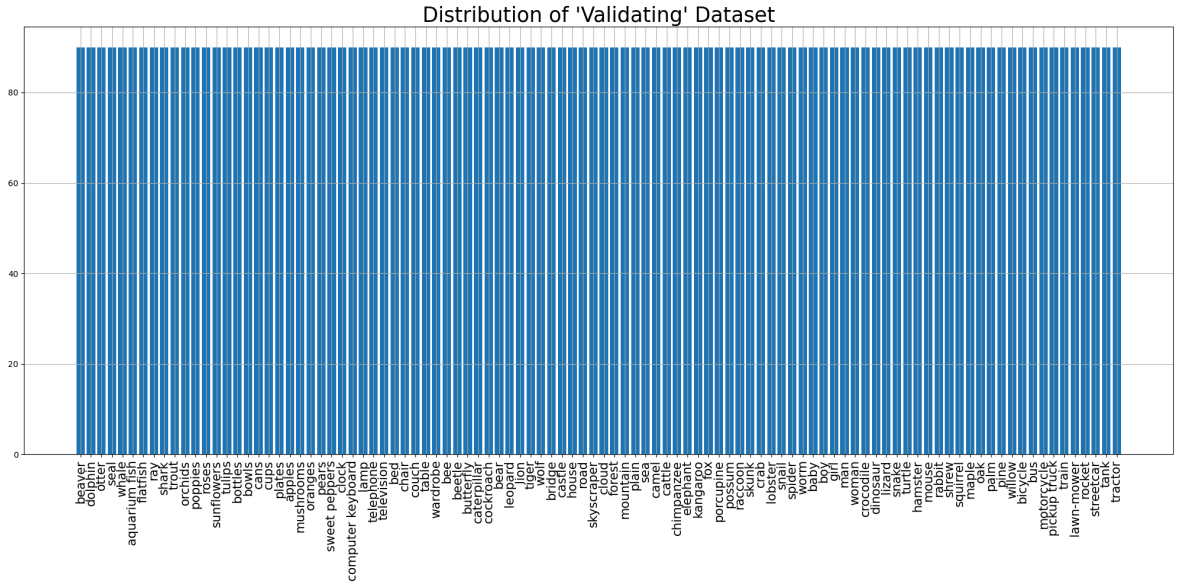

```
labels[28649].shape: (1,)
imgs[28649].shape: (32, 32, 3)
```



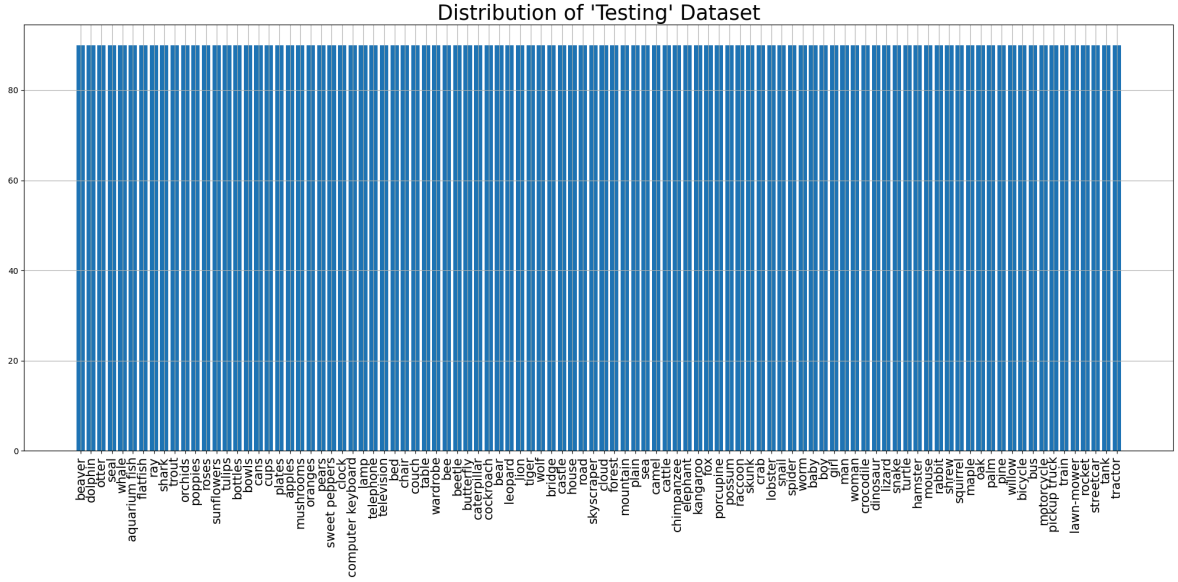

```
<ipython-input-6-1810f983c3fb>:70: UserWarning: set_ticklabels() should only be u
sed with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```




```
<ipython-input-6-1810f983c3fb>:70: UserWarning: set_ticklabels() should only be u
sed with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```




```
<ipython-input-6-1810f983c3fb>:70: UserWarning: set_ticklabels() should only be u
sed with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
Training dataset:
 <_ConcatenateDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=t
f.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=Non
e))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)

Validation dataset:
 <_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.f
loat32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)

Testing dataset:
 <_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.f
loat32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.f
ramework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_4"

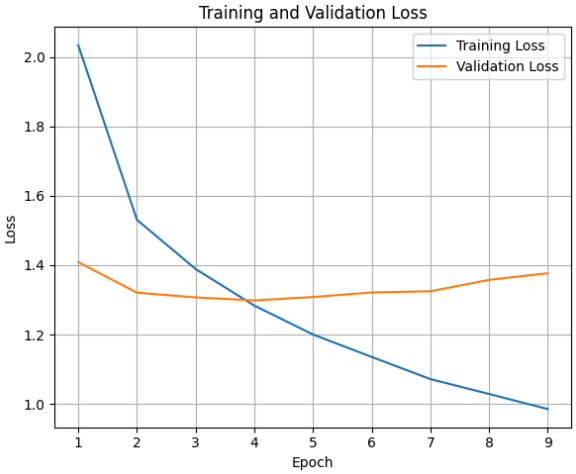
Layer (type)	Output Shape
resnet50v2 (Functional)	(None, 7, 7, 2048)
global_average_pooling2d_4 (GlobalAveragePooling2D)	(None, 2048)
dense_8 (Dense)	(None, 512)
dropout_4 (Dropout)	(None, 512)
dense_9 (Dense)	(None, 100)

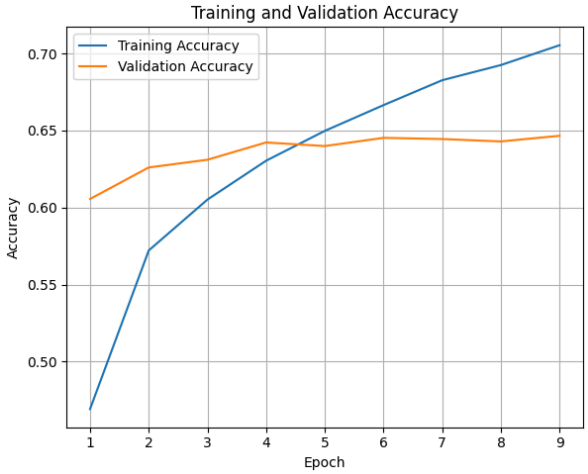
Total params: 24,665,188 (94.09 MB)

Trainable params: 1,100,388 (4.20 MB)

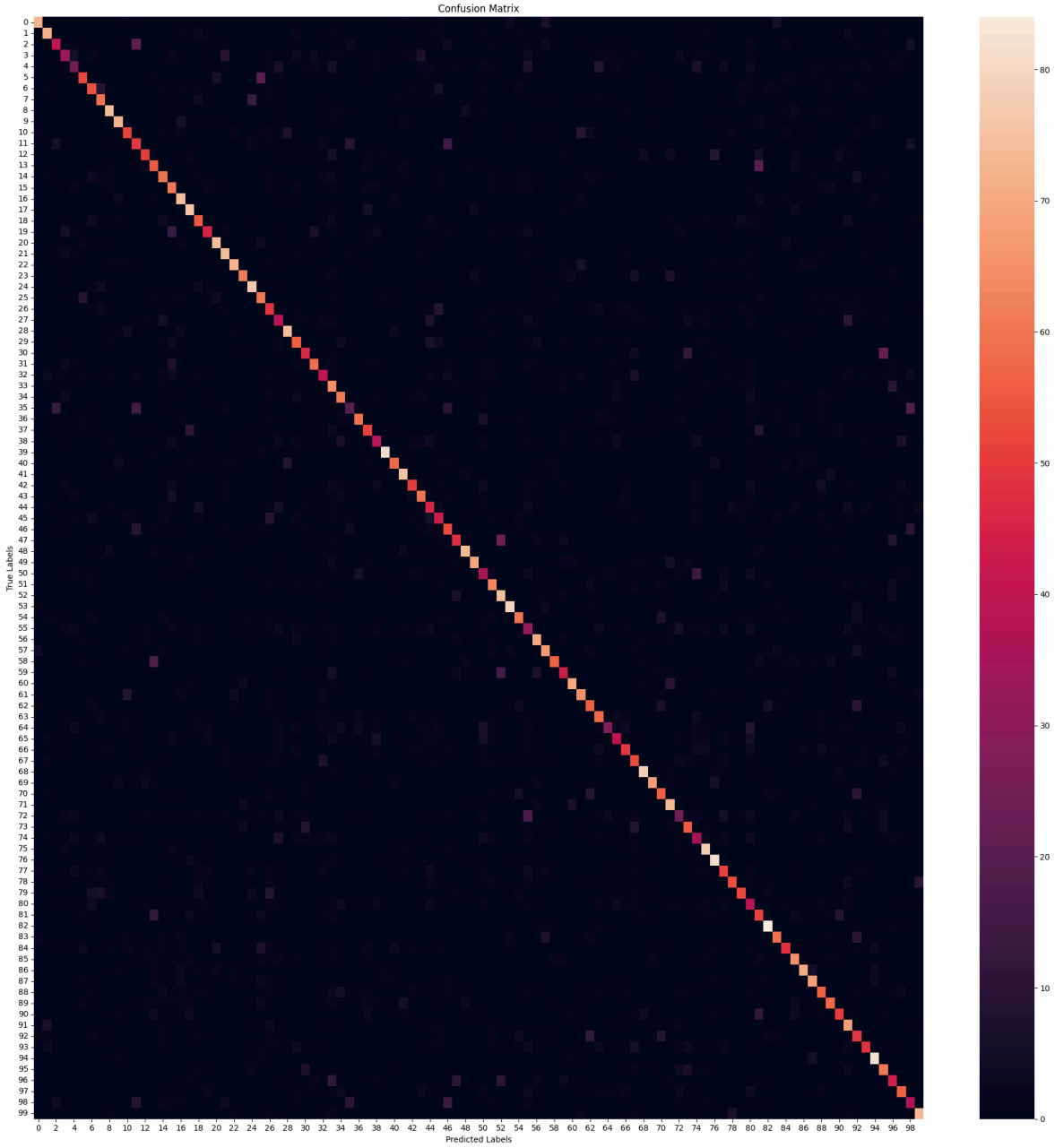
Non-trainable params: 23,564,800 (89.89 MB)

```
Epoch 1/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 55s 34ms/step - accuracy: 0.3834 - f1_score: 0.379
2 - loss: 2.5305 - precision: 0.6899 - val_accuracy: 0.6056 - val_f1_score: 0.602
4 - val_loss: 1.4087 - val_precision: 0.7995
Epoch 2/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 34s 26ms/step - accuracy: 0.5644 - f1_score: 0.561
7 - loss: 1.5599 - precision: 0.7566 - val_accuracy: 0.6260 - val_f1_score: 0.622
3 - val_loss: 1.3208 - val_precision: 0.8058
Epoch 3/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 34s 26ms/step - accuracy: 0.6024 - f1_score: 0.599
6 - loss: 1.3933 - precision: 0.7774 - val_accuracy: 0.6310 - val_f1_score: 0.628
1 - val_loss: 1.3071 - val_precision: 0.8019
Epoch 4/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 35s 26ms/step - accuracy: 0.6314 - f1_score: 0.629
3 - loss: 1.2743 - precision: 0.7877 - val_accuracy: 0.6422 - val_f1_score: 0.641
0 - val_loss: 1.2984 - val_precision: 0.7923
Epoch 5/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 34s 26ms/step - accuracy: 0.6533 - f1_score: 0.651
1 - loss: 1.1848 - precision: 0.7973 - val_accuracy: 0.6399 - val_f1_score: 0.639
6 - val_loss: 1.3083 - val_precision: 0.7867
Epoch 6/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 34s 26ms/step - accuracy: 0.6715 - f1_score: 0.669
8 - loss: 1.1090 - precision: 0.8041 - val_accuracy: 0.6452 - val_f1_score: 0.644
4 - val_loss: 1.3214 - val_precision: 0.7748
Epoch 7/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 34s 26ms/step - accuracy: 0.6898 - f1_score: 0.687
9 - loss: 1.0368 - precision: 0.8124 - val_accuracy: 0.6444 - val_f1_score: 0.642
7 - val_loss: 1.3248 - val_precision: 0.7814
Epoch 8/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 35s 26ms/step - accuracy: 0.7029 - f1_score: 0.701
3 - loss: 0.9807 - precision: 0.8214 - val_accuracy: 0.6429 - val_f1_score: 0.643
2 - val_loss: 1.3577 - val_precision: 0.7702
Epoch 9/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 34s 26ms/step - accuracy: 0.7147 - f1_score: 0.713
0 - loss: 0.9382 - precision: 0.8232 - val_accuracy: 0.6466 - val_f1_score: 0.645
8 - val_loss: 1.3768 - val_precision: 0.7628
```





```
141/141 ━━━━━━━━━━━━━━━━━━━━ 3s 23ms/step - accuracy: 0.6292 - f1_score: 0.6210 -
loss: 1.3223 - precision: 0.7796
Test Accuracy: 63.70%
Test Loss: 129.90%
Test Precision: 79.07%
Test F1 Scores (Per Class): [82.285706 75.93583  50.306744 43.137253 32.653057 6
3.030296 59.016384
 64.17112  79.99999  80.89887  61.445778 49.999992 64.51612  56.701027
 63.82978  57.547165 76.28865  76.381905 59.7826   56.249992 80.87432
 77.08333  82.75862  72.94117  79.38144  60.396034 47.999996 42.391296
 81.52173  60.540535 56.44171  67.04545  49.079746 51.821865 65.59139
 28.169008 72.289154 60.355026 49.056595 91.01122  70.303024 80.85106
 63.291134 68.181816 46.39175  48.863632 52.525253 58.024685 82.02247
 80.23255  40.677963 67.74193  68.51852  87.77777  67.0391   32.653057
 74.46808  70.526306 74.509796 54.08804  79.54544  71.428566 62.222218
 61.702126 40.90909  49.397583 60.49382  55.208332 84.7826   80.95238
 63.276833 71.92117  30.967735 60.773476 37.837833 79.59184  80.597
 56.98323  67.92453  65.       37.681156 47.887318 84.84848  71.00591
 60.75949  67.01031  80.       76.40449  62.222218 67.44185  59.171593
 62.96296  47.572807 62.745094 85.41666  61.928932 48.61878  64.
 41.57303  79.347824]
Average Test F1 Scores:63.50

282/282 ━━━━━━━━━━━━━━━━━━━━ 11s 28ms/step
```


Confusion Matrix
True Labels
Predicted Labels

	precision	recall	f1-score	support
beaver	0.85	0.80	0.82	90
dolphin	0.73	0.79	0.76	90
otter	0.56	0.46	0.50	90
seal	0.52	0.37	0.43	90
whale	0.42	0.27	0.33	90
aquarium fish	0.69	0.58	0.63	90
flatfish	0.58	0.60	0.59	90
ray	0.62	0.67	0.64	90
shark	0.78	0.82	0.80	90
trout	0.82	0.80	0.81	90
orchids	0.67	0.57	0.61	90
poppies	0.46	0.54	0.50	90
roses	0.77	0.56	0.65	90
sunflowers	0.53	0.61	0.57	90
tulips	0.61	0.67	0.64	90
bottles	0.50	0.68	0.58	90
bowls	0.70	0.82	0.76	90
cans	0.70	0.84	0.76	90
cups	0.59	0.61	0.60	90
plates	0.64	0.50	0.56	90
apples	0.80	0.82	0.81	90
mushrooms	0.73	0.82	0.77	90
oranges	0.86	0.80	0.83	90
pears	0.78	0.69	0.73	90
sweet peppers	0.74	0.86	0.79	90
clock	0.54	0.68	0.60	90
computer keyboard	0.44	0.53	0.48	90
lamp	0.41	0.43	0.42	90
telephone	0.80	0.82	0.81	90
television	0.59	0.62	0.61	90
bed	0.63	0.51	0.56	90
chair	0.69	0.66	0.67	90
couch	0.55	0.44	0.49	90
table	0.41	0.71	0.52	90
wardrobe	0.64	0.68	0.66	90
bee	0.38	0.22	0.28	90
beetle	0.79	0.67	0.72	90
butterfly	0.65	0.57	0.60	90
caterpillar	0.57	0.43	0.49	90
cockroach	0.92	0.90	0.91	90
bear	0.77	0.64	0.70	90
leopard	0.78	0.84	0.81	90
lion	0.74	0.56	0.63	90
tiger	0.70	0.67	0.68	90
wolf	0.43	0.50	0.46	90
bridge	0.50	0.48	0.49	90
castle	0.48	0.59	0.53	90
house	0.65	0.52	0.58	90
road	0.83	0.81	0.82	90
skyscraper	0.84	0.77	0.80	90
cloud	0.41	0.40	0.41	90
forest	0.66	0.70	0.68	90
mountain	0.59	0.82	0.69	90
plain	0.88	0.88	0.88	90
sea	0.67	0.67	0.67	90
camel	0.30	0.36	0.33	90
cattle	0.71	0.78	0.74	90
chimpanzee	0.67	0.74	0.71	90

elephant	0.90	0.63	0.75	90
kangaroo	0.62	0.48	0.54	90
fox	0.81	0.78	0.80	90
porcupine	0.71	0.72	0.71	90
possum	0.62	0.62	0.62	90
raccoon	0.59	0.64	0.62	90
skunk	0.66	0.30	0.41	90
crab	0.54	0.46	0.49	90
lobster	0.68	0.54	0.60	90
snail	0.52	0.59	0.55	90
spider	0.83	0.87	0.85	90
worm	0.87	0.76	0.81	90
baby	0.64	0.62	0.63	90
boy	0.65	0.81	0.72	90
girl	0.37	0.27	0.31	90
man	0.60	0.61	0.61	90
woman	0.37	0.39	0.38	90
crocodile	0.74	0.87	0.80	90
dinosaur	0.73	0.90	0.81	90
lizard	0.57	0.57	0.57	90
snake	0.78	0.60	0.68	90
turtle	0.74	0.58	0.65	90
hamster	0.33	0.43	0.38	90
mouse	0.41	0.57	0.48	90
rabbit	0.78	0.93	0.85	90
shrew	0.76	0.67	0.71	90
squirrel	0.71	0.53	0.61	90
maple	0.62	0.72	0.67	90
oak	0.82	0.78	0.80	90
palm	0.77	0.76	0.76	90
pine	0.62	0.62	0.62	90
willow	0.71	0.64	0.67	90
bicycle	0.63	0.56	0.59	90
bus	0.54	0.76	0.63	90
motorcycle	0.42	0.54	0.48	90
pickup truck	0.76	0.53	0.63	90
train	0.80	0.91	0.85	90
lawn-mower	0.57	0.68	0.62	90
rocket	0.48	0.49	0.49	90
streetcar	0.66	0.62	0.64	90
tank	0.43	0.41	0.42	90
tractor	0.78	0.81	0.79	90
accuracy			0.64	9000
macro avg	0.64	0.64	0.63	9000
weighted avg	0.64	0.64	0.63	9000

Model: "sequential_4"

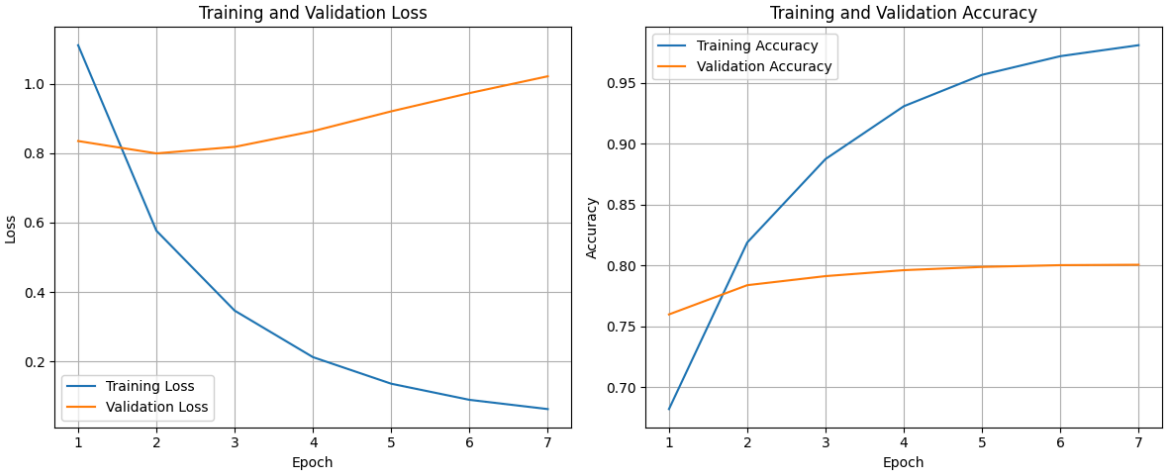
Layer (type)	Output Shape
resnet50v2 (Functional)	(None, 7, 7, 2048)
global_average_pooling2d_4 (GlobalAveragePooling2D)	(None, 2048)
dense_8 (Dense)	(None, 512)
dropout_4 (Dropout)	(None, 512)
dense_9 (Dense)	(None, 100)

Total params: 24,665,188 (94.09 MB)

Trainable params: 24,619,748 (93.92 MB)

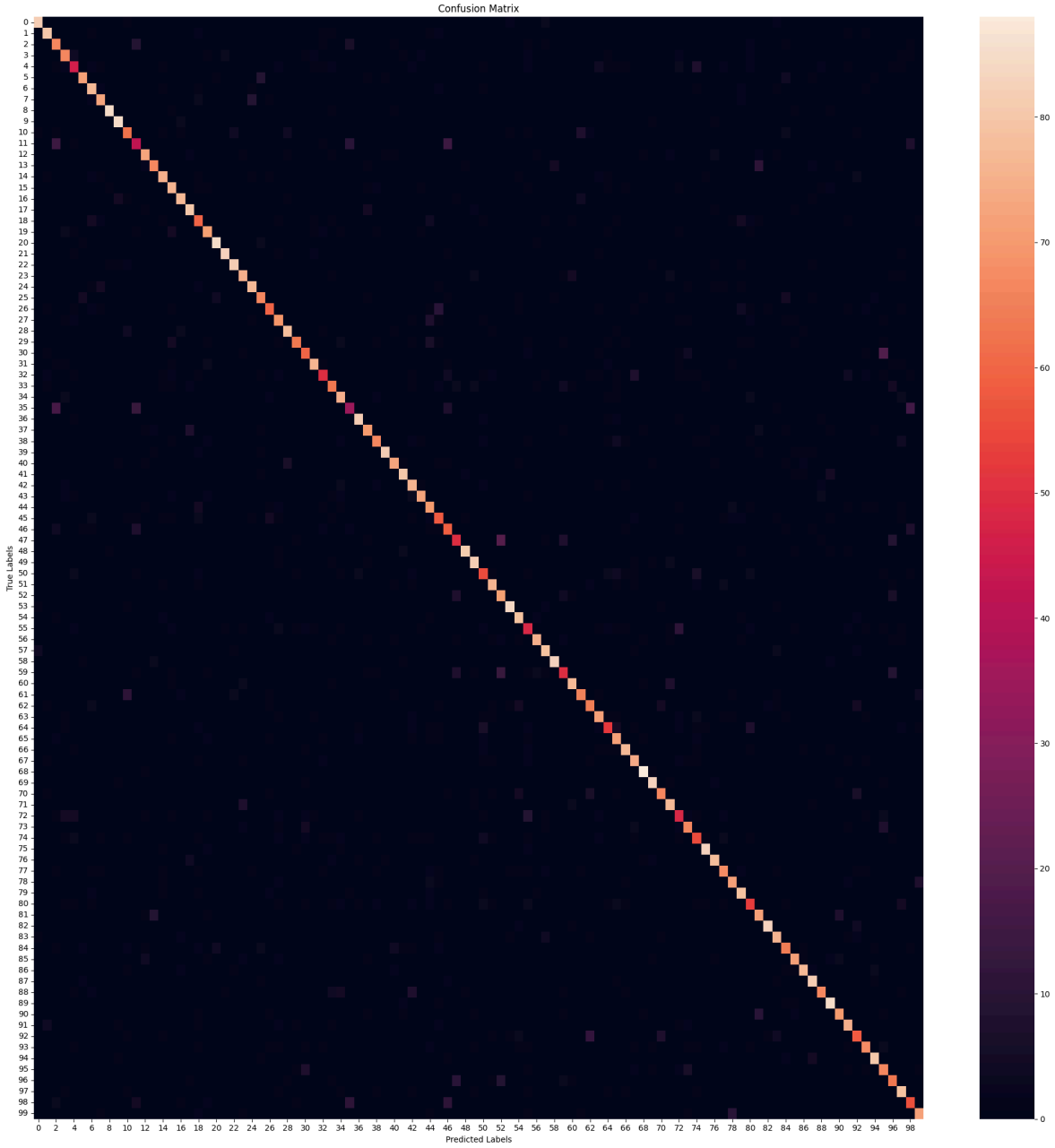
Non-trainable params: 45,440 (177.50 KB)

```
Epoch 1/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 210s 81ms/step - accuracy: 0.6002 - f1_score: 0.59
81 - loss: 1.4670 - precision: 0.7870 - val_accuracy: 0.7598 - val_f1_score: 0.75
84 - val_loss: 0.8347 - val_precision: 0.8478
Epoch 2/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 135s 68ms/step - accuracy: 0.8016 - f1_score: 0.80
01 - loss: 0.6373 - precision: 0.8821 - val_accuracy: 0.7838 - val_f1_score: 0.78
30 - val_loss: 0.7992 - val_precision: 0.8435
Epoch 3/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 135s 68ms/step - accuracy: 0.8822 - f1_score: 0.88
13 - loss: 0.3627 - precision: 0.9265 - val_accuracy: 0.7912 - val_f1_score: 0.79
05 - val_loss: 0.8179 - val_precision: 0.8358
Epoch 4/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 136s 69ms/step - accuracy: 0.9317 - f1_score: 0.93
11 - loss: 0.2107 - precision: 0.9549 - val_accuracy: 0.7961 - val_f1_score: 0.79
60 - val_loss: 0.8631 - val_precision: 0.8322
Epoch 5/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 135s 69ms/step - accuracy: 0.9594 - f1_score: 0.95
88 - loss: 0.1284 - precision: 0.9716 - val_accuracy: 0.7988 - val_f1_score: 0.79
91 - val_loss: 0.9202 - val_precision: 0.8286
Epoch 6/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 136s 69ms/step - accuracy: 0.9750 - f1_score: 0.97
45 - loss: 0.0804 - precision: 0.9818 - val_accuracy: 0.8002 - val_f1_score: 0.80
04 - val_loss: 0.9727 - val_precision: 0.8241
Epoch 7/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 135s 69ms/step - accuracy: 0.9837 - f1_score: 0.98
32 - loss: 0.0552 - precision: 0.9873 - val_accuracy: 0.8006 - val_f1_score: 0.80
09 - val_loss: 1.0211 - val_precision: 0.8252
```



141/141 ━━━━━━━━━━━━━━━━━━━━ **3s** 24ms/step - accuracy: 0.7784 - f1_score: 0.7703 - loss: 0.8032 - precision: 0.8459
Test Accuracy: 78.37%
Test Loss: 78.67%
Test Precision: 84.76%
Test F1 Scores (Per Class): [91.6201 87.43169 65.686264 74.03315 55.421684 82.75862 78.97436
82.02247 95.505615 91.891884 69.27374 49.411755 82.48587 77.906975
83.33333 79.166664 85.08286 86.02149 64.17112 78.453026 92.3913
93.25842 90.60773 81.08108 85.08286 75.28088 70.58823 73.01586
86.18784 78.75 69.36415 86.36363 63.694263 71.99999 78.947365
45.161285 90.60773 80. 77.64706 93.71427 82.95453 88.888885
76.76767 85.38011 70.70706 64.44444 60.20408 58.823524 92.04544
87.70053 61.452507 84.91619 69.26829 92.81767 82.7225 53.631275
86.20689 84.491974 92.73743 60.60605 85.55555 76.92307 71.50838
79.12087 64.197525 74.99999 86.03351 79.569885 94.62365 90.71037
77.906975 82.7957 54.857136 75.706215 62.85714 92.73743 86.666664
78.16091 79.55801 82.291664 60.227264 75.789474 93.7853 82.608696
72.31638 82.75862 87.999985 87.23404 79.51807 89.00522 81.39534
81.52173 66.666664 79.069756 88.888885 68.0203 67.02127 81.44328
61.538452 81.35593]
Average Test F1 Scores:78.26

282/282 ━━━━━━━━━━━━━━━━━━━━ **12s** 29ms/step

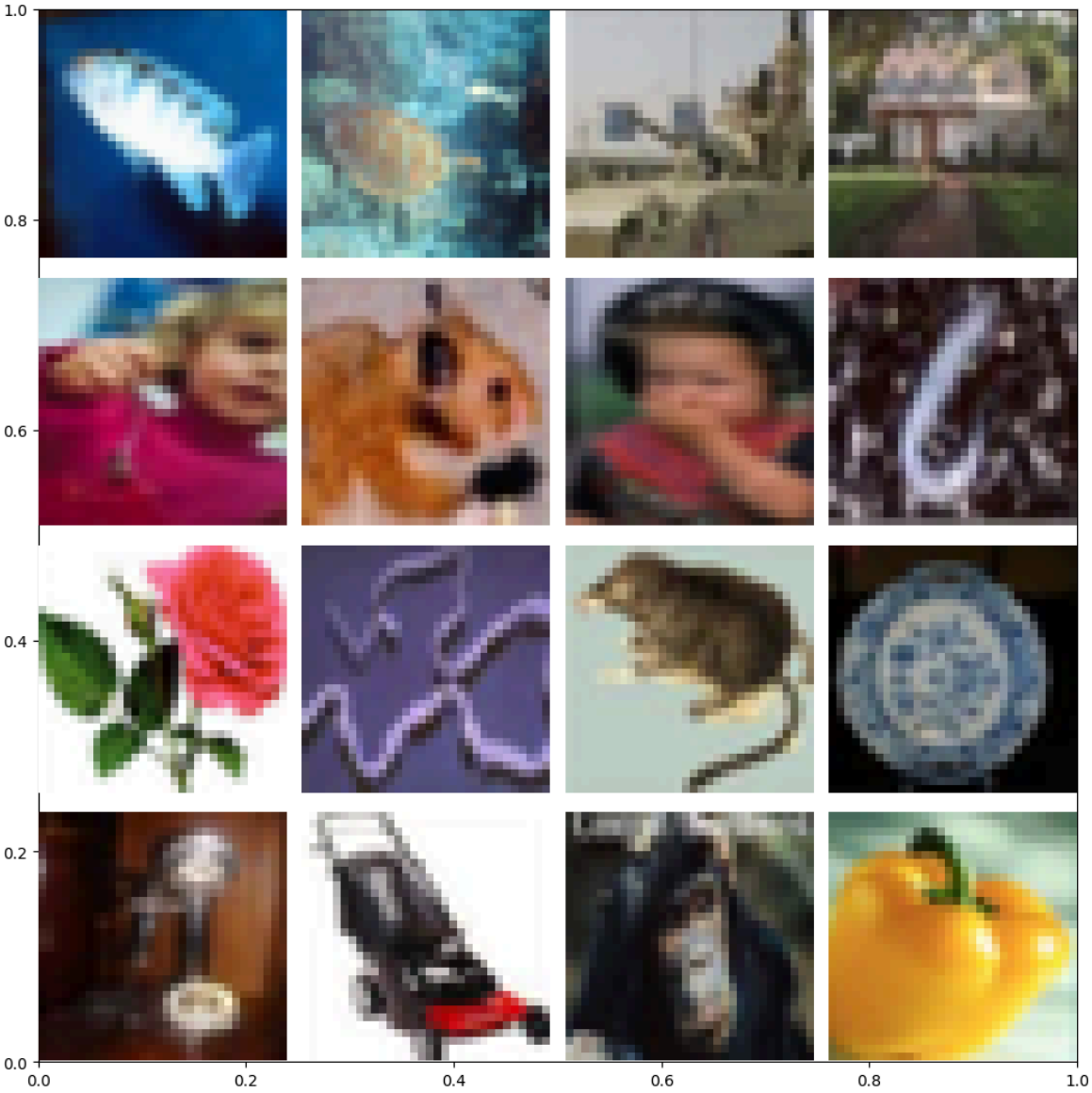

Confusion Matrix
True Labels
Predicted Labels

	precision	recall	f1-score	support
beaver	0.92	0.91	0.92	90
dolphin	0.86	0.89	0.87	90
otter	0.59	0.74	0.66	90
seal	0.74	0.74	0.74	90
whale	0.61	0.51	0.55	90
aquarium fish	0.86	0.80	0.83	90
flatfish	0.73	0.86	0.79	90
ray	0.83	0.81	0.82	90
shark	0.97	0.94	0.96	90
trout	0.89	0.94	0.92	90
orchids	0.70	0.69	0.69	90
poppies	0.53	0.47	0.49	90
roses	0.84	0.81	0.82	90
sunflowers	0.82	0.74	0.78	90
tulips	0.83	0.83	0.83	90
bottles	0.75	0.84	0.79	90
bowls	0.85	0.86	0.85	90
cans	0.83	0.89	0.86	90
cups	0.62	0.67	0.64	90
plates	0.78	0.79	0.78	90
apples	0.90	0.94	0.92	90
mushrooms	0.94	0.92	0.93	90
oranges	0.90	0.91	0.91	90
pears	0.79	0.83	0.81	90
sweet peppers	0.85	0.86	0.85	90
clock	0.76	0.74	0.75	90
computer keyboard	0.75	0.67	0.71	90
lamp	0.70	0.77	0.73	90
telephone	0.86	0.87	0.86	90
television	0.90	0.70	0.79	90
bed	0.72	0.67	0.69	90
chair	0.88	0.84	0.86	90
couch	0.75	0.56	0.64	90
table	0.74	0.70	0.72	90
wardrobe	0.75	0.83	0.79	90
bee	0.54	0.39	0.45	90
beetle	0.90	0.91	0.91	90
butterfly	0.82	0.78	0.80	90
caterpillar	0.82	0.73	0.78	90
cockroach	0.96	0.91	0.94	90
bear	0.85	0.81	0.83	90
leopard	0.89	0.89	0.89	90
lion	0.70	0.84	0.77	90
tiger	0.90	0.81	0.85	90
wolf	0.65	0.78	0.71	90
bridge	0.64	0.64	0.64	90
castle	0.56	0.66	0.60	90
house	0.62	0.56	0.59	90
road	0.94	0.90	0.92	90
skyscraper	0.85	0.91	0.88	90
cloud	0.62	0.61	0.61	90
forest	0.85	0.84	0.85	90
mountain	0.62	0.79	0.69	90
plain	0.92	0.93	0.93	90
sea	0.78	0.88	0.83	90
camel	0.54	0.53	0.54	90
cattle	0.89	0.83	0.86	90
chimpanzee	0.81	0.88	0.84	90

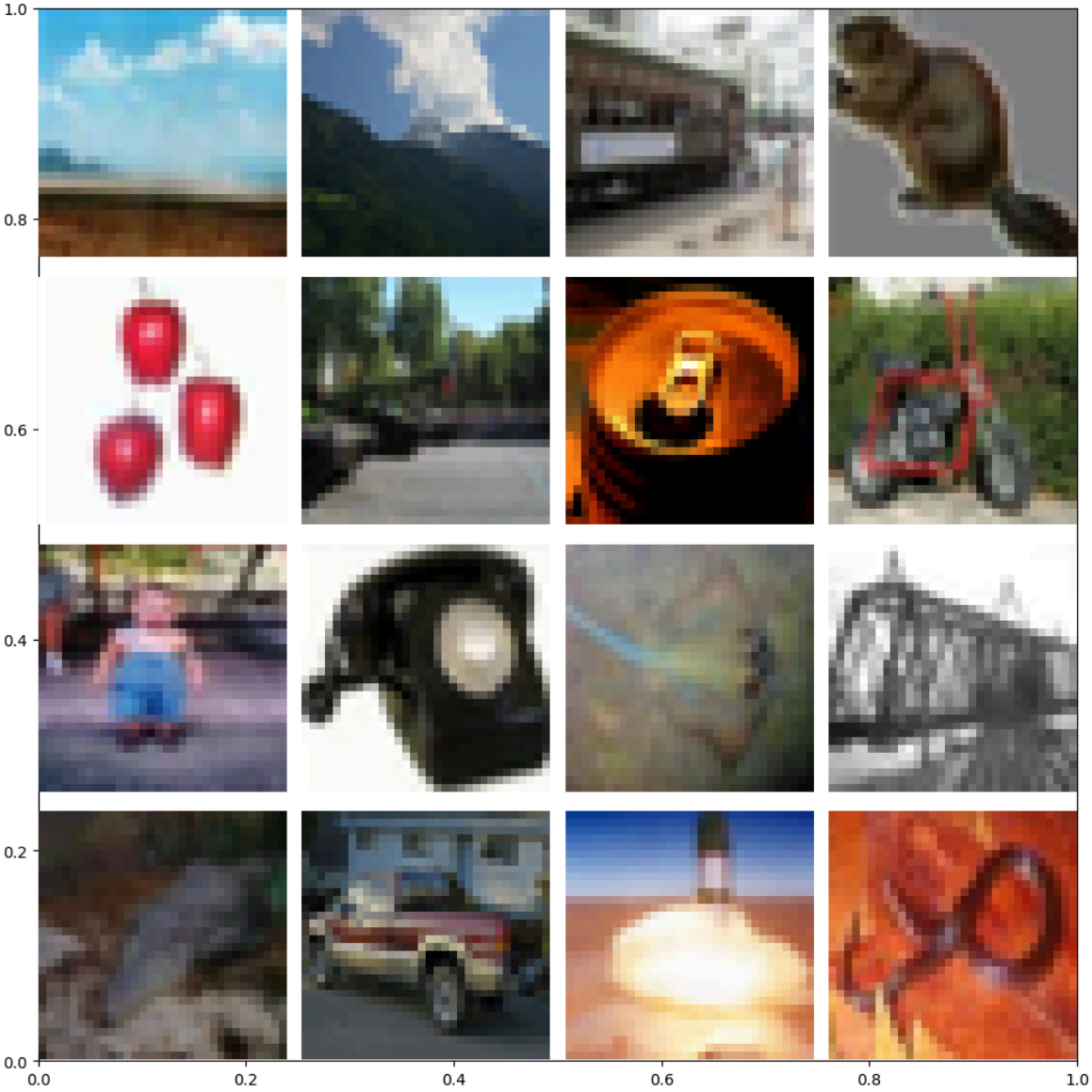
elephant	0.93	0.92	0.93	90
kangaroo	0.67	0.56	0.61	90
fox	0.86	0.86	0.86	90
porcupine	0.82	0.72	0.77	90
possum	0.72	0.71	0.72	90
raccoon	0.78	0.80	0.79	90
skunk	0.72	0.58	0.64	90
crab	0.71	0.80	0.75	90
lobster	0.87	0.86	0.86	90
snail	0.77	0.82	0.80	90
spider	0.92	0.98	0.95	90
worm	0.89	0.92	0.91	90
baby	0.82	0.74	0.78	90
boy	0.80	0.86	0.83	90
girl	0.56	0.53	0.55	90
man	0.77	0.74	0.76	90
woman	0.65	0.61	0.63	90
crocodile	0.93	0.92	0.93	90
dinosaur	0.87	0.87	0.87	90
lizard	0.81	0.76	0.78	90
snake	0.79	0.80	0.80	90
turtle	0.77	0.88	0.82	90
hamster	0.62	0.59	0.60	90
mouse	0.72	0.80	0.76	90
rabbit	0.95	0.92	0.94	90
shrew	0.81	0.84	0.83	90
squirrel	0.74	0.71	0.72	90
maple	0.86	0.80	0.83	90
oak	0.91	0.86	0.88	90
palm	0.84	0.91	0.87	90
pine	0.87	0.73	0.80	90
willow	0.84	0.94	0.89	90
bicycle	0.85	0.78	0.81	90
bus	0.80	0.83	0.82	90
motorcycle	0.69	0.64	0.67	90
pickup truck	0.83	0.76	0.79	90
train	0.89	0.89	0.89	90
lawn-mower	0.63	0.74	0.68	90
rocket	0.64	0.70	0.67	90
streetcar	0.76	0.88	0.81	90
tank	0.61	0.62	0.62	90
tractor	0.83	0.80	0.81	90
accuracy			0.78	9000
macro avg	0.79	0.78	0.78	9000
weighted avg	0.79	0.78	0.78	9000

```
X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)
y_temp_val.shape: (4000, 1)
X_temp_test.shape: (4000, 32, 32, 3)
y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)
y_train.shape: (42000, 1)
X_val.shape: (9000, 32, 32, 3)
y_val.shape: (9000, 1)
X_test.shape: (9000, 32, 32, 3)
y_test.shape: (9000, 1)
```

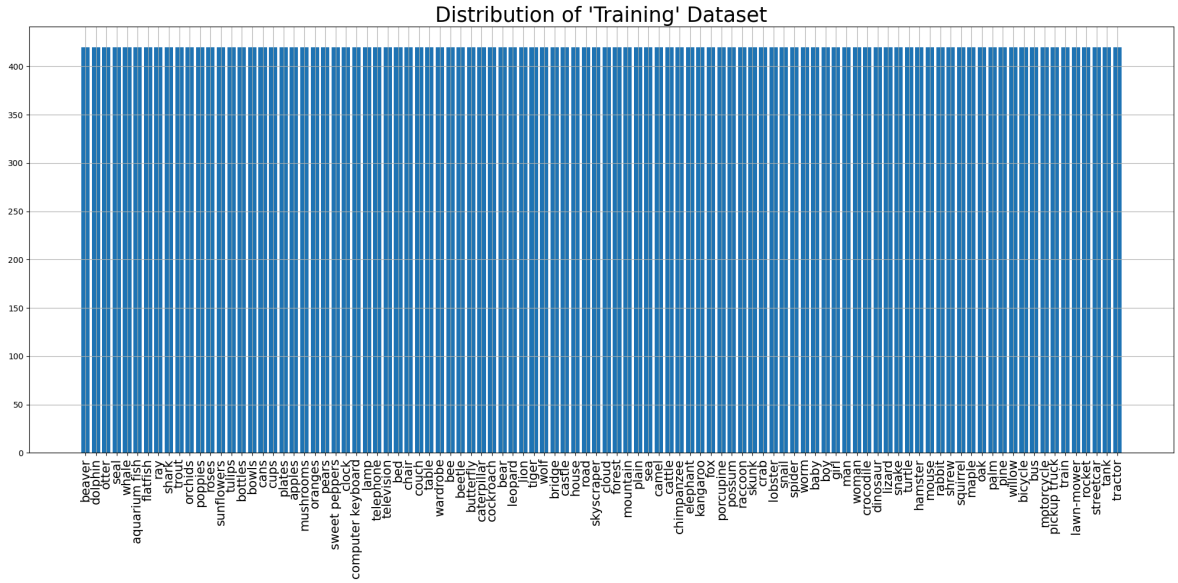
```
labels[7413].shape: (1,)
imgs[7413].shape: (32, 32, 3)
```



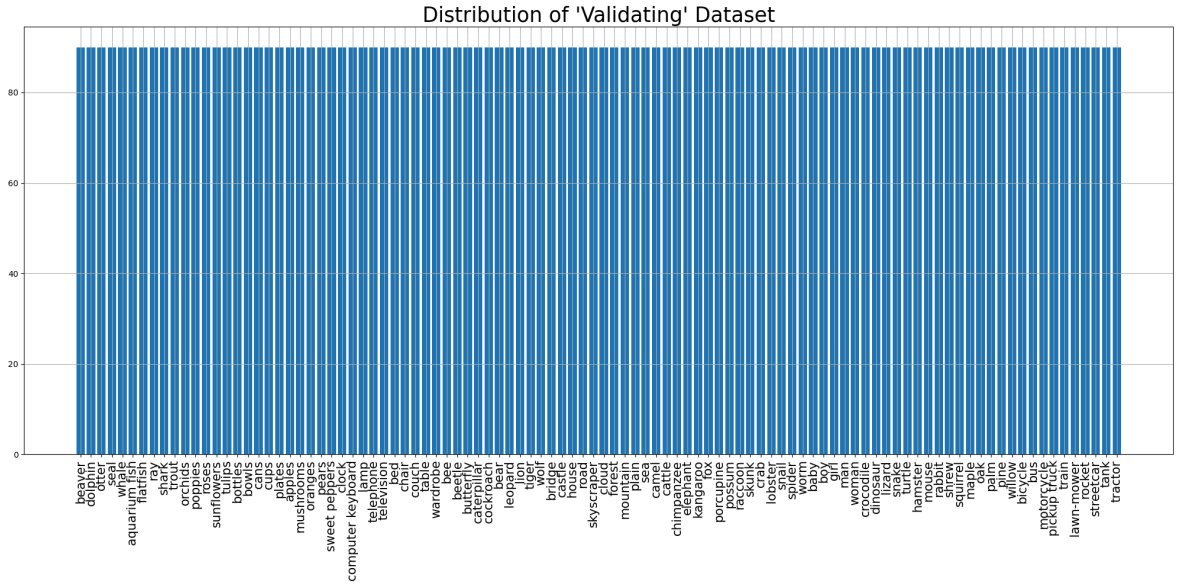

```
labels[16698].shape: (1,)
imgs[16698].shape: (32, 32, 3)
```



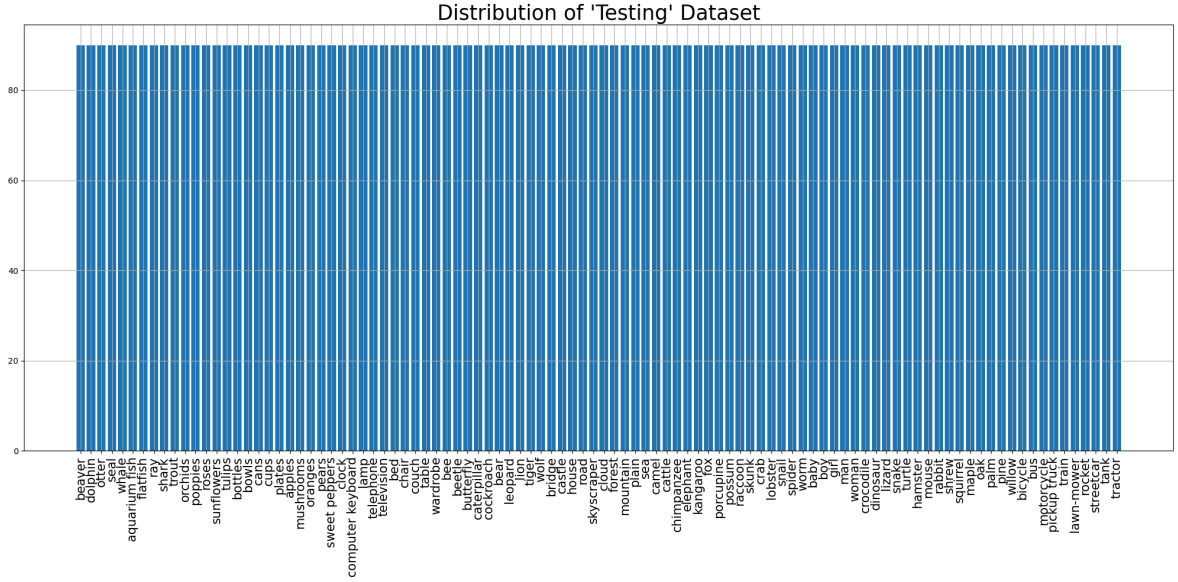

```
<ipython-input-6-1810f983c3fb>:70: UserWarning: set_ticklabels() should only be u
sed with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```




```
<ipython-input-6-1810f983c3fb>:70: UserWarning: set_ticklabels() should only be u
sed with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```




```
<ipython-input-6-1810f983c3fb>:70: UserWarning: set_ticklabels() should only be u
sed with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
Training dataset:
 <_ConcatenateDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=t
f.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=Non
e))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)

Validation dataset:
 <_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.f
loat32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)

Testing dataset:
 <_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.f
loat32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.f
ramework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_5"

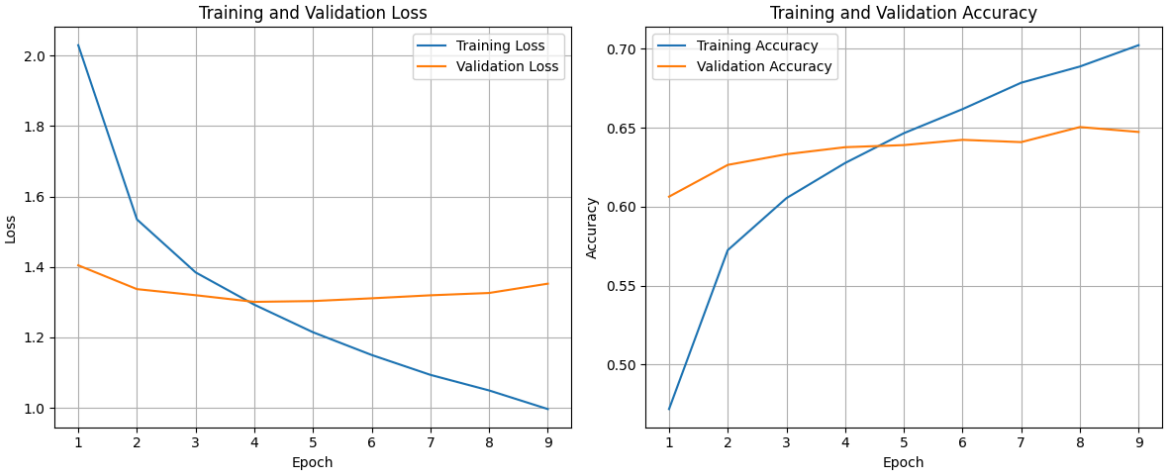
Layer (type)	Output Shape
resnet50v2 (Functional)	(None, 7, 7, 2048)
global_average_pooling2d_5 (GlobalAveragePooling2D)	(None, 2048)
dense_10 (Dense)	(None, 512)
dropout_5 (Dropout)	(None, 512)
dense_11 (Dense)	(None, 100)

Total params: 24,665,188 (94.09 MB)

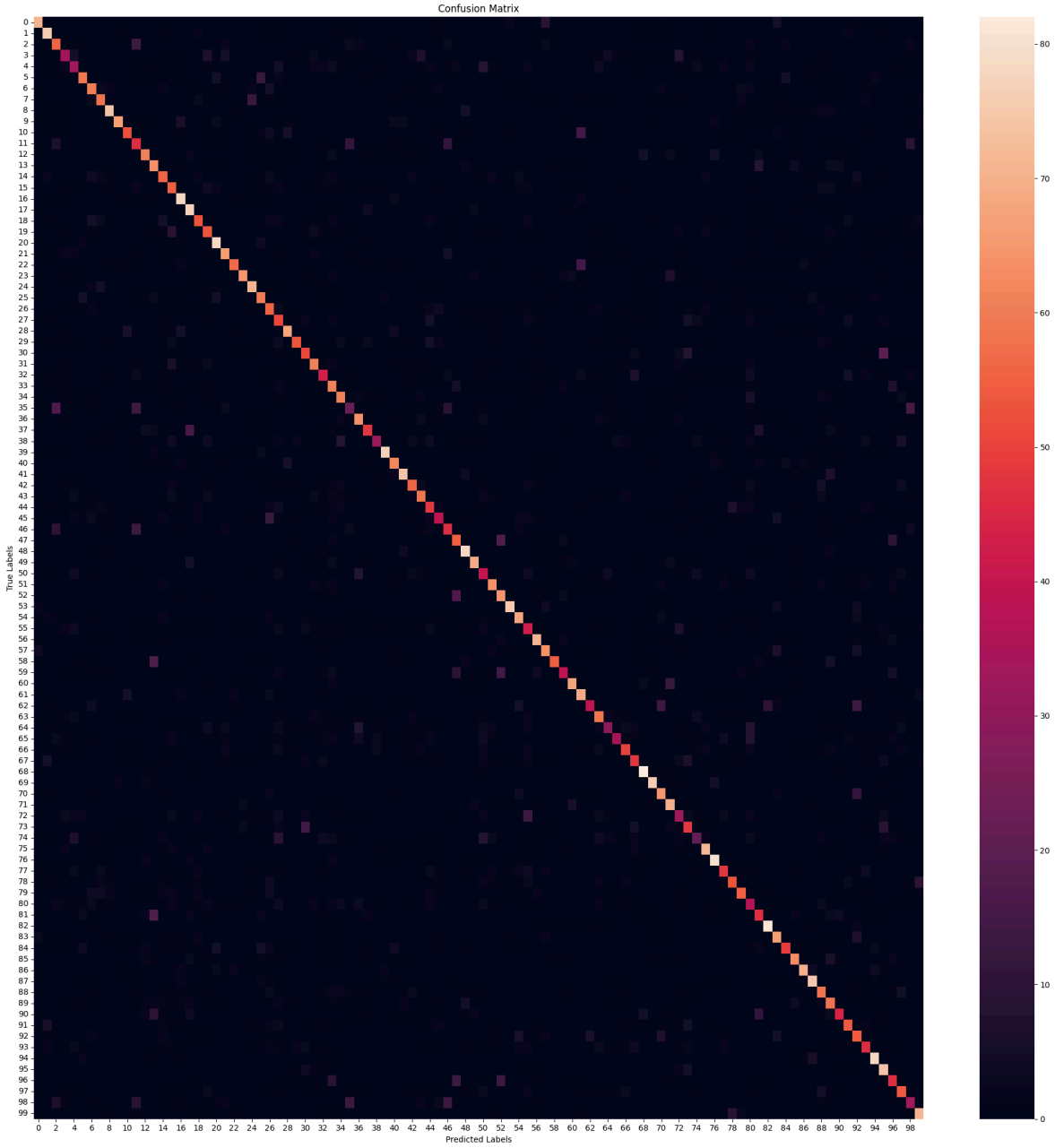
Trainable params: 1,100,388 (4.20 MB)

Non-trainable params: 23,564,800 (89.89 MB)

```
Epoch 1/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 60s 35ms/step - accuracy: 0.3805 - f1_score: 0.376
1 - loss: 2.5326 - precision: 0.6850 - val_accuracy: 0.6063 - val_f1_score: 0.603
4 - val_loss: 1.4048 - val_precision: 0.8087
Epoch 2/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 36s 28ms/step - accuracy: 0.5641 - f1_score: 0.560
9 - loss: 1.5686 - precision: 0.7595 - val_accuracy: 0.6264 - val_f1_score: 0.623
9 - val_loss: 1.3372 - val_precision: 0.8015
Epoch 3/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 36s 27ms/step - accuracy: 0.6016 - f1_score: 0.598
8 - loss: 1.3942 - precision: 0.7745 - val_accuracy: 0.6332 - val_f1_score: 0.630
9 - val_loss: 1.3199 - val_precision: 0.7959
Epoch 4/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 36s 27ms/step - accuracy: 0.6292 - f1_score: 0.626
7 - loss: 1.2872 - precision: 0.7868 - val_accuracy: 0.6377 - val_f1_score: 0.635
8 - val_loss: 1.3010 - val_precision: 0.7931
Epoch 5/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 36s 27ms/step - accuracy: 0.6489 - f1_score: 0.646
9 - loss: 1.1995 - precision: 0.7971 - val_accuracy: 0.6390 - val_f1_score: 0.638
2 - val_loss: 1.3033 - val_precision: 0.7897
Epoch 6/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 36s 27ms/step - accuracy: 0.6645 - f1_score: 0.662
9 - loss: 1.1305 - precision: 0.8006 - val_accuracy: 0.6423 - val_f1_score: 0.639
2 - val_loss: 1.3111 - val_precision: 0.7878
Epoch 7/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 36s 27ms/step - accuracy: 0.6862 - f1_score: 0.684
2 - loss: 1.0616 - precision: 0.8131 - val_accuracy: 0.6409 - val_f1_score: 0.640
5 - val_loss: 1.3197 - val_precision: 0.7783
Epoch 8/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 36s 27ms/step - accuracy: 0.6976 - f1_score: 0.695
6 - loss: 1.0063 - precision: 0.8157 - val_accuracy: 0.6504 - val_f1_score: 0.650
8 - val_loss: 1.3262 - val_precision: 0.7819
Epoch 9/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 36s 27ms/step - accuracy: 0.7118 - f1_score: 0.710
3 - loss: 0.9500 - precision: 0.8222 - val_accuracy: 0.6473 - val_f1_score: 0.647
3 - val_loss: 1.3526 - val_precision: 0.7775
```



```
141/141 ━━━━━━━━━━━━━━━━━━━━ 3s 24ms/step - accuracy: 0.6285 - f1_score: 0.6190 -
loss: 1.3332 - precision: 0.7872
Test Accuracy: 63.94%
Test Loss: 130.75%
Test Precision: 79.72%
Test F1 Scores (Per Class): [83.04093  77.948715 54.36893  44.736843 37.499992 6
8.604645 60.60605
 65.90909  79.99999  81.98757  63.030296 46.7005   68.53932  58.715595
 60.869556 54.726364 76.84728  73.33332  60.227264 56.382973 79.18781
 72.92817  72.61146  78.313255 78.02197  62.5      52.093018 47.0046
 77.71428  68.38709  57.303364 67.03297  52.12121  54.46428  66.66666
 30.872477 67.36841  60.377354 46.37681  87.999985 68.888885 80.43478
 63.276833 66.666664 51.6129   49.36708  47.872334 56.852787 79.18781
 78.85714  42.553185 69.18918  63.414627 87.71929  70.46631  43.97905
 78.88888  68.449196 74.324326 52.702694 77.71428  68.99999  55.31914
 63.101597 42.02898  49.29577  60.975605 54.545456 84.53608  86.85714
 66.666664 72.916664 37.714283 53.33333  31.65467  77.00535  83.33333
 60.75949  63.473045 65.47618  37.18593  51.396645 88.04347  67.6923
 62.025307 71.18643  81.87134  79.365074 59.793808 57.14285  59.060402
 63.905323 52.941174 55.75757  83.422455 67.27272  51.396645 60.33519
 39.75903  78.453026]
Average Test F1 Scores:63.72

282/282 ━━━━━━━━━━━━━━━━━━━━ 12s 29ms/step
```


Confusion Matrix
True Labels
Predicted Labels

	precision	recall	f1-score	support
beaver	0.88	0.79	0.83	90
dolphin	0.72	0.84	0.78	90
otter	0.48	0.62	0.54	90
seal	0.55	0.38	0.45	90
whale	0.38	0.37	0.38	90
aquarium fish	0.72	0.66	0.69	90
flatfish	0.56	0.67	0.61	90
ray	0.67	0.64	0.66	90
shark	0.78	0.82	0.80	90
trout	0.93	0.73	0.82	90
orchids	0.69	0.58	0.63	90
poppies	0.43	0.51	0.47	90
roses	0.69	0.68	0.69	90
sunflowers	0.50	0.71	0.59	90
tulips	0.60	0.62	0.61	90
bottles	0.50	0.61	0.55	90
bowls	0.69	0.87	0.77	90
cans	0.64	0.86	0.73	90
cups	0.62	0.59	0.60	90
plates	0.54	0.59	0.56	90
apples	0.73	0.87	0.79	90
mushrooms	0.73	0.73	0.73	90
oranges	0.85	0.63	0.73	90
pears	0.86	0.72	0.78	90
sweet peppers	0.77	0.79	0.78	90
clock	0.59	0.67	0.62	90
computer keyboard	0.45	0.62	0.52	90
lamp	0.40	0.57	0.47	90
telephone	0.80	0.76	0.78	90
television	0.80	0.59	0.68	90
bed	0.58	0.57	0.57	90
chair	0.66	0.68	0.67	90
couch	0.57	0.48	0.52	90
table	0.46	0.68	0.54	90
wardrobe	0.66	0.68	0.67	90
bee	0.39	0.26	0.31	90
beetle	0.64	0.71	0.67	90
butterfly	0.70	0.53	0.60	90
caterpillar	0.67	0.36	0.46	90
cockroach	0.91	0.86	0.88	90
bear	0.69	0.69	0.69	90
leopard	0.79	0.82	0.80	90
lion	0.64	0.62	0.63	90
tiger	0.68	0.66	0.67	90
wolf	0.50	0.53	0.52	90
bridge	0.57	0.43	0.49	90
castle	0.46	0.51	0.49	90
house	0.52	0.61	0.56	90
road	0.73	0.87	0.79	90
skyscraper	0.81	0.77	0.79	90
cloud	0.41	0.44	0.43	90
forest	0.67	0.71	0.69	90
mountain	0.56	0.72	0.63	90
plain	0.93	0.83	0.88	90
sea	0.66	0.76	0.70	90
camel	0.42	0.47	0.44	90
cattle	0.79	0.79	0.79	90
chimpanzee	0.66	0.71	0.68	90

elephant	0.95	0.61	0.74	90
kangaroo	0.67	0.43	0.53	90
fox	0.80	0.76	0.78	90
porcupine	0.63	0.77	0.69	90
possum	0.76	0.43	0.55	90
raccoon	0.61	0.66	0.63	90
skunk	0.60	0.32	0.42	90
crab	0.67	0.39	0.49	90
lobster	0.68	0.56	0.61	90
snail	0.56	0.53	0.55	90
spider	0.79	0.91	0.85	90
worm	0.89	0.84	0.87	90
baby	0.62	0.72	0.67	90
boy	0.69	0.78	0.73	90
girl	0.39	0.37	0.38	90
man	0.53	0.53	0.53	90
woman	0.45	0.24	0.32	90
crocodile	0.74	0.80	0.77	90
dinosaur	0.78	0.89	0.83	90
lizard	0.71	0.53	0.61	90
snake	0.69	0.59	0.63	90
turtle	0.71	0.61	0.65	90
hamster	0.34	0.41	0.37	90
mouse	0.52	0.51	0.51	90
rabbit	0.86	0.90	0.88	90
shrew	0.63	0.73	0.68	90
squirrel	0.72	0.54	0.62	90
maple	0.73	0.70	0.72	90
oak	0.86	0.78	0.82	90
palm	0.76	0.83	0.79	90
pine	0.56	0.64	0.60	90
willow	0.51	0.64	0.57	90
bicycle	0.75	0.49	0.59	90
bus	0.68	0.60	0.64	90
motorcycle	0.48	0.60	0.53	90
pickup truck	0.61	0.51	0.56	90
train	0.80	0.87	0.83	90
lawn-mower	0.57	0.82	0.67	90
rocket	0.51	0.51	0.51	90
streetcar	0.61	0.60	0.60	90
tank	0.43	0.37	0.40	90
tractor	0.78	0.79	0.78	90
accuracy			0.64	9000
macro avg	0.65	0.64	0.64	9000
weighted avg	0.65	0.64	0.64	9000

Model: "sequential_5"

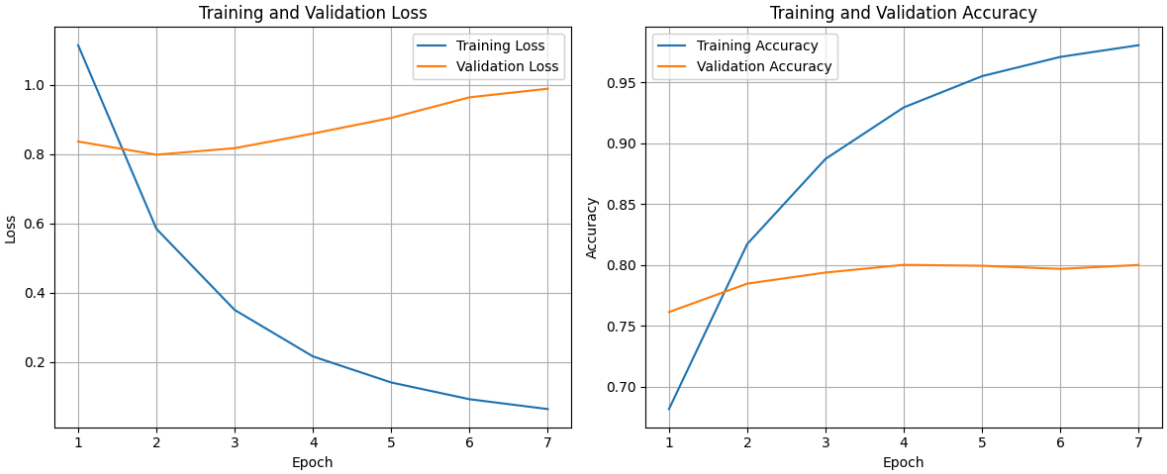
Layer (type)	Output Shape
resnet50v2 (Functional)	(None, 7, 7, 2048)
global_average_pooling2d_5 (GlobalAveragePooling2D)	(None, 2048)
dense_10 (Dense)	(None, 512)
dropout_5 (Dropout)	(None, 512)
dense_11 (Dense)	(None, 100)

Total params: 24,665,188 (94.09 MB)

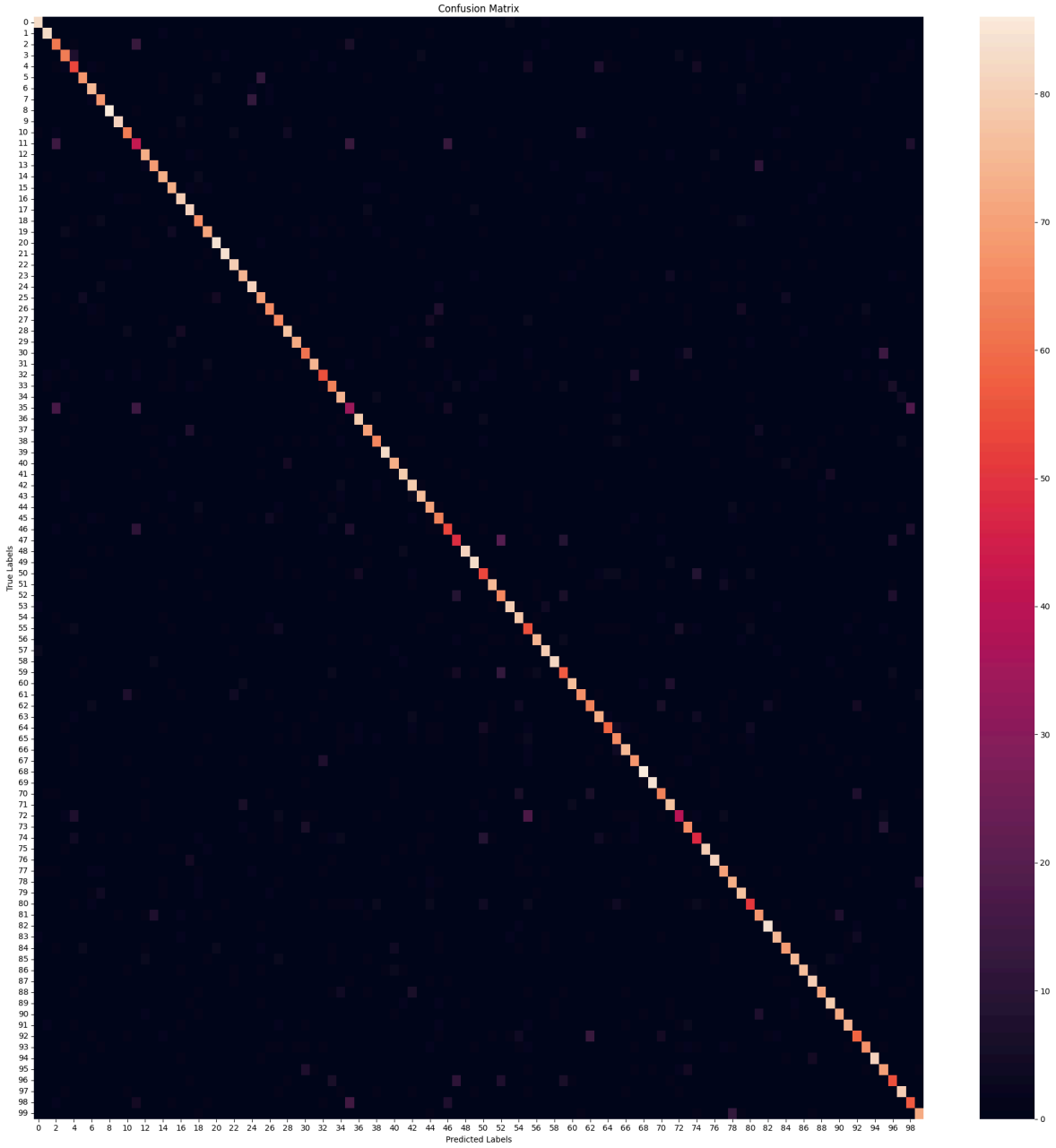
Trainable params: 24,619,748 (93.92 MB)

Non-trainable params: 45,440 (177.50 KB)

```
Epoch 1/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 215s 83ms/step - accuracy: 0.6007 - f1_score: 0.59
91 - loss: 1.4740 - precision: 0.7874 - val_accuracy: 0.7613 - val_f1_score: 0.75
97 - val_loss: 0.8368 - val_precision: 0.8464
Epoch 2/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 137s 70ms/step - accuracy: 0.8004 - f1_score: 0.79
86 - loss: 0.6457 - precision: 0.8829 - val_accuracy: 0.7847 - val_f1_score: 0.78
42 - val_loss: 0.7989 - val_precision: 0.8479
Epoch 3/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 137s 69ms/step - accuracy: 0.8826 - f1_score: 0.88
16 - loss: 0.3661 - precision: 0.9273 - val_accuracy: 0.7938 - val_f1_score: 0.79
35 - val_loss: 0.8176 - val_precision: 0.8424
Epoch 4/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 136s 69ms/step - accuracy: 0.9296 - f1_score: 0.92
89 - loss: 0.2152 - precision: 0.9539 - val_accuracy: 0.8001 - val_f1_score: 0.80
01 - val_loss: 0.8600 - val_precision: 0.8349
Epoch 5/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 136s 69ms/step - accuracy: 0.9587 - f1_score: 0.95
82 - loss: 0.1321 - precision: 0.9713 - val_accuracy: 0.7993 - val_f1_score: 0.79
98 - val_loss: 0.9051 - val_precision: 0.8306
Epoch 6/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 137s 69ms/step - accuracy: 0.9743 - f1_score: 0.97
37 - loss: 0.0840 - precision: 0.9808 - val_accuracy: 0.7968 - val_f1_score: 0.79
73 - val_loss: 0.9646 - val_precision: 0.8230
Epoch 7/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 137s 69ms/step - accuracy: 0.9838 - f1_score: 0.98
32 - loss: 0.0556 - precision: 0.9875 - val_accuracy: 0.8000 - val_f1_score: 0.80
06 - val_loss: 0.9894 - val_precision: 0.8233
```




```
141/141 ━━━━━━━━━━━━━━━━━━━━ 4s 25ms/step - accuracy: 0.7760 - f1_score: 0.7687 -
loss: 0.8041 - precision: 0.8410
Test Accuracy: 78.21%
Test Loss: 78.76%
Test Precision: 84.43%
Test F1 Scores (Per Class): [91.208786 91.7127   63.589745 71.590904 58.56353  7
9.99999  80.64516
 75.97765  95.55555  92.57143  73.2558   45.65217  82.02247  80.23255
 80.66298  80.87432  85.56149  84.37499  71.35134  79.329605 90.81081
 93.854744 88.888885 83.615814 84.81675  75.675674 76.300575 71.03825
 86.516846 83.236984 71.764694 83.14606  67.90123  68.449196 78.723404
 39.534874 87.29281  80.92485  74.71264  94.318184 79.347824 87.77777
 80.61224  85.8757   72.82051  70.329666 60.571426 57.64705  92.57143
 89.24731  58.56353  83.798874 66.32652  90.90909  81.24999  54.187183
 85.549126 86.48647  92.65536  64.04494  87.356316 79.761894 71.1111
 78.49462  71.16564  70.588234 83.33333  75.138115 92.47311  93.854744
 73.56321  81.481476 51.315792 73.33333  55.49132  91.428566 88.52458
 78.65168  77.65957  81.05263  59.999992 73.513504 93.33333  81.08108
 78.40909  84.74575  88.372086 86.02149  81.60919  85.86956  81.35593
 82.68156  67.44185  77.01149  89.502754 71.794876 62.85714  85.10638
 61.62162  81.81818 ]
Average Test F1 Scores:78.17

282/282 ━━━━━━━━━━━━━━━━━━━━ 13s 30ms/step
```


Confusion Matrix
True Labels
Predicted Labels

	precision	recall	f1-score	support
beaver	0.90	0.92	0.91	90
dolphin	0.91	0.92	0.92	90
otter	0.59	0.69	0.64	90
seal	0.74	0.70	0.72	90
whale	0.58	0.59	0.59	90
aquarium fish	0.85	0.76	0.80	90
flatfish	0.78	0.83	0.81	90
ray	0.76	0.76	0.76	90
shark	0.96	0.96	0.96	90
trout	0.95	0.90	0.93	90
orchids	0.77	0.70	0.73	90
poppies	0.45	0.47	0.46	90
roses	0.83	0.81	0.82	90
sunflowers	0.84	0.77	0.80	90
tulips	0.80	0.81	0.81	90
bottles	0.79	0.81	0.80	90
bowls	0.82	0.89	0.86	90
cans	0.79	0.90	0.84	90
cups	0.69	0.73	0.71	90
plates	0.80	0.79	0.79	90
apples	0.88	0.93	0.91	90
mushrooms	0.94	0.93	0.94	90
oranges	0.89	0.89	0.89	90
pears	0.85	0.82	0.84	90
sweet peppers	0.80	0.90	0.85	90
clock	0.74	0.78	0.76	90
computer keyboard	0.80	0.73	0.76	90
lamp	0.70	0.72	0.71	90
telephone	0.88	0.86	0.87	90
television	0.87	0.80	0.83	90
bed	0.76	0.68	0.72	90
chair	0.84	0.82	0.83	90
couch	0.76	0.61	0.68	90
table	0.66	0.71	0.68	90
wardrobe	0.76	0.82	0.79	90
bee	0.41	0.38	0.40	90
beetle	0.87	0.88	0.87	90
butterfly	0.84	0.78	0.81	90
caterpillar	0.76	0.72	0.74	90
cockroach	0.97	0.92	0.94	90
bear	0.78	0.81	0.79	90
leopard	0.88	0.88	0.88	90
lion	0.75	0.88	0.81	90
tiger	0.87	0.84	0.86	90
wolf	0.68	0.79	0.73	90
bridge	0.70	0.71	0.70	90
castle	0.62	0.59	0.61	90
house	0.61	0.54	0.58	90
road	0.95	0.90	0.93	90
skyscraper	0.86	0.92	0.89	90
cloud	0.58	0.59	0.59	90
forest	0.84	0.83	0.84	90
mountain	0.61	0.72	0.66	90
plain	0.93	0.89	0.91	90
sea	0.76	0.87	0.81	90
camel	0.49	0.61	0.54	90
cattle	0.89	0.82	0.86	90
chimpanzee	0.84	0.89	0.86	90

elephant	0.94	0.91	0.93	90
kangaroo	0.65	0.63	0.64	90
fox	0.90	0.84	0.87	90
porcupine	0.86	0.74	0.80	90
possum	0.71	0.71	0.71	90
raccoon	0.76	0.81	0.78	90
skunk	0.78	0.64	0.71	90
crab	0.68	0.73	0.71	90
lobster	0.83	0.83	0.83	90
snail	0.75	0.76	0.75	90
spider	0.90	0.96	0.92	90
worm	0.94	0.93	0.94	90
baby	0.76	0.71	0.74	90
boy	0.78	0.86	0.81	90
girl	0.63	0.43	0.51	90
man	0.73	0.73	0.73	90
woman	0.58	0.53	0.55	90
crocodile	0.94	0.89	0.91	90
dinosaur	0.87	0.90	0.89	90
lizard	0.80	0.78	0.79	90
snake	0.74	0.81	0.78	90
turtle	0.77	0.86	0.81	90
hamster	0.64	0.57	0.60	90
mouse	0.72	0.76	0.74	90
rabbit	0.93	0.93	0.93	90
shrew	0.79	0.83	0.81	90
squirrel	0.80	0.77	0.78	90
maple	0.86	0.83	0.85	90
oak	0.93	0.84	0.88	90
palm	0.83	0.89	0.86	90
pine	0.85	0.79	0.82	90
willow	0.84	0.88	0.86	90
bicycle	0.83	0.80	0.81	90
bus	0.83	0.82	0.83	90
motorcycle	0.71	0.64	0.67	90
pickup truck	0.80	0.74	0.77	90
train	0.89	0.90	0.90	90
lawn-mower	0.67	0.78	0.72	90
rocket	0.65	0.61	0.63	90
streetcar	0.82	0.89	0.85	90
tank	0.60	0.63	0.62	90
tractor	0.84	0.80	0.82	90
accuracy			0.78	9000
macro avg	0.78	0.78	0.78	9000
weighted avg	0.78	0.78	0.78	9000

Weight decay

In [7]:
```
for repeat_2_times in range(2):
    #### <<<<<<<<<<Load and process data>>>>>>>>>>
    # Load CIFAR-100 dataset
    (X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

    # Split (8000) of training data into temporary set
    X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_s
    print(f"X_temp.shape: {X_temp.shape}\n")
```

```
# Split temp data into equal validation (4000) and testing (4000) data
X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp,
print(f"X_temp_val.shape: {X_temp_val.shape}")
print(f"y_temp_val.shape: {y_temp_val.shape}")
print(f"X_temp_test.shape: {X_temp_test.shape}")
print(f"y_temp_test.shape: {y_temp_test.shape}\n")

# Split test data into validation (5000) and testing (5000)
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.

# Add temp_val to validation (9000) and temp_test to testing (9000) to get a
X_val = np.concatenate((X_val, X_temp_val), axis=0)
y_val = np.concatenate((y_val, y_temp_val), axis=0)
X_test = np.concatenate((X_test, X_temp_test), axis=0)
y_test = np.concatenate((y_test, y_temp_test), axis=0)

print(f"X_train.shape: {X_train.shape}")
print(f"y_train.shape: {y_train.shape}")
print(f"X_val.shape: {X_val.shape}")
print(f"y_val.shape: {y_val.shape}")
print(f"X_test.shape: {X_test.shape}")
print(f"y_test.shape: {y_test.shape}\n")

def display_imgs(imgs, labels):
    plt.subplots(figsize=(10,10))
    for i in range(16):
        plt.subplot(4, 4, i+1)
        k = np.random.randint(0, imgs.shape[0])
        if i == 0:
            print(f"labels[{k}].shape: {labels[k].shape}")
            print(f"imgs[{k}].shape: {imgs[k].shape}")
        plt.imshow(imgs[k])
        #plt.title(labels[k])
        plt.axis('off')
    plt.tight_layout()
    plt.show()

display_imgs(X_train, y_train)

# Normalise images (scale to range [0, 1]) - Improves convergence speed & ac
X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
display_imgs(X_train, y_train)

labels_names = ['beaver','dolphin','otter','seal','whale','aquarium fish','f
                'orchids','poppies','roses','sunflowers','tulips','bottles','
                'apples','mushrooms','oranges','pears','sweet peppers','clock
                'telephone','television','bed','chair','couch','table','wardr
                'caterpillar','cockroach','bear','leopard','lion','tiger','wo
                'road','skyscraper','cloud','forest','mountain','plain','sea'
                'elephant','kangaroo','fox','porcupine','possum','raccoon','s
                'spider','worm','baby','boy','girl','man','woman','crocodile'
                'turtle','hamster','mouse','rabbit','shrew','squirrel','maple
                'bicycle','bus','motorcycle','pickup truck','train','lawn-mow
                'tractor']

def class_distrib(y, labels_names, dataset_name):
    counts = pd.DataFrame(data=y).value_counts().sort_index()
    #print(f"counts:\n{counts}")
    fig, ax = plt.subplots(figsize=(20,10))
```

```
    ax.bar(labels_names, counts)
    ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
    plt.title(f"Distribution of '{dataset_name}' Dataset", fontsize=25)
    plt.grid()
    plt.tight_layout()
    plt.show()
class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64
train_dataset_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                     tf.squeeze(tf.one_hot(y, depth=100, dtyp
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE))

def augment_dataset(x, y):
    x = tf.image.resize(x, (224, 224))  # Resize images
    x = tf.image.random_flip_left_right(x)  # Random horizontal flip
    x = tf.image.random_brightness(x, max_delta=0.2)  # Adjust brightness
    x = tf.image.random_contrast(x, lower=0.8, upper=1.2)  # Adjust contrast
    y = tf.squeeze(tf.one_hot(y, depth=100, dtype=tf.float32))  # One-hot en
    return x, y
train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(augment_dataset, num_parallel_calls=tf.data.experiment
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE))

# Combine the original dataset and the augmented dataset
train_dataset = train_dataset_.concatenate(train_dataset_aug)

# Optionally, shuffle the combined dataset if needed
#combined_dataset = combined_dataset.shuffle(buffer_size=1000)  # Adjust buf

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
                .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
                .batch(batch_size)
                .prefetch(tf.data.experimental.AUTOTUNE))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                     tf.squeeze(tf.one_hot(y, depth=100, dtyp
                 .batch(batch_size)
                 .prefetch(tf.data.experimental.AUTOTUNE))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}")  # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}")  # Should be (batch_size, 10)
    del img,lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}")  # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}")  # Should be (batch_size, 10)
```

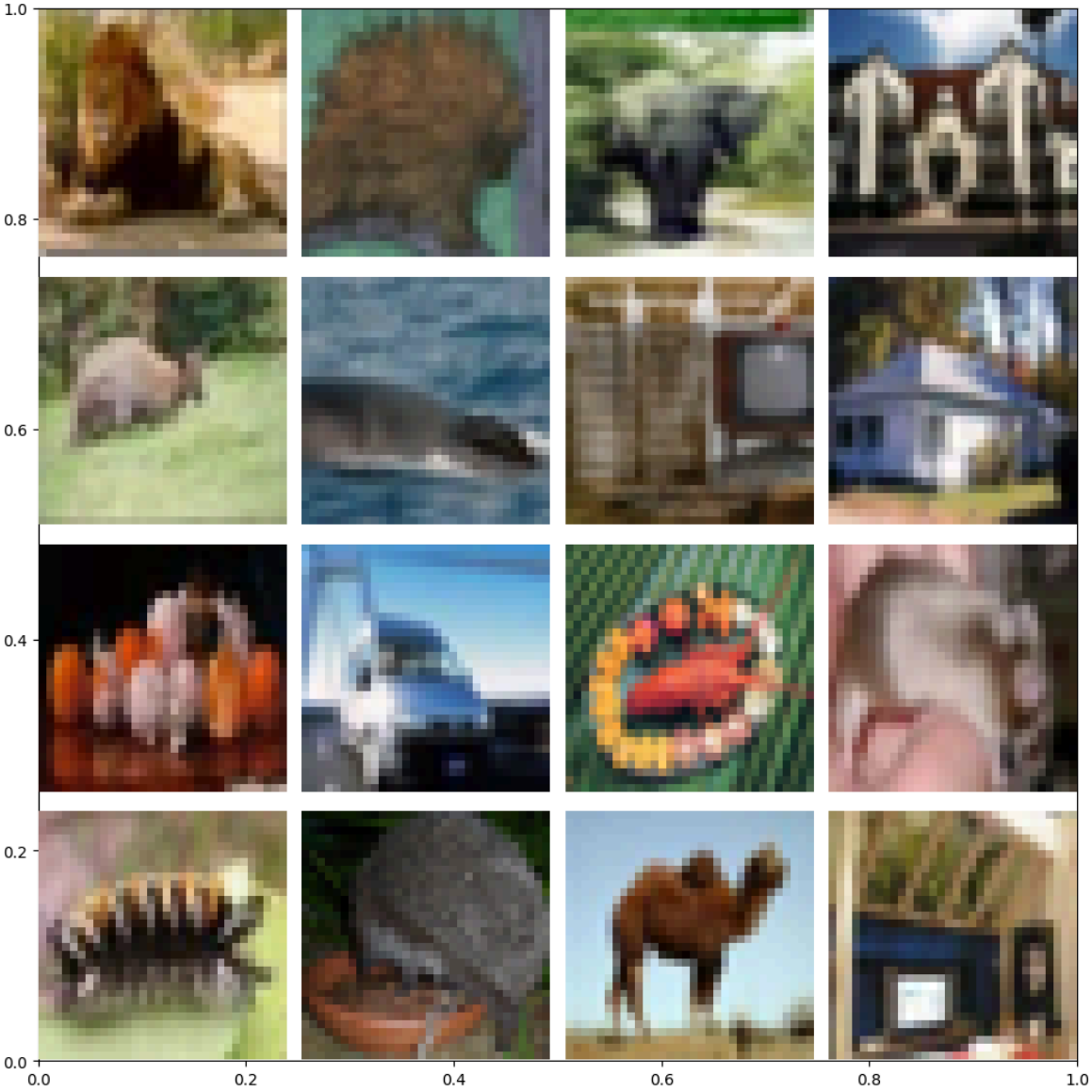
```
    del img,lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}")  # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}")  # Should be (batch_size, 10)
    del img,lbl


#### <<<<<<<<<<Pre-trained model>>>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

# Freeze the layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.4),
    layers.Dense(100, activation='softmax')  # CIFAR-100 has 100 classes
])

for sample in test_dataset.take(1):
    print(type(sample))  # Should be <class 'tuple'>
    print(len(sample))  # Should be 2
    print(type(sample[0]), type(sample[1]))  # Both should be <class 'tensor
    print(sample[0].shape)  # Should be (batch_size, 224, 224, 3)
    print(sample[1].shape)  # Should be (batch_size, 100)
print(f"Model input shape: {model.input_shape}")
print(f"Model output shape: {model.output_shape}")
sample = next(iter(test_dataset.as_numpy_iterator()))
print(len(sample))  # Should be 2
print(type(sample[0]), type(sample[1]))  # Both should be <class 'numpy.ndar
print(sample[0].shape, sample[1].shape)  # Should match model input and outp
print("\n")
#for x, y in test_dataset.take(1):
#    print(type(x), type(y))  # Both should be <class 'tensorflow.Tensor'>
#for x_batch, y_batch in test_dataset.take(1):
#    test_loss, test_acc = model.evaluate(x_batch, y_batch)
#    print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

# Compile the model
#tensorboard_callback = keras.callbacks.TensorBoard(log_dir="./logs")
model.compile(optimizer=optimizers.Adam(learning_rate=1e-3, weight_decay=1e-
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>
early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
                               patience=5, # Num. epochs with no improvement
                               restore_best_weights=True)
#reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
#                              factor=0.1, # Reduce lr by a factor
#                              patience=3, # Num epochs w/ no improvement
#                              min_lr=1e-6, # Min lr
```

```
#                                verbose=1)
#tensorboard = TensorBoard(log_dir='./logs', # Logs directory
#                           histogram_freq=1, # Logs histograms for weights/ac
#                           write_graph=True, # Logs graph of model
#                           write_images=True) # Log images like weight histog
#checkpoint = ModelCheckpoint('best_model.h5',
#                             monitor='val_loss', # or val_accuracy
#                             save_best_only=True, # Save only best model
#                             mode='min', # min for loss or max for accuracy
#                             verbose=1)
#cvs_logger = CSVLogger('training_log.csv', seperator=',', append=True) # Sa

# Train the model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
                    batch_size=batch_size, callbacks=[early_stopping], verbo

#### <<<<<<<<<<Plot Training & Validation Error>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1,len(history.history['loss'])+1)
train_loss = history.history['loss']
val_loss = history.history['val_loss']
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

def plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc):
    # Plot Training and Validation Loss
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.plot(epochs, train_loss, label='Training Loss')
    plt.plot(epochs, val_loss, label='Validation Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Training and Validation Loss')
    plt.legend()
    plt.grid()

    # Plot Training and Validation Accuracy
    plt.subplot(1, 2, 2)
    plt.plot(epochs, train_acc, label='Training Accuracy')
    plt.plot(epochs, val_acc, label='Validation Accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.title('Training and Validation Accuracy')
    plt.legend()
    plt.grid()

    plt.tight_layout()
    plt.show()

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>

# Evaluate on test data
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
```

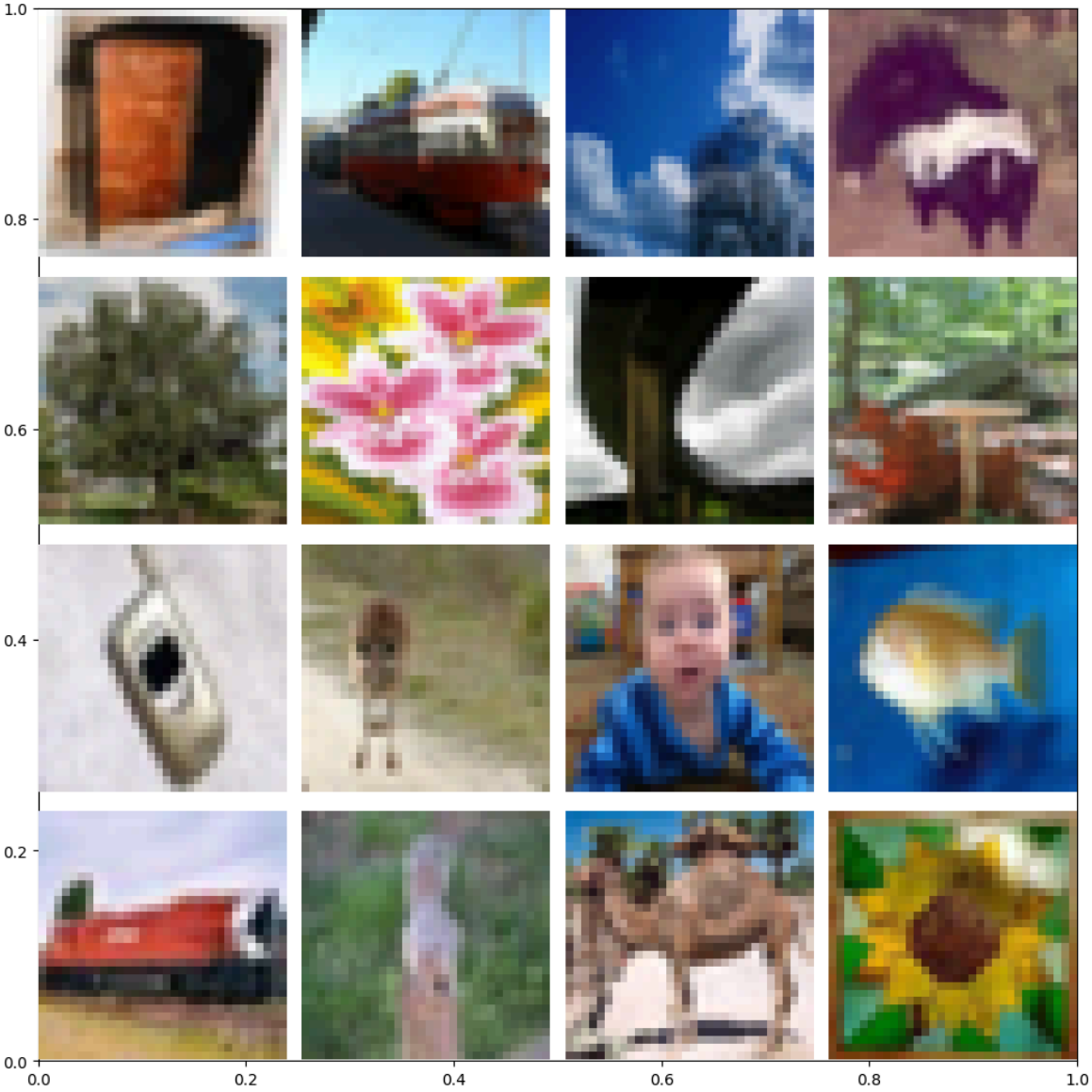
```
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss*100:.2f}%")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<<<Generate Confusion Matrix>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<<<Fine-Tune>>>>>>>>>>
#### <<<<<<<<<<Adapt Model>>>>>>>>>>
# Unfreeze last 10 layers
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ lower learning rate (prevents destroying learned features
model.compile(optimizer=optimizers.Adam(learning_rate=1e-5, weight_decay=1e-
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<<<Modify Dataset>>>>>>>>>>
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                .map(augment_dataset, num_parallel_calls=tf.data.experiment
                .batch(batch_size)
                .prefetch(tf.data.experimental.AUTOTUNE))
train_dataset_aug = train_dataset.concatenate(train_dataset_aug)

# Not val or test as augment train helps generalise better, but want to prov

#### <<<<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>
```

```
# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset
                              batch_size=batch_size, callbacks=[early_stoppi

#### <<<<<<<<<<Plot Training & Validation Error>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1,len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>

# Evaluate on test data
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss*100:.2f}%")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<<<Generate Confusion Matrix>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

# Finally, display the hist DataFrame.
hist
```

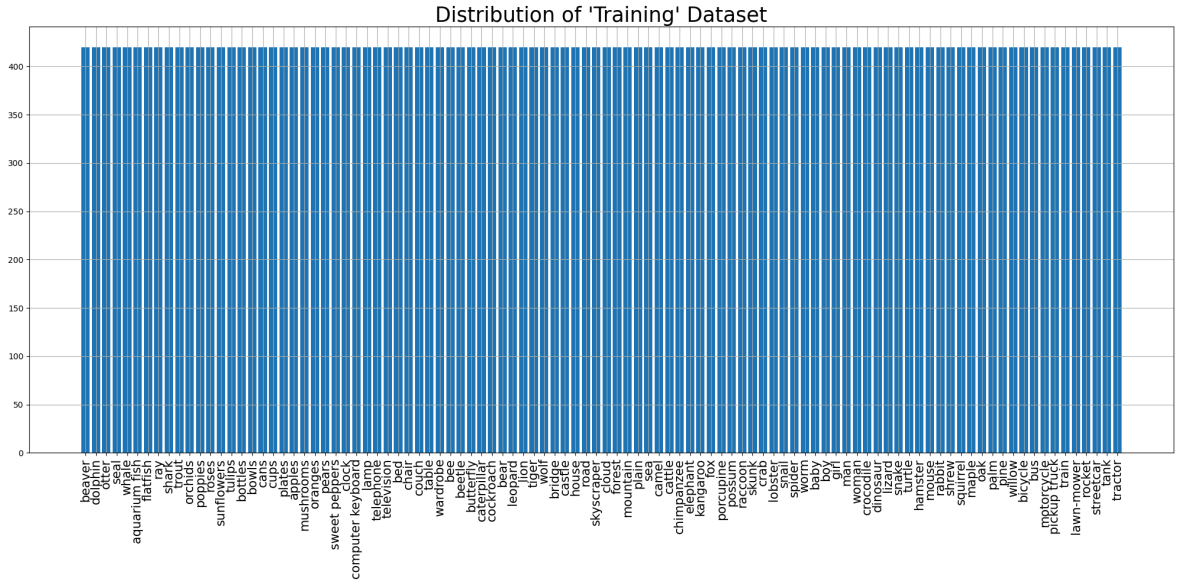
```
X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)
y_temp_val.shape: (4000, 1)
X_temp_test.shape: (4000, 32, 32, 3)
y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)
y_train.shape: (42000, 1)
X_val.shape: (9000, 32, 32, 3)
y_val.shape: (9000, 1)
X_test.shape: (9000, 32, 32, 3)
y_test.shape: (9000, 1)

labels[32535].shape: (1,)
imgs[32535].shape: (32, 32, 3)
```



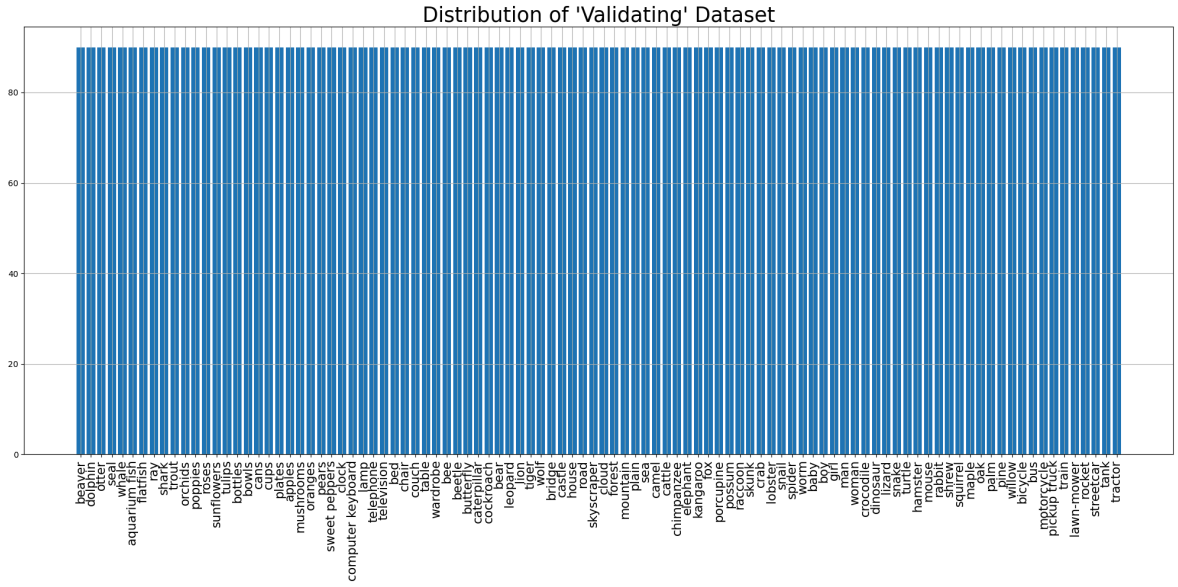

```
labels[21207].shape: (1,)
imgs[21207].shape: (32, 32, 3)
```



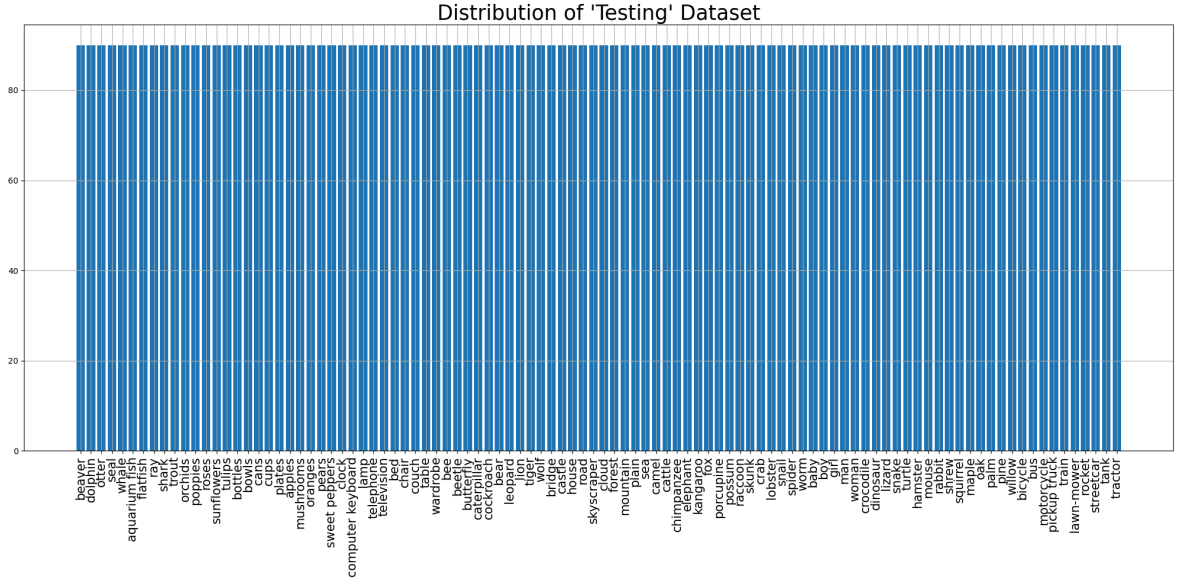

```
<ipython-input-7-634b80239b21>:70: UserWarning: set_ticklabels() should only be u
sed with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```




```
<ipython-input-7-634b80239b21>:70: UserWarning: set_ticklabels() should only be u
sed with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```




```
<ipython-input-7-634b80239b21>:70: UserWarning: set_ticklabels() should only be u
sed with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
Training dataset:
 <_ConcatenateDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=t
f.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=Non
e))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)

Validation dataset:
 <_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.f
loat32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)

Testing dataset:
 <_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.f
loat32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.f
ramework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_6"

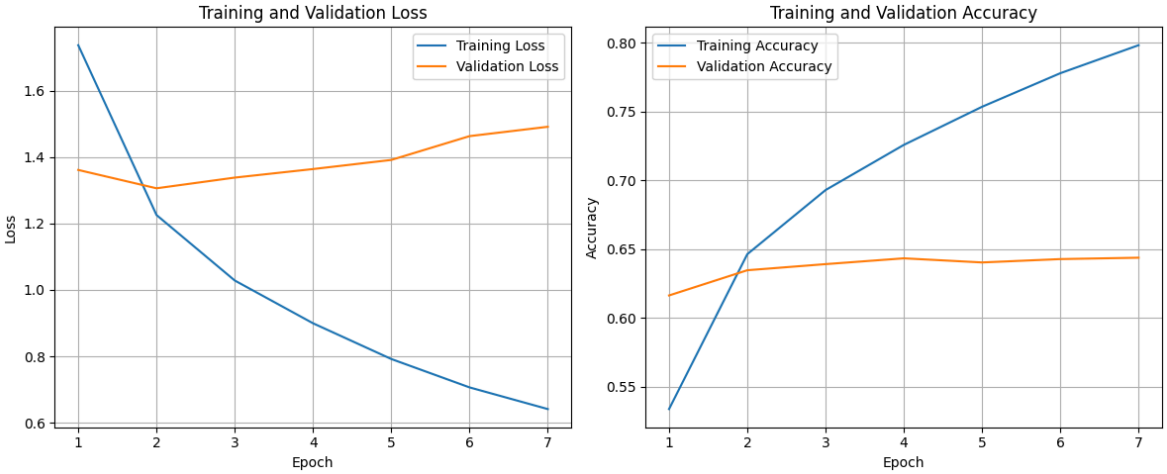
Layer (type)	Output Shape
resnet50v2 (Functional)	(None, 7, 7, 2048)
global_average_pooling2d_6 (GlobalAveragePooling2D)	(None, 2048)
dense_12 (Dense)	(None, 512)
dropout_6 (Dropout)	(None, 512)
dense_13 (Dense)	(None, 100)

Total params: 24,665,188 (94.09 MB)

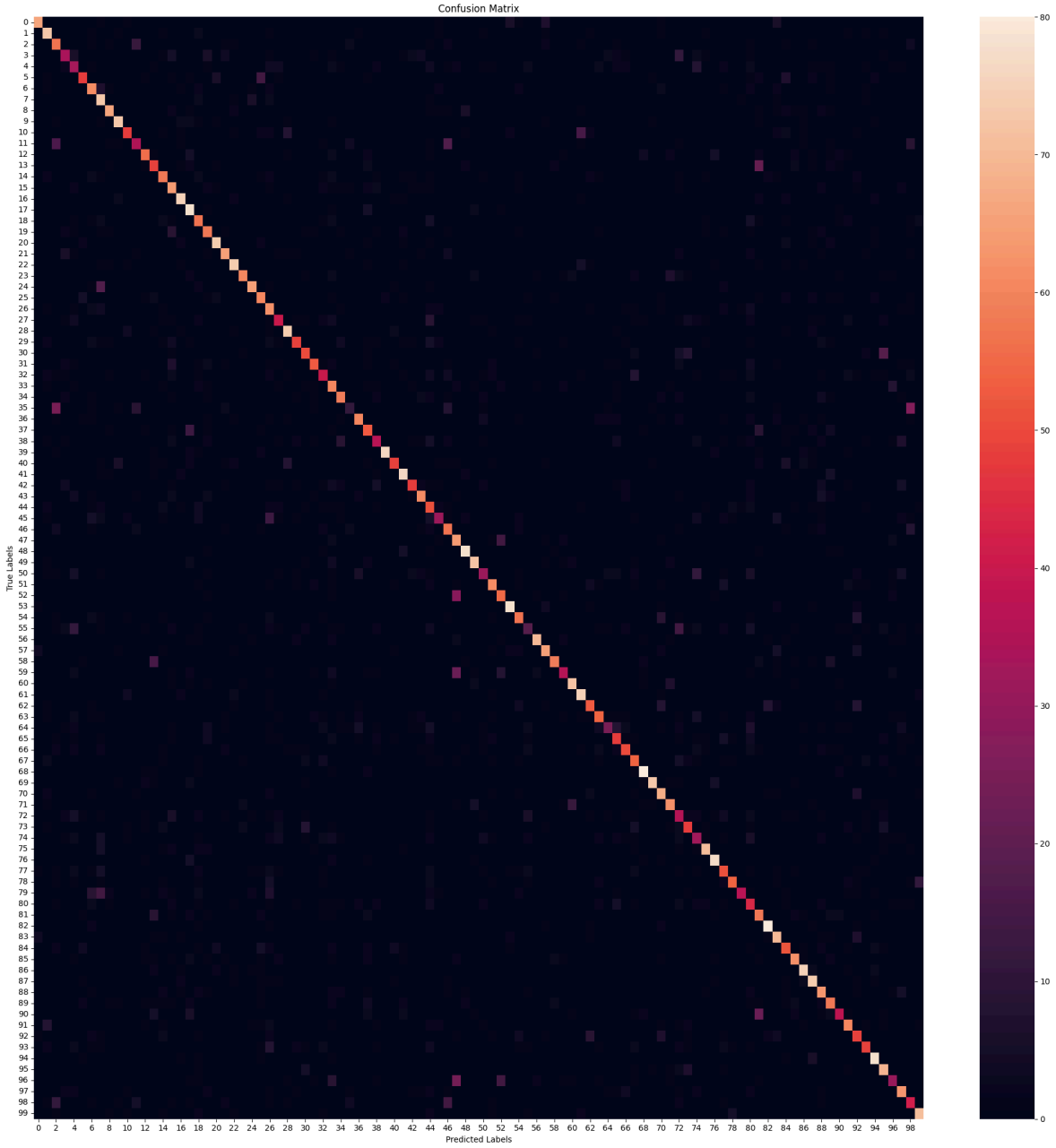
Trainable params: 1,100,388 (4.20 MB)

Non-trainable params: 23,564,800 (89.89 MB)

```
Epoch 1/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 60s 37ms/step - accuracy: 0.4486 - f1_score: 0.445
3 - loss: 2.1855 - precision: 0.7266 - val_accuracy: 0.6162 - val_f1_score: 0.612
2 - val_loss: 1.3611 - val_precision: 0.7893
Epoch 2/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 37s 28ms/step - accuracy: 0.6386 - f1_score: 0.636
5 - loss: 1.2597 - precision: 0.7986 - val_accuracy: 0.6346 - val_f1_score: 0.631
8 - val_loss: 1.3058 - val_precision: 0.7873
Epoch 3/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 37s 28ms/step - accuracy: 0.6919 - f1_score: 0.690
1 - loss: 1.0296 - precision: 0.8204 - val_accuracy: 0.6390 - val_f1_score: 0.636
0 - val_loss: 1.3381 - val_precision: 0.7705
Epoch 4/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 37s 28ms/step - accuracy: 0.7281 - f1_score: 0.726
3 - loss: 0.8871 - precision: 0.8355 - val_accuracy: 0.6432 - val_f1_score: 0.641
9 - val_loss: 1.3638 - val_precision: 0.7644
Epoch 5/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 37s 28ms/step - accuracy: 0.7597 - f1_score: 0.758
4 - loss: 0.7654 - precision: 0.8513 - val_accuracy: 0.6402 - val_f1_score: 0.638
3 - val_loss: 1.3912 - val_precision: 0.7525
Epoch 6/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 37s 28ms/step - accuracy: 0.7881 - f1_score: 0.786
9 - loss: 0.6656 - precision: 0.8652 - val_accuracy: 0.6427 - val_f1_score: 0.642
3 - val_loss: 1.4629 - val_precision: 0.7420
Epoch 7/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 37s 28ms/step - accuracy: 0.8119 - f1_score: 0.810
7 - loss: 0.5892 - precision: 0.8794 - val_accuracy: 0.6437 - val_f1_score: 0.641
7 - val_loss: 1.4911 - val_precision: 0.7374
```



```
141/141 ━━━━━━━━━━━━━━━━━━━━ 4s 25ms/step - accuracy: 0.6271 - f1_score: 0.6182 -
loss: 1.3173 - precision: 0.7848
Test Accuracy: 63.41%
Test Loss: 130.31%
Test Precision: 78.97%
Test F1 Scores (Per Class): [79.04191  73.36682  52.054794 44.155838 34.736835 6
1.146492 60.396034
 59.109306 77.19298  81.56424  61.93548  44.871788 68.71165  53.260864
 63.387978 58.986176 80.21389  67.82608  56.15763  60.416664 81.31868
 73.863625 80.43478  74.99999  76.92307  63.492058 50.612247 49.382713
 78.30688  64.10256  59.171593 67.51592  48.78048  51.948048 66.29213
 19.999996 70.58823  57.92349  47.13375  90.47619  64.052284 80.42328
 63.157898 71.345024 43.03797  44.60431  54.285706 52.459015 83.87096
 79.55801  41.891888 70.93022  59.459454 85.71427  64.04494  25.37313
 82.84023  73.44632  74.21383  51.063824 79.55801  76.923065 61.9883
 63.905323 38.75969  54.23729  61.349686 58.378376 86.02149  86.390526
 66.666664 70.45454  35.12195  57.142853 36.781605 80.681816 83.24323
 55.737698 68.78981  56.488544 38.766514 52.488686 86.48647  69.99999
 57.142853 71.26436  80.64516  75.5102   62.06896  61.052628 52.054794
 67.796616 51.041664 62.420376 82.53968  67.980286 41.095886 64.94845
 43.076916 78.02197 ]
Average Test F1 Scores:63.23

282/282 ━━━━━━━━━━━━━━━━━━━━ 12s 29ms/step
```


Confusion Matrix
True Labels
Predicted Labels

	precision	recall	f1-score	support
beaver	0.86	0.73	0.79	90
dolphin	0.67	0.81	0.73	90
otter	0.44	0.63	0.52	90
seal	0.53	0.38	0.44	90
whale	0.33	0.37	0.35	90
aquarium fish	0.72	0.53	0.61	90
flatfish	0.54	0.68	0.60	90
ray	0.46	0.81	0.59	90
shark	0.81	0.73	0.77	90
trout	0.82	0.81	0.82	90
orchids	0.74	0.53	0.62	90
poppies	0.52	0.39	0.45	90
roses	0.77	0.62	0.69	90
sunflowers	0.52	0.54	0.53	90
tulips	0.62	0.64	0.63	90
bottles	0.50	0.71	0.59	90
bowls	0.77	0.83	0.80	90
cans	0.56	0.87	0.68	90
cups	0.50	0.63	0.56	90
plates	0.57	0.64	0.60	90
apples	0.80	0.82	0.81	90
mushrooms	0.76	0.72	0.74	90
oranges	0.79	0.82	0.80	90
pears	0.86	0.67	0.75	90
sweet peppers	0.82	0.72	0.77	90
clock	0.61	0.67	0.63	90
computer keyboard	0.40	0.69	0.51	90
lamp	0.56	0.44	0.50	90
telephone	0.75	0.82	0.78	90
television	0.75	0.54	0.63	90
bed	0.63	0.56	0.59	90
chair	0.79	0.59	0.68	90
couch	0.54	0.44	0.49	90
table	0.43	0.67	0.52	90
wardrobe	0.67	0.66	0.66	90
bee	0.38	0.12	0.18	90
beetle	0.75	0.67	0.71	90
butterfly	0.56	0.59	0.58	90
caterpillar	0.55	0.41	0.47	90
cockroach	0.97	0.84	0.90	90
bear	0.78	0.54	0.64	90
leopard	0.77	0.84	0.80	90
lion	0.76	0.53	0.63	90
tiger	0.75	0.68	0.71	90
wolf	0.35	0.57	0.43	90
bridge	0.63	0.34	0.45	90
castle	0.47	0.63	0.54	90
house	0.42	0.71	0.52	90
road	0.81	0.87	0.84	90
skyscraper	0.79	0.80	0.80	90
cloud	0.53	0.34	0.42	90
forest	0.74	0.68	0.71	90
mountain	0.58	0.61	0.59	90
plain	0.85	0.87	0.86	90
sea	0.65	0.63	0.64	90
camel	0.40	0.20	0.27	90
cattle	0.89	0.78	0.83	90
chimpanzee	0.75	0.72	0.73	90

elephant	0.86	0.66	0.74	90
kangaroo	0.71	0.40	0.51	90
fox	0.79	0.80	0.80	90
porcupine	0.71	0.83	0.77	90
possum	0.65	0.59	0.62	90
raccoon	0.68	0.60	0.64	90
skunk	0.64	0.28	0.39	90
crab	0.55	0.53	0.54	90
lobster	0.69	0.56	0.62	90
snail	0.57	0.60	0.58	90
spider	0.83	0.89	0.86	90
worm	0.92	0.81	0.86	90
baby	0.60	0.76	0.67	90
boy	0.72	0.69	0.70	90
girl	0.31	0.40	0.35	90
man	0.62	0.53	0.57	90
woman	0.38	0.36	0.37	90
crocodile	0.83	0.79	0.81	90
dinosaur	0.81	0.86	0.83	90
lizard	0.55	0.57	0.56	90
snake	0.81	0.60	0.69	90
turtle	0.90	0.41	0.56	90
hamster	0.32	0.49	0.39	90
mouse	0.44	0.64	0.52	90
rabbit	0.84	0.89	0.86	90
shrew	0.64	0.78	0.70	90
squirrel	0.57	0.58	0.57	90
maple	0.74	0.69	0.71	90
oak	0.79	0.83	0.81	90
palm	0.70	0.82	0.76	90
pine	0.56	0.70	0.62	90
willow	0.59	0.64	0.61	90
bicycle	0.68	0.42	0.52	90
bus	0.69	0.67	0.68	90
motorcycle	0.48	0.54	0.51	90
pickup truck	0.73	0.54	0.62	90
train	0.79	0.87	0.83	90
lawn-mower	0.61	0.77	0.68	90
rocket	0.53	0.33	0.41	90
streetcar	0.61	0.70	0.65	90
tank	0.40	0.47	0.43	90
tractor	0.77	0.79	0.78	90
accuracy			0.63	9000
macro avg	0.65	0.63	0.63	9000
weighted avg	0.65	0.63	0.63	9000

Model: "sequential_6"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None, 7, 7, 2048)
global_average_pooling2d_6 (GlobalAveragePooling2D)	(None, 2048)
dense_12 (Dense)	(None, 512)
dropout_6 (Dropout)	(None, 512)
dense_13 (Dense)	(None, 100)

Total params: 24,665,188 (94.09 MB)

Trainable params: 24,619,748 (93.92 MB)

Non-trainable params: 45,440 (177.50 KB)

```
Epoch 1/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 218s 84ms/step - accuracy: 0.6530 - f1_score: 0.65
20 - loss: 1.2360 - precision: 0.8174 - val_accuracy: 0.7723 - val_f1_score: 0.77
07 - val_loss: 0.8001 - val_precision: 0.8495
Epoch 2/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 138s 70ms/step - accuracy: 0.8568 - f1_score: 0.85
56 - loss: 0.4579 - precision: 0.9127 - val_accuracy: 0.7880 - val_f1_score: 0.78
73 - val_loss: 0.7867 - val_precision: 0.8356
Epoch 3/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 137s 70ms/step - accuracy: 0.9365 - f1_score: 0.93
57 - loss: 0.2067 - precision: 0.9585 - val_accuracy: 0.7936 - val_f1_score: 0.79
33 - val_loss: 0.8135 - val_precision: 0.8324
Epoch 4/15
1971/1971 ━━━━━━━━━━━━━━━━━━━━ 137s 69ms/step - accuracy: 0.9725 - f1_score: 0.97
19 - loss: 0.0988 - precision: 0.9808 - val_accuracy: 0.7937 - val_f1_score: 0.79
38 - val_loss: 0.8838 - val_precision: 0.8262
Epoch 5/15
1209/1971 ━━━━━━━━━━━━──────── 51s 68ms/step - accuracy: 0.9891 - f1_score: 0.988
3 - loss: 0.0470 - precision: 0.9920
```

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
<ipython-input-7-634b80239b21> in <cell line: 0>()
    308
    309     # Train the model
--> 310     history_fine_tune = model.fit(train_dataset_aug, validation_data=val_
dataset, epochs=15,
    311                                   batch_size=batch_size, callbacks=[early
_stopping], verbose=1)
    312

/usr/local/lib/python3.11/dist-packages/keras/src/utils/traceback_utils.py in err
or_handler(*args, **kwargs)
    115         filtered_tb = None
    116         try:
--> 117             return fn(*args, **kwargs)
    118         except Exception as e:
    119             filtered_tb = _process_traceback_frames(e.__traceback__)

/usr/local/lib/python3.11/dist-packages/keras/src/backend/tensorflow/trainer.py i
n fit(self, x, y, batch_size, epochs, verbose, callbacks, validation_split, valid
ation_data, shuffle, class_weight, sample_weight, initial_epoch, steps_per_epoch,
validation_steps, validation_batch_size, validation_freq)
    369                 for step, iterator in epoch_iterator:
    370                     callbacks.on_train_batch_begin(step)
--> 371                     logs = self.train_function(iterator)
    372                     callbacks.on_train_batch_end(step, logs)
    373                     if self.stop_training:

/usr/local/lib/python3.11/dist-packages/keras/src/backend/tensorflow/trainer.py i
n function(iterator)
    217                 iterator, (tf.data.Iterator, tf.distribute.DistributedIte
rator)
    218             ):
--> 219                 opt_outputs = multi_step_on_iterator(iterator)
    220                 if not opt_outputs.has_value():
    221                     raise StopIteration

/usr/local/lib/python3.11/dist-packages/tensorflow/python/util/traceback_utils.py
in error_handler(*args, **kwargs)
    148     filtered_tb = None
    149     try:
--> 150       return fn(*args, **kwargs)
    151     except Exception as e:
    152       filtered_tb = _process_traceback_frames(e.__traceback__)

/usr/local/lib/python3.11/dist-packages/tensorflow/python/eager/polymorphic_funct
ion/polymorphic_function.py in __call__(self, *args, **kwds)
    831
    832       with OptionalXlaContext(self._jit_compile):
--> 833         result = self._call(*args, **kwds)
    834
    835       new_tracing_count = self.experimental_get_tracing_count()

/usr/local/lib/python3.11/dist-packages/tensorflow/python/eager/polymorphic_funct
ion/polymorphic_function.py in _call(self, *args, **kwds)
    876       # In this case we have not created variables on the first call. So
we can
    877       # run the first trace but we should fail if variables are created.
--> 878       results = tracing_compilation.call_function(
```

```
    879           args, kwds, self._variable_creation_config
    880       )

/usr/local/lib/python3.11/dist-packages/tensorflow/python/eager/polymorphic_funct
ion/tracing_compilation.py in call_function(args, kwargs, tracing_options)
    137   bound_args = function.function_type.bind(*args, **kwargs)
    138   flat_inputs = function.function_type.unpack_inputs(bound_args)
--> 139   return function._call_flat(  # pylint: disable=protected-access
    140       flat_inputs, captured_inputs=function.captured_inputs
    141   )

/usr/local/lib/python3.11/dist-packages/tensorflow/python/eager/polymorphic_funct
ion/concrete_function.py in _call_flat(self, tensor_inputs, captured_inputs)
   1320         and executing_eagerly):
   1321       # No tape is watching; skip to running the function.
-> 1322       return self._inference_function.call_preflattened(args)
   1323     forward_backward = self._select_forward_and_backward_functions(
   1324         args,

/usr/local/lib/python3.11/dist-packages/tensorflow/python/eager/polymorphic_funct
ion/atomic_function.py in call_preflattened(self, args)
    214   def call_preflattened(self, args: Sequence[core.Tensor]) -> Any:
    215     """Calls with flattened tensor inputs and returns the structured outp
ut."""
--> 216     flat_outputs = self.call_flat(*args)
    217     return self.function_type.pack_output(flat_outputs)
    218

/usr/local/lib/python3.11/dist-packages/tensorflow/python/eager/polymorphic_funct
ion/atomic_function.py in call_flat(self, *args)
    249         with record.stop_recording():
    250           if self._bound_context.executing_eagerly():
--> 251             outputs = self._bound_context.call_function(
    252                 self.name,
    253                 list(args),

/usr/local/lib/python3.11/dist-packages/tensorflow/python/eager/context.py in cal
l_function(self, name, tensor_inputs, num_outputs)
   1681     cancellation_context = cancellation.context()
   1682     if cancellation_context is None:
-> 1683       outputs = execute.execute(
   1684           name.decode("utf-8"),
   1685           num_outputs=num_outputs,

/usr/local/lib/python3.11/dist-packages/tensorflow/python/eager/execute.py in qui
ck_execute(op_name, num_outputs, inputs, attrs, ctx, name)
     51   try:
     52     ctx.ensure_initialized()
---> 53     tensors = pywrap_tfe.TFE_Py_Execute(ctx._handle, device_name, op_nam
e,
     54                                         inputs, attrs, num_outputs)
     55   except core._NotOkStatusException as e:

KeyboardInterrupt:
```

Use weight decay inside layers

In [10]:
```
for repeat_2_times in range(1):
    #### <<<<<<<<<<Load and process data>>>>>>>>>>
    # Load CIFAR-100 dataset
    (X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

    # Split (8000) of training data into temporary set
    X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_s
    print(f"X_temp.shape: {X_temp.shape}\n")

    # Split temp data into equal validation (4000) and testing (4000) data
    X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp,
    print(f"X_temp_val.shape: {X_temp_val.shape}")
    print(f"y_temp_val.shape: {y_temp_val.shape}")
    print(f"X_temp_test.shape: {X_temp_test.shape}")
    print(f"y_temp_test.shape: {y_temp_test.shape}\n")

    # Split test data into validation (5000) and testing (5000)
    X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.

    # Add temp_val to validation (9000) and temp_test to testing (9000) to get a
    X_val = np.concatenate((X_val, X_temp_val), axis=0)
    y_val = np.concatenate((y_val, y_temp_val), axis=0)
    X_test = np.concatenate((X_test, X_temp_test), axis=0)
    y_test = np.concatenate((y_test, y_temp_test), axis=0)

    print(f"X_train.shape: {X_train.shape}")
    print(f"y_train.shape: {y_train.shape}")
    print(f"X_val.shape: {X_val.shape}")
    print(f"y_val.shape: {y_val.shape}")
    print(f"X_test.shape: {X_test.shape}")
    print(f"y_test.shape: {y_test.shape}\n")

    def display_imgs(imgs, labels):
        plt.subplots(figsize=(10,10))
        for i in range(16):
            plt.subplot(4, 4, i+1)
            k = np.random.randint(0, imgs.shape[0])
            if i == 0:
                print(f"labels[{k}].shape: {labels[k].shape}")
                print(f"imgs[{k}].shape: {imgs[k].shape}")
            plt.imshow(imgs[k])
            #plt.title(labels[k])
            plt.axis('off')
        plt.tight_layout()
        plt.show()

    display_imgs(X_train, y_train)

    # Normalise images (scale to range [0, 1]) - Improves convergence speed & ac
    X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
    display_imgs(X_train, y_train)

    labels_names = ['beaver','dolphin','otter','seal','whale','aquarium fish','f
                    'orchids','poppies','roses','sunflowers','tulips','bottles','
                    'apples','mushrooms','oranges','pears','sweet peppers','clock
                    'telephone','television','bed','chair','couch','table','wardr
                    'caterpillar','cockroach','bear','leopard','lion','tiger','wo
                    'road','skyscraper','cloud','forest','mountain','plain','sea'
                    'elephant','kangaroo','fox','porcupine','possum','raccoon','s
```

```
                'spider','worm','baby','boy','girl','man','woman','crocodile'
                'turtle','hamster','mouse','rabbit','shrew','squirrel','maple
                'bicycle','bus','motorcycle','pickup truck','train','lawn-mow
                'tractor']

def class_distrib(y, labels_names, dataset_name):
    counts = pd.DataFrame(data=y).value_counts().sort_index()
    #print(f"counts:\n{counts}")
    fig, ax = plt.subplots(figsize=(20,10))
    ax.bar(labels_names, counts)
    ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
    plt.title(f"Distribution of '{dataset_name}' Dataset", fontsize=25)
    plt.grid()
    plt.tight_layout()
    plt.show()
class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64
train_dataset_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtyp
                .batch(batch_size)
                .prefetch(tf.data.experimental.AUTOTUNE))

def augment_dataset(x, y):
    x = tf.image.resize(x, (224, 224))  # Resize images
    x = tf.image.random_flip_left_right(x)  # Random horizontal flip
    x = tf.image.random_brightness(x, max_delta=0.2)  # Adjust brightness
    x = tf.image.random_contrast(x, lower=0.8, upper=1.2)  # Adjust contrast
    y = tf.squeeze(tf.one_hot(y, depth=100, dtype=tf.float32))  # One-hot en
    return x, y
train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                .map(augment_dataset, num_parallel_calls=tf.data.experiment
                .batch(batch_size)
                .prefetch(tf.data.experimental.AUTOTUNE))

# Combine the original dataset and the augmented dataset
train_dataset = train_dataset_.concatenate(train_dataset_aug)

# Optionally, shuffle the combined dataset if needed
#combined_dataset = combined_dataset.shuffle(buffer_size=1000)  # Adjust buf

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                  tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
                .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtyp
                .batch(batch_size)
                .prefetch(tf.data.experimental.AUTOTUNE))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
```

```
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}")  # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}")  # Should be (batch_size, 10)
    del img,lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}")  # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}")  # Should be (batch_size, 10)
    del img,lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}")  # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}")  # Should be (batch_size, 10)
    del img,lbl


#### <<<<<<<<<<Pre-trained model>>>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

for layer in resnet_50_base.layers:
    if isinstance(layer, tf.keras.layers.Conv2D):
        layer.kernel_regularizer = tf.keras.regularizers.l2(3e-4)
# Freeze the layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.4),
    layers.Dense(100, activation='softmax')  # CIFAR-100 has 100 classes
])

for sample in test_dataset.take(1):
    print(type(sample))  # Should be <class 'tuple'>
    print(len(sample))  # Should be 2
    print(type(sample[0]), type(sample[1]))  # Both should be <class 'tensor
    print(sample[0].shape)  # Should be (batch_size, 224, 224, 3)
    print(sample[1].shape)  # Should be (batch_size, 100)
print(f"Model input shape: {model.input_shape}")
print(f"Model output shape: {model.output_shape}")
sample = next(iter(test_dataset.as_numpy_iterator()))
print(len(sample))  # Should be 2
print(type(sample[0]), type(sample[1]))  # Both should be <class 'numpy.ndar
print(sample[0].shape, sample[1].shape)  # Should match model input and outp
print("\n")
#for x, y in test_dataset.take(1):
#    print(type(x), type(y))  # Both should be <class 'tensorflow.Tensor'>
#for x_batch, y_batch in test_dataset.take(1):
#    test_loss, test_acc = model.evaluate(x_batch, y_batch)
#    print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

# Compile the model
#tensorboard_callback = keras.callbacks.TensorBoard(log_dir="./logs")
model.compile(optimizer=optimizers.Adam(learning_rate=1e-3),
              loss='categorical_crossentropy',
```

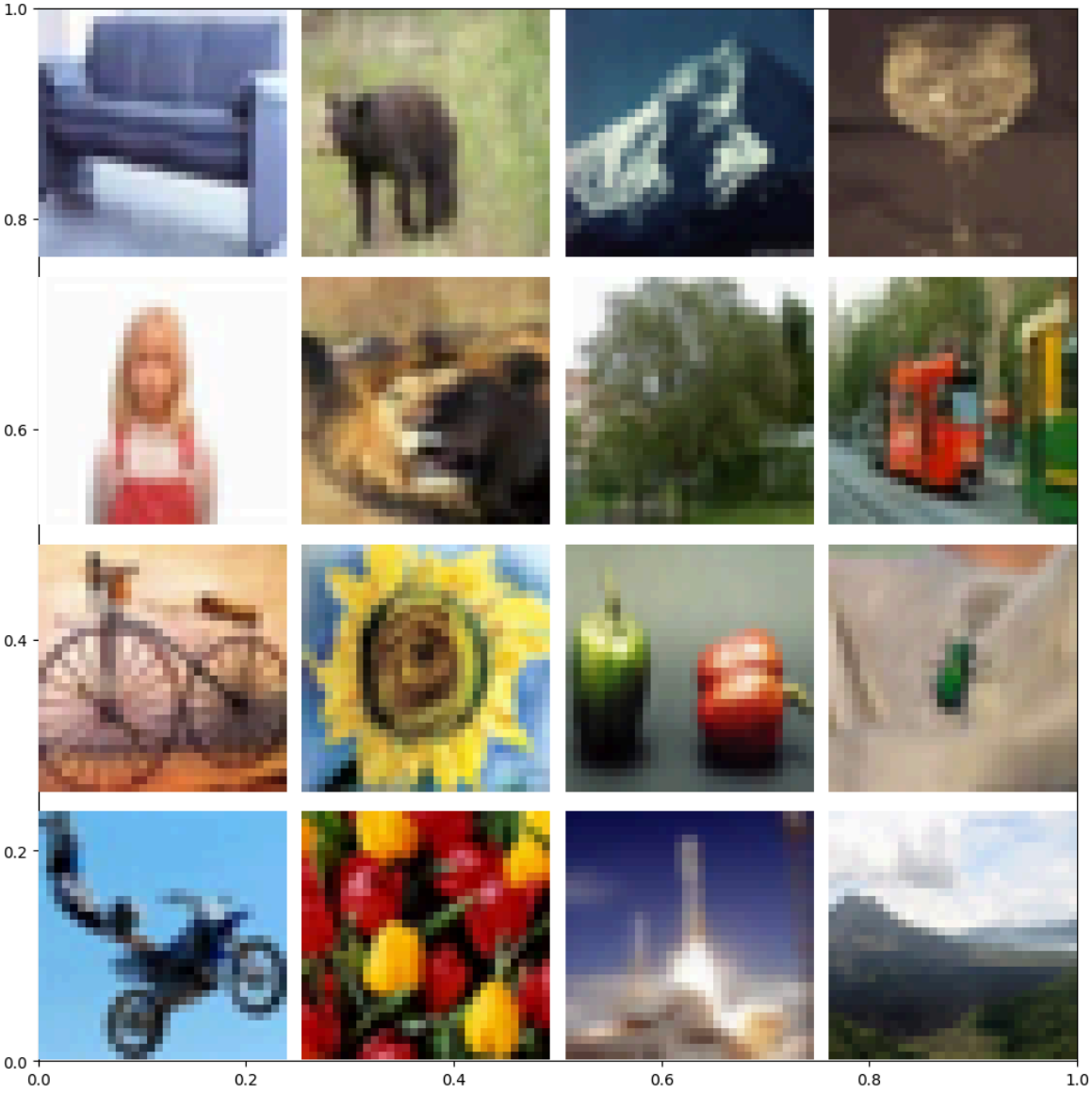
```
                metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>
early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
                               patience=5, # Num. epochs with no improvement
                               restore_best_weights=True)
#reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
#                              factor=0.1, # Reduce lr by a factor
#                              patience=3, # Num epochs w/ no improvement
#                              min_lr=1e-6, # Min lr
#                              verbose=1)
#tensorboard = TensorBoard(log_dir='./logs', # Logs directory
#                          histogram_freq=1, # Logs histograms for weights/ac
#                          write_graph=True, # Logs graph of model
#                          write_images=True) # Log images like weight histog
#checkpoint = ModelCheckpoint('best_model.h5',
#                             monitor='val_loss', # or val_accuracy
#                             save_best_only=True, # Save only best model
#                             mode='min', # min for loss or max for accuracy
#                             verbose=1)
#cvs_logger = CSVLogger('training_log.csv', seperator=',', append=True) # Sa

# Train the model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
                    batch_size=batch_size, callbacks=[early_stopping], verbo

#### <<<<<<<<<<Plot Training & Validation Error>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1,len(history.history['loss'])+1)
train_loss = history.history['loss']
val_loss = history.history['val_loss']
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

def plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc):
    # Plot Training and Validation Loss
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.plot(epochs, train_loss, label='Training Loss')
    plt.plot(epochs, val_loss, label='Validation Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Training and Validation Loss')
    plt.legend()
    plt.grid()

    # Plot Training and Validation Accuracy
    plt.subplot(1, 2, 2)
    plt.plot(epochs, train_acc, label='Training Accuracy')
    plt.plot(epochs, val_acc, label='Validation Accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.title('Training and Validation Accuracy')
    plt.legend()
    plt.grid()

    plt.tight_layout()
```

```
    plt.show()

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>

# Evaluate on test data
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss*100:.2f}%")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<<<Generate Confusion Matrix>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<<<Fine-Tune>>>>>>>>>>
#### <<<<<<<<<<Adapt Model>>>>>>>>>>
# Unfreeze last 10 layers
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ lower learning rate (prevents destroying learned features
model.compile(optimizer=optimizers.Adam(learning_rate=1e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()
```

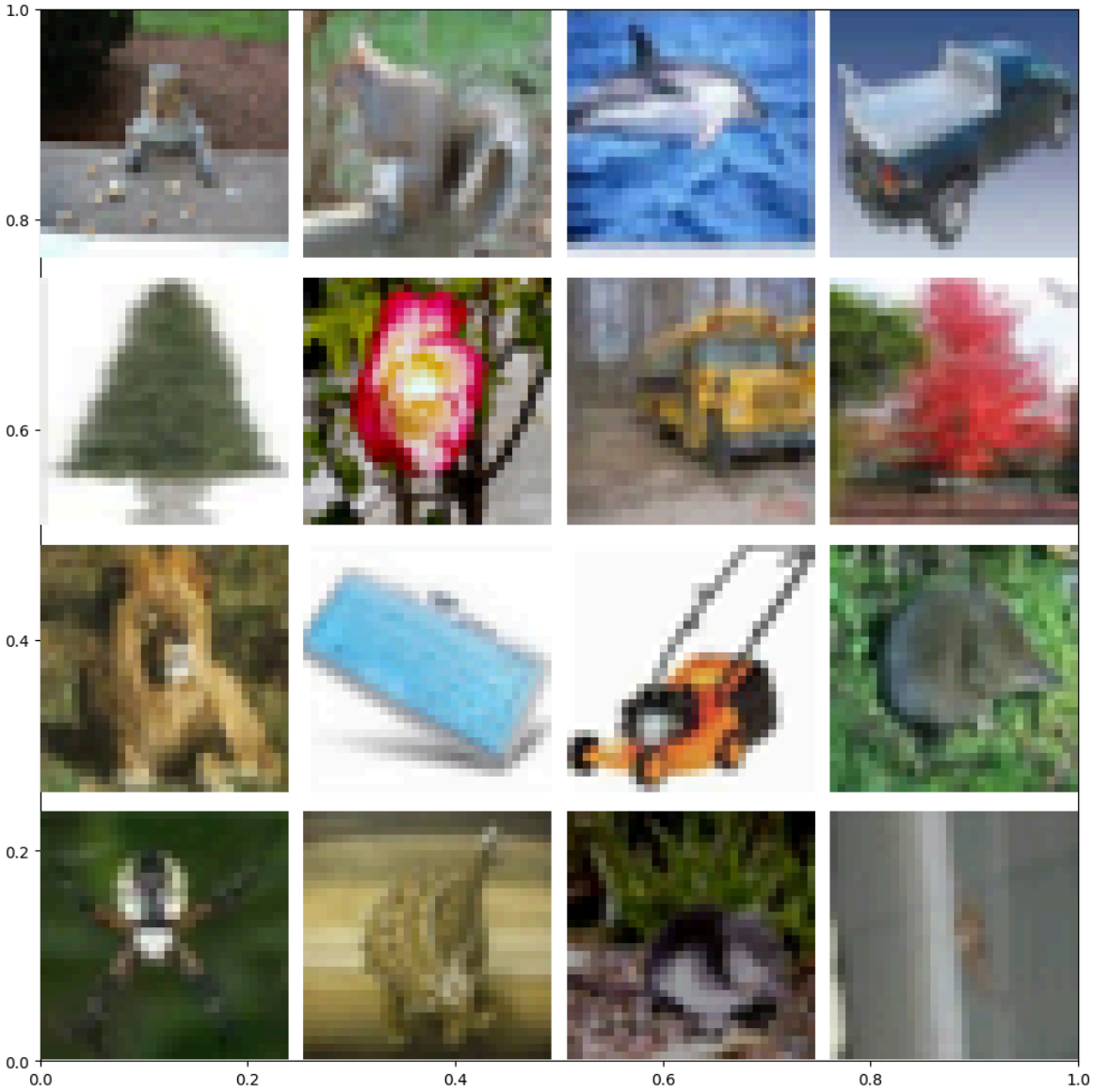
```
#### <<<<<<<<<<Modify Dataset>>>>>>>>>>
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                .map(augment_dataset, num_parallel_calls=tf.data.experiment
                .batch(batch_size)
                .prefetch(tf.data.experimental.AUTOTUNE))
train_dataset_aug = train_dataset.concatenate(train_dataset_aug)

# Not val or test as augment train helps generalise better, but want to prov

#### <<<<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>

# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset
                              batch_size=batch_size, callbacks=[early_stoppi

#### <<<<<<<<<<Plot Training & Validation Error>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1,len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>

# Evaluate on test data
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss*100:.2f}%")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<<<Generate Confusion Matrix>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()
```

```
    print(classification_report(y_true_classes, y_pred_classes, target_names=lab

    # Create a DataFrame from the history of the training and store the epoch va
    hist = pd.DataFrame(history_fine_tune.history)
    hist['epoch'] = history_fine_tune.epoch

    # Finally, display the hist DataFrame.
    hist
```

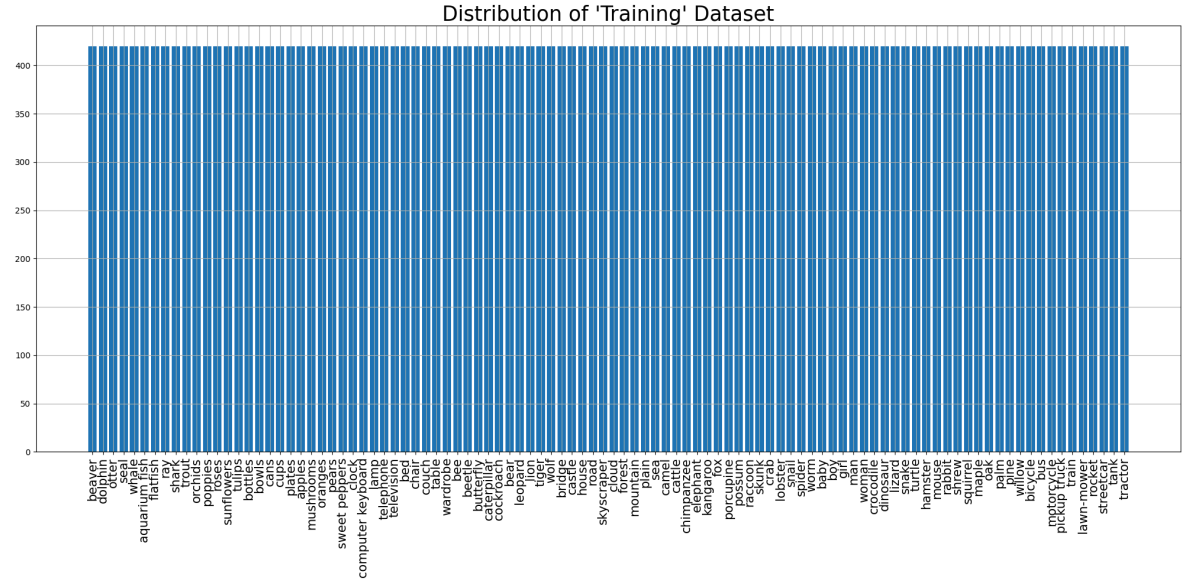
```
X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)
y_temp_val.shape: (4000, 1)
X_temp_test.shape: (4000, 32, 32, 3)
y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)
y_train.shape: (42000, 1)
X_val.shape: (9000, 32, 32, 3)
y_val.shape: (9000, 1)
X_test.shape: (9000, 32, 32, 3)
y_test.shape: (9000, 1)

labels[37344].shape: (1,)
imgs[37344].shape: (32, 32, 3)
```



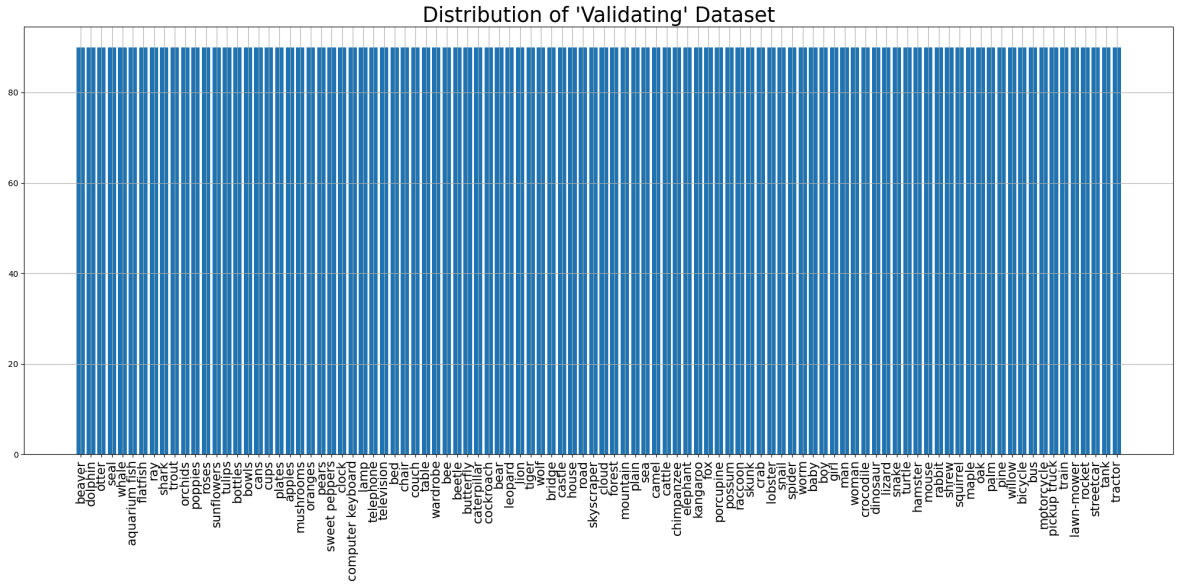
```
labels[31675].shape: (1,)
imgs[31675].shape: (32, 32, 3)
```



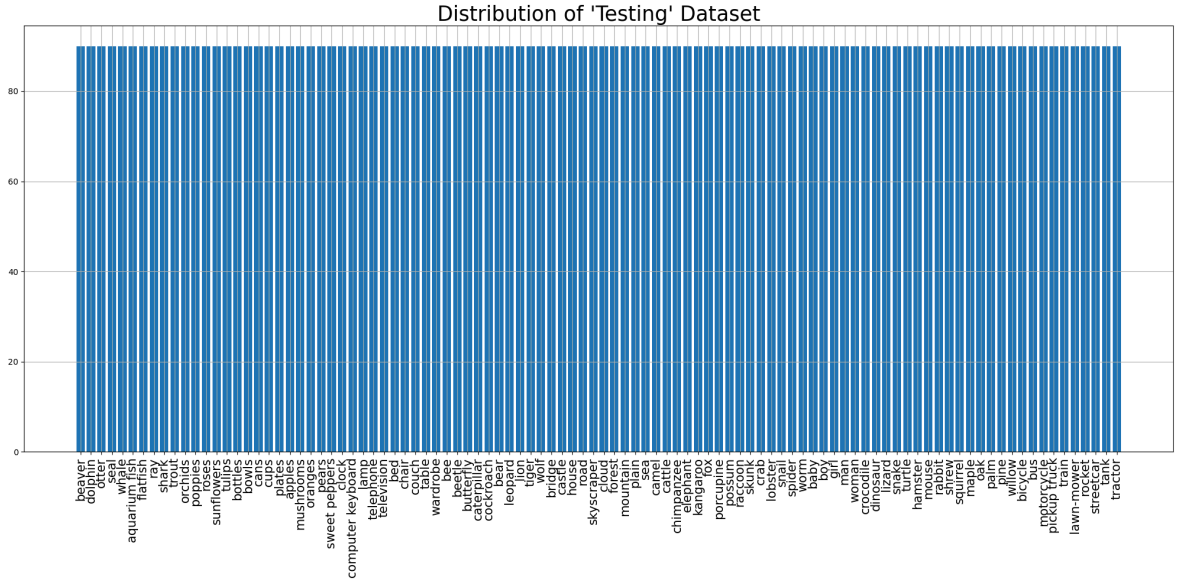
```
<ipython-input-10-e3908594f96b>:70: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-10-e3908594f96b>:70: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```




```
<ipython-input-10-e3908594f96b>:70: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
Training dataset:
 <_ConcatenateDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=t
f.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=Non
e))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)

Validation dataset:
 <_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.f
loat32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)

Testing dataset:
 <_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.f
loat32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.f
ramework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_7"

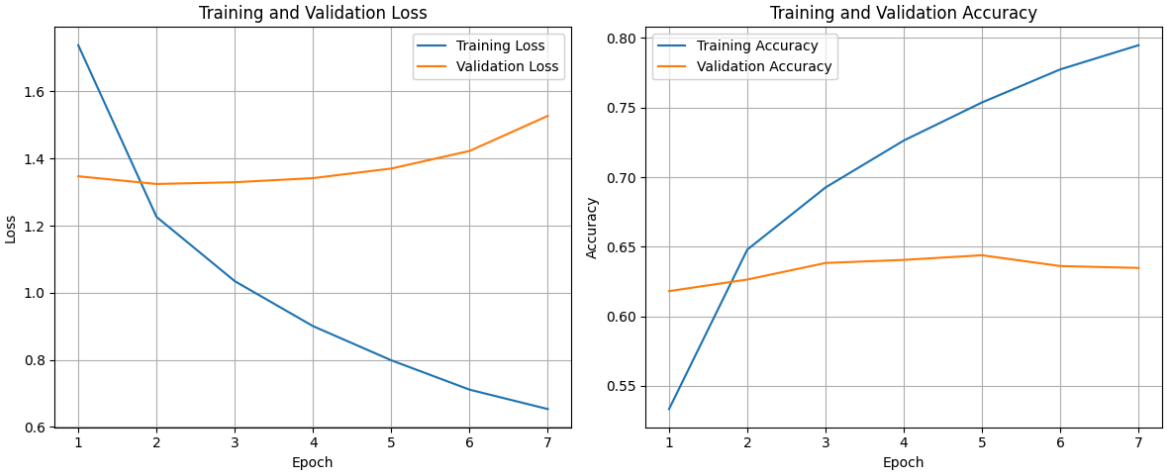
Layer (type)	Output Shape
resnet50v2 (Functional)	(None, 7, 7, 2048)
global_average_pooling2d_7 (GlobalAveragePooling2D)	(None, 2048)
dense_14 (Dense)	(None, 512)
dropout_7 (Dropout)	(None, 512)
dense_15 (Dense)	(None, 100)

Total params: 24,665,188 (94.09 MB)

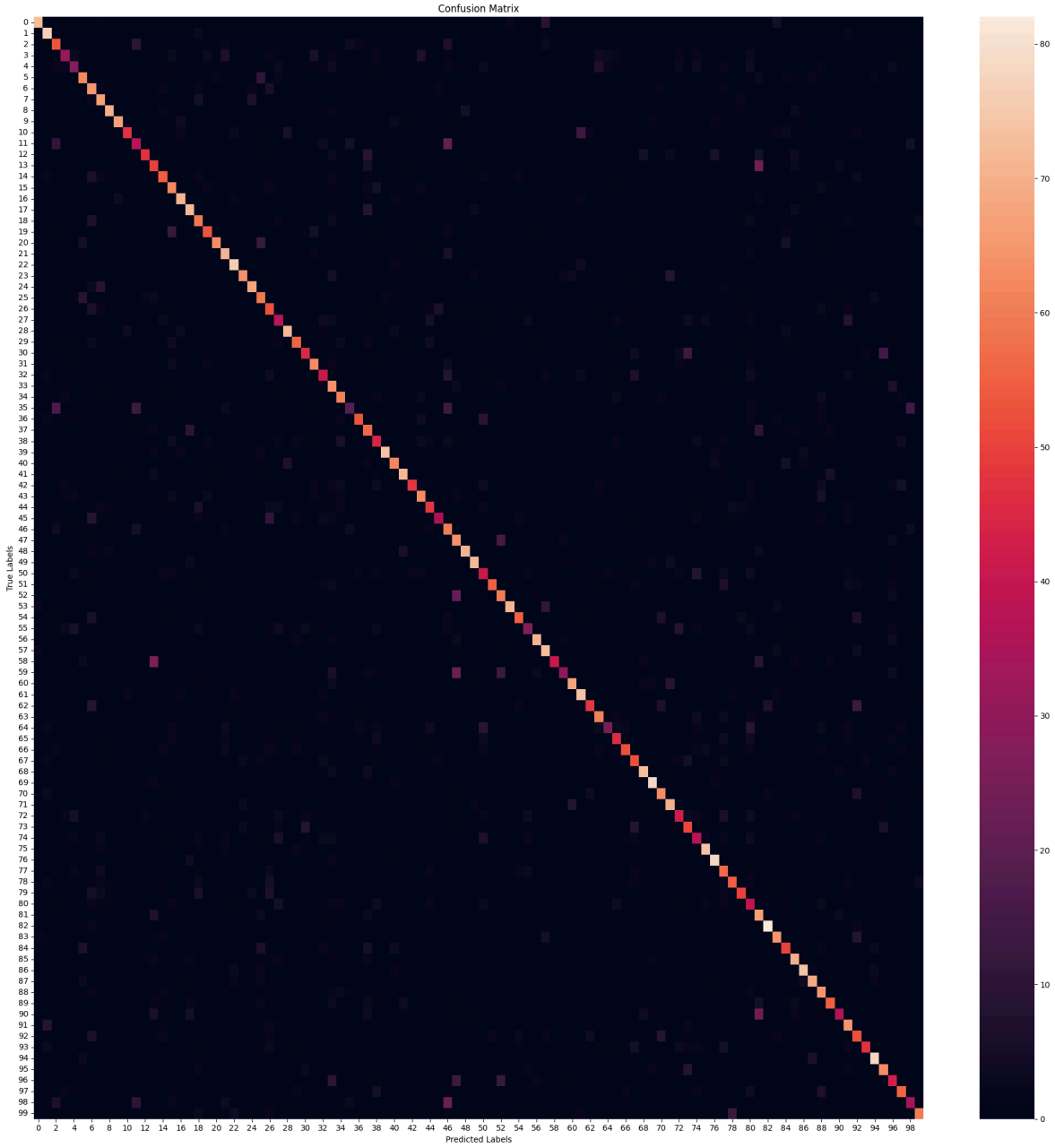
Trainable params: 1,100,388 (4.20 MB)

Non-trainable params: 23,564,800 (89.89 MB)

```
Epoch 1/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 58s 35ms/step - accuracy: 0.4497 - f1_score: 0.445
6 - loss: 2.1850 - precision: 0.7247 - val_accuracy: 0.6181 - val_f1_score: 0.616
1 - val_loss: 1.3472 - val_precision: 0.7966
Epoch 2/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 36s 28ms/step - accuracy: 0.6399 - f1_score: 0.637
7 - loss: 1.2613 - precision: 0.7955 - val_accuracy: 0.6264 - val_f1_score: 0.625
0 - val_loss: 1.3242 - val_precision: 0.7815
Epoch 3/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 36s 27ms/step - accuracy: 0.6941 - f1_score: 0.692
3 - loss: 1.0345 - precision: 0.8228 - val_accuracy: 0.6383 - val_f1_score: 0.637
0 - val_loss: 1.3295 - val_precision: 0.7724
Epoch 4/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 36s 27ms/step - accuracy: 0.7289 - f1_score: 0.727
3 - loss: 0.8890 - precision: 0.8386 - val_accuracy: 0.6406 - val_f1_score: 0.637
1 - val_loss: 1.3417 - val_precision: 0.7596
Epoch 5/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 36s 27ms/step - accuracy: 0.7606 - f1_score: 0.759
2 - loss: 0.7694 - precision: 0.8521 - val_accuracy: 0.6439 - val_f1_score: 0.642
0 - val_loss: 1.3703 - val_precision: 0.7636
Epoch 6/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 36s 27ms/step - accuracy: 0.7855 - f1_score: 0.784
3 - loss: 0.6767 - precision: 0.8659 - val_accuracy: 0.6361 - val_f1_score: 0.634
9 - val_loss: 1.4227 - val_precision: 0.7421
Epoch 7/25
1314/1314 ━━━━━━━━━━━━━━━━━━━━ 36s 27ms/step - accuracy: 0.8107 - f1_score: 0.809
6 - loss: 0.5982 - precision: 0.8736 - val_accuracy: 0.6348 - val_f1_score: 0.633
7 - val_loss: 1.5273 - val_precision: 0.7265
```



```
141/141 ━━━━━━━━━━━━━━━━━━━━ 3s 24ms/step - accuracy: 0.6283 - f1_score: 0.6206 -
loss: 1.3400 - precision: 0.7812
Test Accuracy: 63.64%
Test Loss: 130.68%
Test Precision: 78.80%
Test F1 Scores (Per Class): [84.883705 75.12195  54.081623 41.059597 35.294117 6
4.921455 55.793983
 65.346535 79.54544  81.21211  64.38356  43.428566 63.087242 49.504948
 60.773476 59.903378 76.756744 74.2268   57.281548 63.473045 78.75
 70.29703  80.85106  76.190475 78.16091  57.00482  48.59812  43.678154
 78.68852  67.46988  56.603767 70.786514 46.857132 52.459015 68.15642
 30.158726 68.38709  53.27102  48.087425 87.57396  69.66292  78.260864
 66.206894 68.1081   54.545456 49.999992 45.80152  53.33333  82.08092
 81.35593  43.850266 72.368416 61.224483 85.54216  66.26506  36.986294
 81.14285  72.99999  62.12121  44.61538  77.71428  74.747475 63.157898
 65.24063  39.682537 54.43787  65.4088   56.216217 80.21977  81.675385
 67.37968  72.53885  49.411755 55.91397  44.44444  77.89473  81.24999
 55.072456 65.868256 63.291134 37.73584  51.162785 88.64864  71.8232
 56.179775 70.05075  80.43478  77.52808  60.46511  66.66666  51.063824
 63.725483 54.450256 61.437904 83.87096  69.23077  49.42528  66.27218
 43.589737 74.07407 ]
Average Test F1 Scores:63.58

282/282 ━━━━━━━━━━━━━━━━━━━━ 12s 29ms/step
```


Confusion Matrix
True Labels
Predicted Labels

	precision	recall	f1-score	support
beaver	0.89	0.81	0.85	90
dolphin	0.67	0.86	0.75	90
otter	0.50	0.59	0.54	90
seal	0.51	0.34	0.41	90
whale	0.43	0.30	0.35	90
aquarium fish	0.61	0.69	0.65	90
flatfish	0.45	0.72	0.56	90
ray	0.59	0.73	0.65	90
shark	0.81	0.78	0.80	90
trout	0.89	0.74	0.81	90
orchids	0.84	0.52	0.64	90
poppies	0.45	0.42	0.43	90
roses	0.80	0.52	0.63	90
sunflowers	0.45	0.56	0.50	90
tulips	0.60	0.61	0.61	90
bottles	0.53	0.69	0.60	90
bowls	0.75	0.79	0.77	90
cans	0.69	0.80	0.74	90
cups	0.51	0.66	0.57	90
plates	0.69	0.59	0.63	90
apples	0.90	0.70	0.79	90
mushrooms	0.63	0.79	0.70	90
oranges	0.78	0.84	0.81	90
pears	0.82	0.71	0.76	90
sweet peppers	0.81	0.76	0.78	90
clock	0.50	0.66	0.57	90
computer keyboard	0.42	0.58	0.49	90
lamp	0.45	0.42	0.43	90
telephone	0.77	0.80	0.79	90
television	0.74	0.62	0.67	90
bed	0.65	0.50	0.57	90
chair	0.72	0.70	0.71	90
couch	0.48	0.46	0.47	90
table	0.42	0.71	0.52	90
wardrobe	0.69	0.68	0.68	90
bee	0.53	0.21	0.30	90
beetle	0.82	0.59	0.68	90
butterfly	0.46	0.63	0.53	90
caterpillar	0.47	0.49	0.48	90
cockroach	0.94	0.82	0.88	90
bear	0.70	0.69	0.70	90
leopard	0.77	0.80	0.78	90
lion	0.87	0.53	0.66	90
tiger	0.66	0.70	0.68	90
wolf	0.56	0.53	0.55	90
bridge	0.70	0.39	0.50	90
castle	0.35	0.67	0.46	90
house	0.43	0.71	0.53	90
road	0.86	0.79	0.82	90
skyscraper	0.83	0.80	0.81	90
cloud	0.42	0.46	0.44	90
forest	0.89	0.61	0.72	90
mountain	0.57	0.67	0.61	90
plain	0.93	0.79	0.86	90
sea	0.72	0.61	0.66	90
camel	0.48	0.30	0.37	90
cattle	0.84	0.79	0.81	90
chimpanzee	0.66	0.81	0.73	90

elephant	0.98	0.46	0.62	90
kangaroo	0.72	0.32	0.45	90
fox	0.80	0.76	0.78	90
porcupine	0.69	0.82	0.75	90
possum	0.77	0.53	0.63	90
raccoon	0.63	0.68	0.65	90
skunk	0.69	0.28	0.40	90
crab	0.58	0.51	0.54	90
lobster	0.75	0.58	0.65	90
snail	0.54	0.58	0.56	90
spider	0.79	0.81	0.80	90
worm	0.77	0.87	0.82	90
baby	0.65	0.70	0.67	90
boy	0.68	0.78	0.73	90
girl	0.53	0.47	0.49	90
man	0.54	0.57	0.55	90
woman	0.47	0.42	0.44	90
crocodile	0.75	0.82	0.78	90
dinosaur	0.76	0.87	0.81	90
lizard	0.49	0.63	0.55	90
snake	0.71	0.61	0.66	90
turtle	0.74	0.56	0.63	90
hamster	0.33	0.44	0.38	90
mouse	0.39	0.73	0.51	90
rabbit	0.86	0.91	0.89	90
shrew	0.71	0.72	0.72	90
squirrel	0.57	0.56	0.56	90
maple	0.65	0.78	0.71	90
oak	0.79	0.82	0.80	90
palm	0.78	0.77	0.78	90
pine	0.52	0.72	0.60	90
willow	0.73	0.61	0.67	90
bicycle	0.71	0.40	0.51	90
bus	0.57	0.72	0.64	90
motorcycle	0.51	0.58	0.54	90
pickup truck	0.75	0.52	0.61	90
train	0.81	0.87	0.84	90
lawn-mower	0.68	0.70	0.69	90
rocket	0.51	0.48	0.49	90
streetcar	0.71	0.62	0.66	90
tank	0.52	0.38	0.44	90
tractor	0.83	0.67	0.74	90
accuracy			0.64	9000
macro avg	0.66	0.64	0.64	9000
weighted avg	0.66	0.64	0.64	9000

Model: "sequential_7"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None, 7, 7, 2048)
global_average_pooling2d_7 (GlobalAveragePooling2D)	(None, 2048)
dense_14 (Dense)	(None, 512)
dropout_7 (Dropout)	(None, 512)
dense_15 (Dense)	(None, 100)

Total params: 24,665,188 (94.09 MB)

Trainable params: 24,619,748 (93.92 MB)

Non-trainable params: 45,440 (177.50 KB)

```
Epoch 1/15
 127/1971 ━━━━━━━━━━━━━━━━━━━━ 2:05 68ms/step - accuracy: 0.4701 - f1_score: 0.47
20 - loss: 2.0749 - precision: 0.7396
```

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
<ipython-input-10-e3908594f96b> in <cell line: 0>()
    311
    312     # Train the model
--> 313     history_fine_tune = model.fit(train_dataset_aug, validation_data=val_
dataset, epochs=15,
    314                                   batch_size=batch_size, callbacks=[early
_stopping], verbose=1)
    315

/usr/local/lib/python3.11/dist-packages/keras/src/utils/traceback_utils.py in err
or_handler(*args, **kwargs)
    115         filtered_tb = None
    116         try:
--> 117             return fn(*args, **kwargs)
    118         except Exception as e:
    119             filtered_tb = _process_traceback_frames(e.__traceback__)

/usr/local/lib/python3.11/dist-packages/keras/src/backend/tensorflow/trainer.py i
n fit(self, x, y, batch_size, epochs, verbose, callbacks, validation_split, valid
ation_data, shuffle, class_weight, sample_weight, initial_epoch, steps_per_epoch,
validation_steps, validation_batch_size, validation_freq)
    369                 for step, iterator in epoch_iterator:
    370                     callbacks.on_train_batch_begin(step)
--> 371                     logs = self.train_function(iterator)
    372                     callbacks.on_train_batch_end(step, logs)
    373                     if self.stop_training:

/usr/local/lib/python3.11/dist-packages/keras/src/backend/tensorflow/trainer.py i
n function(iterator)
    218             ):
    219                 opt_outputs = multi_step_on_iterator(iterator)
--> 220                 if not opt_outputs.has_value():
    221                     raise StopIteration
    222                 return opt_outputs.get_value()

/usr/local/lib/python3.11/dist-packages/tensorflow/python/data/ops/optional_ops.p
y in has_value(self, name)
    174   def has_value(self, name=None):
    175     with ops.colocate_with(self._variant_tensor):
--> 176       return gen_optional_ops.optional_has_value(
    177           self._variant_tensor, name=name
    178       )

/usr/local/lib/python3.11/dist-packages/tensorflow/python/ops/gen_optional_ops.py
in optional_has_value(optional, name)
    170   if tld.is_eager:
    171     try:
--> 172       _result = pywrap_tfe.TFE_Py_FastPathExecute(
    173         _ctx, "OptionalHasValue", name, optional)
    174       return _result

KeyboardInterrupt:
```